

江南大学

全日制专业学位硕士学位论文

题 目: 基于 FPGA 的超低延迟量化金融计算方法
研究与实现

英文并列题目: A Study of FPGA-based Ultra-low Latency
Quantitative Financial Computing Method

研 究 生: 丁楠 专 业 名 称: 计算机技术

研 究 方 向: 计算机应用与技术

导师①姓名: 张曦煌 职 称: 教授

导师②姓名: 周丹平 职 称: 高级工程师

学位授予日期: 2022 年 6 月

答辩委员会主席: 孙俊

江 南 大 学

地址: 无锡市蠡湖大道 1800 号

二〇二二年六月

摘要

量化金融是指通过数量化方式及计算机程序发出交易指令,以获取收益为目的的金融投资方式。在此背景下,越早获得行情数据进行计算并执行交易策略就越可能抓住最佳交易机会点。以证券的量化交易为例,行情数据从获取、传输、计算、到执行的整个链路,延迟越低越好。其中包含两个关键的阶段:一个阶段是从交易所获得行情数据,由于交易所通常会对行情数据进行 FAST 编码,接收方在获取数据时需要对其进行 FAST 解码;另一个阶段是从行情数据中解析出盘口数据。上面两个阶段当前面临的计算难题是:纯软件处理方式的延迟高,采用 FPGA 硬件处理方式则开发周期长、策略更新困难。

为了解决上述问题,本文提出了一种基于 OpenCL 和 HLS 的 FPGA 硬件解码模式,设计并实现了 FAST 协议的硬件解码器和行情数据的硬件解析器。基于上述工作,最终实现了一套面向股票量化交易的 FPGA 超低延迟量化金融计算系统,并在证券交易所的真实数据上进行了实验和验证。具体工作如下:

设计并实现了基于 FPGA 的 FAST 硬件解码器。对于 FAST 协议解码,基于 Vitis 平台,采用高层次综合语言和 OpenCL 对 FAST 数据进行解码加速,将协议分为 Pmap 值提取、字段域标记和分割、有效数据位合并、各字段域解码四个部分,对其进行流水处理。此外,针对不同实时数据类型进行分类并行处理,并通过数组切分存储映射、内核复制、存储优化等方式降低数据解码的延迟并提高 FPGA 硬件资源的利用率。最终实验结果测试单条 FAST 数据的解析延迟可以控制在 183ns 以内。

设计并实现了基于 FPGA 的行情数据硬件解析器。针对行情盘口快照解析阶段的硬件加速,通过逐笔委托和逐笔成交的交易逻辑推算出行情盘口快照。依靠交易时间相同,委托在前,成交在后的交易顺序,按照证券编号对数据进行分类,通过内核复制使得不同的证券并行处理,对存储单元进行优化。通过偏移量存储、自定义数据类型、二分查找优化等方式提高 FPGA 硬件存储资源的利用率并降低数据处理的延迟。最终实验结果测试单条行情盘口快照的解析延迟控制在 189.8ns。

实现了一套面向股票量化交易的 FPGA 超低延迟量化金融计算系统。将以上两部分计算模块结合构成量化金融计算加速模块,并与现有的网络协议卸载模块构成整个基于 FPGA 的量化金融计算加速系统,并通过 OpenCL 在 Xilinx Alveo U50 加速卡上实现,在同时处理 80 支股票的情况下,相比 CPU I9-9900X 10C20T,行情数据解析速度提升了 8 倍。同时相比传统的 HDL 硬件 FPGA 开发模式,开发效率提高了 3-4 倍,能够更好地适应金融市场的迭代需求。

关键词: 超低延迟; 高频量化交易; 面向流的金融信息交换协议; 开放运算语言; 现场可编程门阵列

Abstract

Quantitative finance refers to the financial investment method that issues trading orders through quantitative methods and computer programs for the purpose of obtaining income. In this context, the earlier you can obtain market data to calculate and execute your trading strategy, the more likely you are to capture the best trading opportunities. Take the quantitative trading of securities for example, the whole link of market data from acquisition, transmission, calculation and execution, the lower the delay, the better. It contains two key stages: one stage is to obtain market data from the exchange, because the exchange usually carries out FAST encoding of market data, the receiver needs to carry out FAST decoding when acquiring data; Another stage is from the market data parsing out data. The current computing problems of the above two stages are as follows: the delay of pure software processing is high, while the development cycle is long and the strategy update is difficult if FPGA hardware processing is adopted.

In order to solve the above problems, this paper proposes a FPGA hardware decoding mode based on OpenCL and HLS, designs and implements the hardware decoder of FAST protocol and the hardware parser of market data. Based on the above work, a set of FPGA ultra-low delay quantitative financial computing system for stock quantitative trading is finally implemented, and experiments and verification are carried out on real data of stock exchange. The specific work is as follows:

FAST hardware decoder based on FPGA is designed and implemented. For FAST protocol decoding, based on Vitis platform, high level comprehensive language and OpenCL are used to decode FAST data, and the protocol is divided into four parts: Pmap value extraction, field field marking and segmentation, effective data bit combination, and each field field decoding. In addition, different real-time data types are classified and parallel, and the delay of data decoding is reduced and the utilization of FPGA hardware resources is improved by array segmentation, storage mapping, kernel replication and storage optimization. The final experimental results show that the parsing delay of a single FAST data can be controlled within 183ns.

A hardware parser for market data based on FPGA is designed and implemented. In view of the hardware acceleration of the market price snapshot analysis stage, the market price snapshot is calculated by the transaction logic of consignment and transaction. Depending on the trading sequence of the same trading time, entrusting before and closing after, the data is classified according to the securities number, and different securities are processed in parallel through kernel replication, corresponding to the optimization of storage units. Through offset storage, custom data structure and binary search optimization, the utilization of FPGA hardware storage resources is improved and the delay of data processing is reduced. The final experimental results test the resolution delay of a single market snapshot in 189.8ns.

A set of FPGA ultra-low delay quantitative financial computing system for stock quantitative trading is realized. The above two computing modules are combined to form the quantitative financial computing acceleration module, and the whole quantitative financial computing acceleration system based on FPGA is formed together with the existing network

protocol unloading module, and is implemented on Xilinx Alveo U50 acceleration card through OpenCL. In the case of processing 80 stocks at the same time, Compared with CPU I9-9900X 10C20T, market data analysis speed increased by 8 times. At the same time, compared with the traditional HDL hardware FPGA development mode, the development efficiency is improved by 3-4 times, which can better adapt to the iterative needs of the financial market.

Keywords: Ultra-low latency; High-frequency quantitative trading; FAST protocol; OpenCL; FPGA

目 录

摘 要	I
Abstract	II
目 录	IV
第一章 绪论	1
1.1 课题研究背景及意义	1
1.2 国内外研究现状	2
1.3 本文主要工作	3
1.4 论文结构	4
第二章 量化金融计算中的数据协议	6
2.1 STEP-FIX 协议	6
2.2 FAST 协议	7
2.2.1 FAST 模板	8
2.2.2 停止位编码实体	9
2.2.3 域数据类型	9
2.2.4 存在位图	10
2.2.5 域操作符	10
2.3 本章小结	12
第三章 基于 Vitis 及 OpenCL 的 FPGA 开发	13
3.1 FPGA 介绍	13
3.2 FPGA 开发语言	14
3.2.1 硬件描述语言	14
3.2.2 高层次综合	14
3.3 Vitis 开发框架	15
3.4 基于 OpenCL 的 FPGA 开发	16
3.5 本章小结	17
第四章 基于 FPGA 的 FAST 硬件解码器设计	18
4.1 设计思路	18
4.1.1 预处理存在位图	19
4.1.2 字段标记与切分	20
4.1.3 处理单元内核复制	21
4.1.4 存储优化设计	23
4.1.5 硬件流水设计	23
4.2 系统实现方法	24
4.2.1 基于 OpenCL 和 HLS 异构系统实现	24
4.2.2 任意精度的数据类型优化	26
4.2.3 FAST 数据字段切分优化	26

4.2.4 解码各模块 Dataflow 流水优化.....	28
4.3 实验结果与分析	29
4.3.1 数据来源和环境模拟.....	29
4.3.2 平台和软硬件环境.....	29
4.3.3 主要资源占比.....	30
4.3.4 解码延迟抖动性对比.....	30
4.3.5 解码延迟与数据字段长度关系.....	31
4.3.6 FAST 解码延迟对比分析.....	31
4.4 本章小结	32
第五章 基于 FPGA 的行情数据硬件解析器设计.....	33
5.1 行情盘口快照解析	33
5.1.1 行情盘口快照和委托字典.....	33
5.1.2 逐笔委托和逐笔成交.....	33
5.1.3 行情信息解析.....	34
5.2 设计思路	36
5.2.1 基于 OpenCL 的主机端设计.....	36
5.2.2 硬件并行设计.....	37
5.2.3 委托字典存储优化设计.....	38
5.2.4 单号价格存储优化设计.....	39
5.3 系统实现方法	42
5.3.1 基于 OpenCL 和 HLS 异构系统实现	42
5.3.2 存储优化实现.....	43
5.3.3 委托成交处理内核复制并行实现.....	43
5.3.4 成交委托模块流水处理.....	45
5.3.5 二分查找硬件优化实现.....	45
5.4 实验结果与分析	47
5.4.1 实验平台环境和硬件资源消耗统计.....	47
5.4.2 行情解析延迟抖动性测试.....	47
5.4.3 行情解析延迟影响因素测试对比.....	48
5.4.4 行情解析延迟对比.....	48
5.5 本章小结	49
第六章 基于 FPGA 的量化金融计算加速系统.....	50
6.1 基于 FPGA 的量化金融计算加速系统软硬件实验环境.....	50
6.1.1 主机端测试平台.....	50
6.1.2 FPGA 端测试平台	50
6.1.3 系统测试对比方案.....	52
6.2 基于 FPGA 的量化金融计算加速系统设计与实现.....	52

6.2.1 量化金融加速系统总体框架设计与实现.....	52
6.2.2 基于 OpenCL 的量化金融加速系统主机端设计.....	53
6.2.3 基于 FPGA 的量化金融加速模块设计与实现	54
6.3 基于 FPGA 的金融计算加速系统实验结果与分析	55
6.3.1 实验数据来源和测试方法.....	55
6.3.2 量化金融计算加速系统延迟对比与分析.....	56
6.3.3 金融计算模块 FPGA 资源消耗和分析	56
6.3.4 金融计算模块结果延迟对比与分析.....	57
6.4 本章小结	58
第七章 总结与展望	59
7.1 课题研究总结	59
7.2 展望	59
致 谢	61
参考文献.....	62
附录：作者在攻读硕士期间发表的论文、成果和科研项目	67

第一章 绪论

1.1 课题研究背景及意义

量化金融是指通过数量化方式及计算机程序发出交易指令,以获取收益为目的的金融投资方式^[1,2]。2021年9月3日,中国第一家公司制证券交易所北京证券交易所正式注册成立,并在2021年11月15日上午正式开市,首批上市公司数目达到了81家,截止2022年1月底,北京证券交易所投资者数目超过480万户,是设立消息宣布前的2.8倍^[3]。这标志着我国金融证券交易从此迈向一个新的阶段。高频量化交易(High-frequency Quantitative Trading, HFQT)通过将计算机程序化交易和数学模型相结合,制定定制化的交易决策方案,实现高效、低廉的交易形式,受到了越来越多投资者的关注^[4,5]。可以说,其正逐渐成为主要金融交易方式之一^[6,7]。

在国际上,高频量化交易开始的契机是美国证券交易委员会在1998年授权通过了电子交易。因此,国外金融交易在量化高频交易市场起步较早,技术上也较为成熟,高频量化交易在市场总交易的占比已经超过了50%。与此同时,美国证监会的数据显示,美国在2010年的高频交易占比已经达到了70%以上,并且存在持续上升的趋势^[8-10]。在亚洲,东京证券交易所在2018年的高频交易占比也超过了30%,相较而言,国内的高频交易发展则显得较为迟缓。但是,随着信息技术的不断发展和高算力计算机的出现,目前国内高频交易的占比已超过了20%,成为了目前金融市场领域较为重要的交易方式之一^[11-13]。

目前,上交所证券成交量从2019年53792.1454亿股增长到2021年的70444.4027亿股,涨幅达到了30.96%^[14]。深交所证券成交量从2019年的72832.14亿股增长到2021年101130.45亿股,涨幅达到了38.85%^[15]。成交量的激增为高频量化交易提供了更大的发展前景。高频量化交易具有智能化、程序化的特点,在交易过程中,能够避免资产价格在短时间内的大幅度波动时,普通散户及中小投资者的追涨杀跌心态,进而跟风,呈现“羊群行为”的风险^[16]。利用计算机的高速算力,能够根据事先设计好的交易逻辑对市场的买入和卖出行为进行分析并作出相应的决策。尽管高频量化交易的单笔利润很低,但是通过大量且频繁地交易,其仍然能够获得稳定的收益。此外,针对部分异于正常交易价格的订单,高频量化交易也能够第一时间进行“捡漏”,从而获取额外收益。由于证券交易所交易的特点,越早接收到达的交易订单请求就能越早进行匹配成交。随着高频量化交易的不断发展,越来越多的投资者希望能够在最短的时间内完成对行情数据的分析和计算,并在第一时间发送订单请求从而实现获利的最大化。因此,寻求一种超低延迟的量化金融计算方法成为了目前金融交易领域的首要需求。

在金融证券领域,行情数据的解析计算获取尤为重要,越早获取行情信息就可以越精准地把握交易时机从而获得更高的收益。为了实现此目标,目前该领域存在三种主流的解决方案:其一采用高性能的CPU^[17-20]。高性能CPU的优势在于其灵活性较强,拥有开源的代码参考,开发周期短,在代码逻辑的更新迭代上拥有较大的优势。而缺点则是编译器和操作系统本身存在资源调度的问题以及解码过程中的时延不够稳定,解码延

迟约在 1ms 左右。在金融行情不断发展的背景下，延迟作为金融交易的首要指标，是投资者需要优先考虑的重要问题。由于纯软件的处理优化方案在延迟指标上逐渐接近瓶颈，因此逐渐被金融投资者所抛弃。为此，为了获取低延迟的处理方案，人们将目光投向第二种方案，即专用集成电路设计（Application Specific Integrated Circuit, ASIC）。专用集成电路设计的优点在于定制化的硬件能够达到更低的延迟和更稳定的解码流程，不过，一旦协议规则和交易决策发生变化，ASIC 一经设计便无法改变内部逻辑的缺点就显得尤为明显，因此，其也很难适应不断变化的金融交易市场。针对上述情况，投资者希望在低延迟的基础上，亦能够拥有较高的灵活性和拓展性。至此，第三种方案，即采用现场可编程门阵列（Field Programmable Gate Array, FPGA）的方案应运而生。由于 FPGA 并行流水以及运行时间可精确到周期的特点，能够实现量化金融计算过程中的低延迟、低抖动，同时 FPGA 可编程的特点也可以解决协议和决策的变化问题。但是，由于近年来金融交易算法的不断变化，交易决策也随之更新频繁。相较于传统的纯软件方案，传统的 FPGA 硬件开发模式仍存在开发周期长，协议和策略更新困难等问题。对此，本文提出了一种低延迟、高灵活性且开发周期短的量化金融计算加速方案，即基于 OpenCL 和 HLS 的 FPGA 量化金融计算加速方案。

1.2 国内外研究现状

随着高频量化交易在证券交易市场的占比不断提升，越来越多的企业投资者开始着手对金融交易计算领域进行深入研究。目前研究机构、高校、企业主要从金融协议解码、行情数据解析以及交易决策等多个方面展开加速优化研究。

首先是针对金融协议解码加速的研究。最初，FAST 协议解码采用基于纯软件的 C++ 编程语言的 QuickFast^[21]和 Mfast^[22]以及基于 Java 语言的 Openfast^[23]。但随着压缩率更高的 FAST1.2 协议的推出，现有的方法很难为其提供支持以及面向过程的开发运行环境。对此，徐广斌、武剑锋等人联合上海证券交易所和复旦大学计算机科学技术学院，提出了基于拓展 FAST 的金融压缩方法 FASTX。相比传统基于 FAST1.1 协议的 QuickFast 和 OpenFast 开源金融压缩方法，FASTX 拥有更好的编码效率和速度。在针对相同的数据编解码处理上，FASTX 的解码效率约是 OpenFast 的 8~10 倍^[24]。但随着证券交易市场的不断发展，基于 CPU 的软件解码优化始终无法适应激增庞大的交易数据量。

基于金融协议解码的特点以及 FPGA 的灵活性和并行设计架构，研究人员开始尝试采用 FPGA 对金融协议解码并进行硬件加速。上海交通大学李函等人基于 FPGA 提出了可配置的并行 FAST 解码结构，实现了并行字段分割，同时可以满足万兆以太网的数据吞吐量。针对影响解码延迟的因素为重复组的字段数和模板中的重复组个数。通过 SystemC 语言进行流水并行设计从而降低解码延迟，最终将解码延迟控制在 300ns 以内^[25]。随后，复旦大学环宇翔等人与上证所信息网络有限公司合作，针对上交所 Level-2 行情发布系统，设计了面向证券行情信息的超低延迟专用硬件解析系统。借助 Xilinx Alveo U200 加速卡，通过 Verilog 实现了低延迟以太网通信，支持 UDP/IP 协议的硬件解析以及 STEP-FAST 协议的数据流硬件解码。解码整体穿透延迟可低至 847ns^[26]。同年，国内国泰君安证券针对上海证券交易所 Level-2 行情解码转发服务，提出了基于 OpenCL

(Open Computing Language) 异构加速平台的 FPGA 硬件解码方案。最终将解码时延控制在了微秒级^[27]。

在上述金融交易协议计算的基础上, 研究人员希望更进一步从行情数据处理和交易决策方面对高频量化金融领域进行加速。其中, 行情数据处理主要涉及存储字典的优化。布尔诺理工大学 Milan 等人基于 FPGA 创建了一套低延迟的交易系统, 并在存储字典的处理上采用了布谷鸟散列, 实现了低延迟的符号查找和高内存利用率。通过对查找延迟和内存利用率进行权衡, 最终实现了平均 253ns 的处理延迟^[28]。无独有偶, 印度塔塔咨询服务公司基于 HLS 语言在 FPGA 上实现了一种低延迟、高数据吞吐量、高内存利用率的键值存储 (Key-value Store, KVS) 模块。与传统的 RTL 语言开发相比, 在处理延迟和资源利用相似的前提下, 开发时间缩短至原来的 18%^[29]。此外, 德国海德堡大学 Christian Leber 等人, 基于 FPGA 针对高频交易应用开发了带有相应指令集的微码引擎和编码器。相较于传统的软件设计, 基于 RTL 代码的实现使延迟减少了 4 倍^[30]。除了行情数据处理, 交易的决策也至关重要。日本筑波大学 Shigeki Kohda 等人针对东京证券交易所的数据进行了研究, 通过订单买卖进出的特征以及价格与买卖价格的潜在关联性, 设计了一种交易策略, 可以获得近 90% 的可靠性盈利。

综合上述金融交易领域的加速方向, 研究人员和企业希望能够构建一个基础通用的金融计算框架。加拿大多伦多大学的 Andrew Boutros 等人基于 FPGA 对网络堆栈、金融协议解析、订单处理以及自定义交易算法进行了加速, 提出了一个完整的高度优化的基础金融计算框架, 使得上层开发人员能够快速利用高级编程语言实现交易决策算法。测试系统的通信往返延迟最低可低至 870ns^[31]。此外, 部分企业也推出了各自的金融加速方案。法国巴黎一家为金融、电信和高性能计算行业提供基于 FPGA 技术和解决方案的提供商 Enyx 提供了一套名为 nxFramework 的产品框架, 框架中提供了协议 IP 内核以及一个附属包, 包中含六十多个 IP 的库。这些库通过数据流处理, 内存映射的 I/O 管理, 哈希表, 缓存, FIFO 以及数据函数等低延迟的 FPGA 优化方式从而满足标准化要求。

除了应用于证券行情外, FPGA 在金融计算加速领域的研究也正在向期货、外汇等其他领域延伸。郑州大学王忠勇教授团队提出了一种基于 FPGA 的低延迟传输方案, 方案思路为将 TCP/IP 协议栈处理和加密运算的过程在 FPGA 上实现, 最终实现系统总延迟达到 1.1 μ s。随后, 该团队在此基础上与郑州商品交易所展开合作, 提出了一种基于 FPGA 的期货行情处理系统, 该行情处理系统通过优化, 其速度达到了每秒处理 60 万笔消息^[32,33]。

综上所述, FPGA 是有效的加速手段, 但仍存在开发困难等问题, 因此, 本文采用了基于 OpenCL 和 HLS 的方案。通过 HLS 并行、流水的指令级优化以及 OpenCL 高层次的底层抽象设计开发模式, 能够解决传统 FPGA 硬件开发周期长的问题。该方案有望在利用 FPGA 优势的同时提升开发效率。

1.3 本文主要工作

本文根据金融交易市场证券交易中对金融计算方法的实际需求, 依据股票行情数据 FAST 协议解码以及行情盘口快照数据解析两部分, 研究并设计了一种基于 FPGA 的超

低延迟金融计算加速方法。该设计实现了 FAST 协议解码过程中和行情盘口快照数据解析的硬件加速功能。完成了对金融行情数据的接收、解码以及解码完成后的行情盘口快照解析。本文主要工作如下：

第一，基于 FPGA 的 FAST 硬件解码器的设计与实现。在 Vitis 开发平台，基于 OpenCL 和 HLS 对 FAST 数据进行解码加速，将解码协议分为四部分进行流水处理，分别是 Pmap 值提取、字段域标记和分割、有效数据位合并以及各字段域解码。针对不同实时数据类型进行分类并行，通过数组切分存储映射、内核复制等方式实现 FAST 数据解码的低延迟，同时，采用自定义数据类型进行存储优化，从而提高 FPGA 硬件资源的利用率。最终测试单条数据的解码延迟。

第二，基于 FPGA 的行情数据硬件解析器的设计与实现。针对行情盘口快照解析阶段的硬件加速需求，通过逐笔委托和逐笔成交的交易逻辑推算出行情盘口快照。根据证券编号对逐笔数据进行分类，处理顺序遵循时间相同，委托在前，成交在后的原则。通过内核复制使不同的证券并行处理，从而降低数据处理的延迟。此外，在降低延迟的基础上，通过偏移量存储、自定义数据类型、二分查找优化等方式提高 FPGA 硬件存储资源的利用率。最终测试每条行情盘口快照的解析延迟。

第三，一种面向股票量化交易的 FPGA 超低延迟量化金融计算系统实现。结合上述两部分计算模块解析器设计构成量化金融计算加速模块，并与现有的网络协议卸载模块构成整个基于 FPGA 的量化金融计算加速系统，并在证券交易所真实数据上，基于 OpenCL 和 HLS 在 Xilinx Alveo U50 加速卡上进行实现和验证。最终与 CPU 进行对比，测试两者同时处理多条证券的总体延迟。

1.4 论文结构

本文章节安排如下：

第一章：绪论。本章首先对高频量化交易的研究背景进行介绍。分析金融市场对量化金融计算的需求，以及量化金融计算未来发展的趋势。其次对近年来国内外基于 FPGA 的量化金融计算方法的研究现状进行回顾。通过对现有解决方案的分析，阐明 FPGA 在量化金融计算方法方面具有的可行性和优势。最后介绍本文的主要工作以及论文的整体结构。

第二章：量化金融计算中的数据协议。本章首先介绍了国际金融证券交易交换协议中的 FIX (Financial Information Exchange) 协议以及国内基于 FIX 协议提出的适应国内金融信息行业的 STEP (Securities Trading Exchange) 协议的特征和协议格式。在此基础上引出根据行情数据特征进行数据压缩采用的 FAST (FIX Adapted for Streaming) 协议，具体展开介绍了 FAST 协议的格式和压缩方法。其中包含压缩的流程、FAST 模板、存在位图、FAST 操作符、数据类型以及停止位规则。

第三章：基于 Vitis 及 OpenCL 的 FPGA 开发。本章首先主要介绍了 FPGA 的基本概念和构成。其次阐述了目前主流的 FPGA 开发语言 HDL (Hardware Description Language) 和 HLS (High Level Synthesis)，其中着重介绍 HLS 的优势和开发流程。接

着介绍了实验采用的 Vitis 开发平台以及开发的基本流程。最后介绍了异构开发平台 OpenCL，并从四个模型出发对其进行了完整的阐述说明。

第四章：基于 FPGA 的 FAST 硬件解码器设计。本章首先介绍了 FAST 协议解码的设计思路，将 FAST 数据解码过程分为 Pmap 值提取，数据标记切分，有效数据位合并以及字段域解码四个部分，从而实现不同数据之间的流水化设计。通过数组分割映射，处理单元内核复制实现不同字段的并行解码。此外，针对 FPGA 内部的硬件存储资源进行优化，采用定制化的数据类型设计，在正确表示数据的情况下，提高内存的资源利用率。其次介绍实现的方式，主要是采用 OpenCL+HLS 的开发框架。其中主机端通过 OpenCL 完成数据的发送、接收以及内存的申请、分配。FPGA 端通过 HLS 优化指令完成单条数据的字段域并行解码以及不同数据之间的流水处理。最终进行实验仿真、上板，并与 CPU 进行对比测试，分析单条数据的解码延迟和抖动性测试以及资源利用情况。

第五章：基于 FPGA 的行情数据硬件解析器设计。本章首先介绍了介绍行情数据解析过程中所需要的计算数据，主要包括对逐笔委托和逐笔成交以及最终计算生成的委托字典与行情盘口快照的关系的说明，最后阐述了整个盘口推演的逻辑过程。然后介绍了基于 OpenCL 的主机端设计，考虑到键值表的资源存储，需要分别对片上和片外的内存资源进行申请和分配。其次介绍了逐笔委托和逐笔成交的单元处理模块以及总体并行度的框架设计。接着分别对买卖委托字典的优化情况进行说明，即通过键值表和存储地址偏移量提高存储利用率，包括对委托单号价格键值表进行存储优化，对键值表二分查找算法进行了优化。再然后对实现的总体架构进行介绍，阐述在加速卡上对存储资源的具体分配和实现情况，以及通过 HLS 实现委托、成交内核处理的并行，成交子模块的流水以及二分查找的优化。最终进行实验仿真、上板，与 CPU 比较测试结果。主要针对单条行情数据的解析延迟和抖动性以及行情解析延迟的影响因素进行分析和说明。

第六章：基于 FPGA 的量化金融计算加速系统。本章在前两章计算加速的基础上，设计出了基于 FPGA 的量化金融计算加速模块，并与目前现有的基于 FPGA 的网络协议卸载模块相结合，构成了整个 FPGA 的金融计算加速系统。此外，在介绍软硬件开发平台和实验采用的 Xilinx Alveo U50 加速卡的参数和架构的过程中，借助 OpenCL 异构开发语言，通过核心的设计代码与 Vitis 平台相结合，测试量化金融加速模块和网络协议卸载模块的延迟占比。最后，对内存的读写速率、多支证券并行处理的延迟、FPGA 内部的资源占用情况进行测试，并与 CPU 进行对比分析。

第七章：总结与展望。一方面总结子模块的设计和总体系统设计的优势和不足，同时，针对不足点提出相应的解决措施。另一方面，对未来可能的研究方向提供展望。

第二章 量化金融计算中的数据协议

本章将根据证券交易数据在传输中涉及的两个阶段，阐述在金融计算中需要遵循的数据协议和解析逻辑。其中涉及的数据协议有证券交易数据交换协议（Securities Trading Exchange Protocol, STEP）、金融信息交换协议（Financial Information Exchange, FIX）以及面向流的金融信息交换协议（FIX Adapted for Streaming, FAST）。

2.1 STEP-FIX 协议

FIX 协议全称金融信息交换协议，目前主要作为公开协议应用于证券交易的数据传输和交换过程中。FIX 协议规定了每一条证券交易信息的内容和格式，这些交易信息内容在一定程度上反映了证券交易的流程，以确保交易信息能够安全准确地送至行情商和金融投资者手中。FIX 主要用于在证券行情商、中小型投资者、个体散户等各类金融交易参与者之间建立起实时的电子化通讯协议，从结构上主要分为会话层和应用层两个层次。其中会话层主要涉及数据通信等连接，而应用层则传输的是金融交易相关数据内容。FIX 协议的目的是将如证券、期货、外汇等各类证券金融业务需求进行统一并按照流程格式化，最终能够通过计算机语言对功能流程进行描述，并统一每个业务功能接口上的交换格式，以便各个功能模块之间能够相互连接，从而提高整个金融行情的信息交换效率^[34-36]。

中国自 2001 年底加入世界贸易组织（World Trade Organization, WTO）后，为了与国际金融业务接轨，中国证监会根据我国当时证券市场的情况，结合国际标准的 FIX4.4 协议，推出了一种适合中国国情的证券交易数据交换协议，简称为 STEP 协议。之后其成为了中国金融行业的标准协议，并开启了我国证券交易技术一个新的里程碑。STEP 协议由于吸收了国际 FIX 协议的优势，不仅能将证券交易所需要的操作指令和通知以标准化的格式表达出来，还能完全与国际 FIX 协议兼容。因此，它的推出和实施应用使得我国证券行业逐步走向标准化与国际化，为我国金融证券市场投资者之间计算机化通讯提供了标准语言，使我国在国际证券市场具备了一定的竞争力^[37-39]。

STEP 协议主要从会话机制、应用环境、数据完整性、拓展方式、消息格式、消息定义、安全加密、数据字典等多个方面对数据内容进行定义。其编码方式采用标准 ASCII 码编码，是一种含标签（Tag）且无序可变长的金融数据交换协议。消息根据不同属性类型通过<SOH>被分为多个域（Field），域中的数据结构类型为“Tag=Value”。STEP 消息从结构上分为三个部分，分别是消息头、消息体和消息尾。STEP 消息总体结构如图 2-1 所示。



图 2-1 STEP 消息结构示意图

其中，整个消息的起始域集合被称为消息头，主要包含域为起始串（通常域内为8=STEP.1.0.0）、消息体数据长度（Tag=9）、消息所属类型（Tag=35）等字段域，且这三个域的位置固定，为整个消息头的前三个域。消息体主要是正文信息，接在消息头之后。消息尾则作为整个消息的结尾，其最后一个域被称作校验和域（Tag=10）。STEP 格式消息示例如下：

STEP 消息示例：

```
8=STEP.1.0.0<SOH>9=125<SOH>35=UA3201<SOH>49=VDE<SOH>56=VSS<SOH>34=264<SOH>52
=20210305-10:02:24<SOH>347=GBK<SOH>10142=7<SOH>10072=264<SOH>95=37<SOH>96=.....
U..60015..7P..}.=.....H.....`..<SOH>10=125<SOH>
```

2.2 FAST 协议

FAST 是 2005 年全球金融企业联盟组织 FPL 提出的一种面向消息流的压缩、解码和传输方式^[40]。在编码方式上，FAST 协议从两个层面降低数据传输中消息流的大小。一方面主要是在消息流中通过域操作符对先后的数据字段域进行增量传输，来压缩原本需要完整传输的数据内容，最终减少冗余数据导致的传输损耗。另一方面，对于不同类型的二进制字段，采用可自描述的域长度进行高效的二进制编码，从而提高数据压缩率，降低传输数据的大小。FAST 协议主要应用在高密度、低延迟的数据传输领域。发送和接收方希望能够在传输带宽一定的情况下，减少不必要的信息传输，从而获取更多的数据信息。

FAST 提出的初衷是为弥补 FIX 协议数据传输中存在的不足，并对其进行进一步优化。其本质是将每一个字段的标签从数据流中除去，仅传输 Value 数据。将字段与字段合并成一整条数据字段组，称之为一个消息。FIX 协议与 FAST 协议数据格式如图 2-2 所示。

Tag1=Value1	Tag2=Value2	Tag3=Value3		TagN=ValueN
-------------	-------------	-------------	-------	-------------

（a）FIX 协议数据格式

Pmap	Template ID	Value1		ValueN
------	-------------	--------	-------	--------

（b）FAST 协议数据格式

图 2-2 FIX 协议与 FAST 协议数据格式

其中，图中的 Pmap 值通常作为 FAST 数据第一个传输的字段域，为后续的字段域分割和域操作符解码提供依据。Template ID 通常作为第二个传输字段域来对应该字段所匹配的模板号。

目前交易所采用的是将实时的行情数据通过 FAST 协议编码加密压缩成二进制数据，再将数据嵌套在 STEP 消息标签为 96 的字段域内，同时将对应的数据长度以 STEP 协议的形式存放在标签为 95 的字段域内。FAST 数据 Tag96 嵌套 STEP 消息格式如图 2-3 所示。

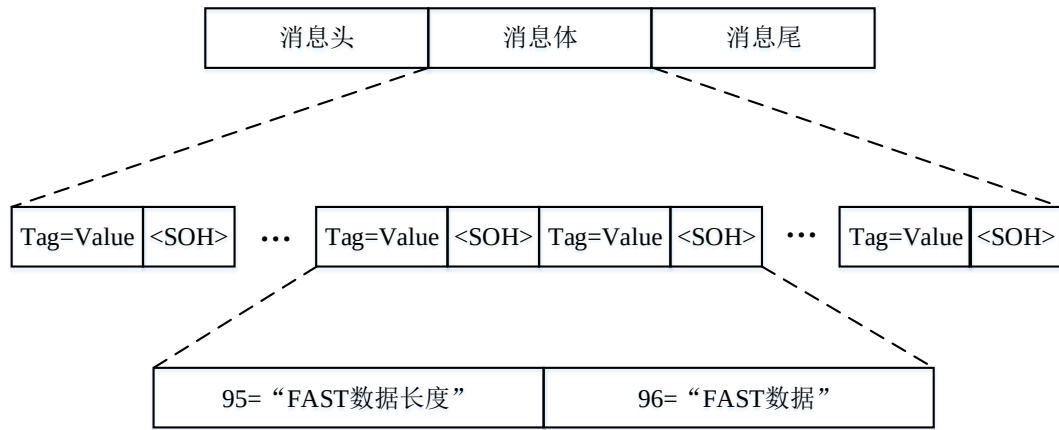


图 2-3 FAST 数据 Tag96 嵌套 STEP 消息格式

FAST 协议主要从五个方面对传输数据进行压缩加密，分别是 FAST 模板（FAST Template）、停止位编码实体（Stop bit encoded entity）、域数据类型、存在位图（Presence Map, Pmap）、域操作符（Field operator）。

2.2.1 FAST 模板

FAST 数据传输过程是从交易所至行情商。交易所和行情商之间事先约定好 FAST 模板，在数据传输过程中，交易所先将数据按照 FAST 协议的压缩格式进行压缩加密，接着将加密压缩后的数据传输给行情商，行情商在接收到加密的数据后，对数据进行 FAST 解压解密，从而获取原始数据。具体传输流程如图 2-4 所示。

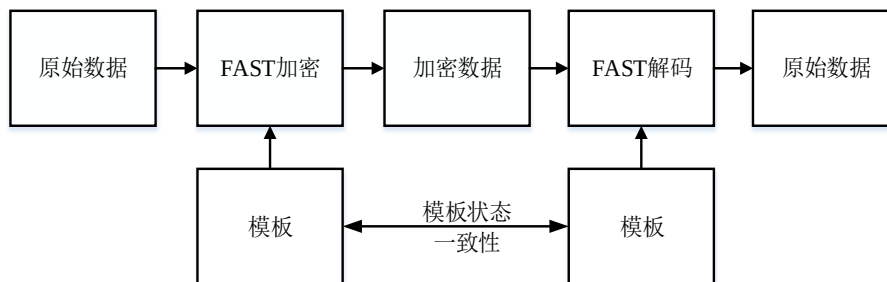


图 2-4 FAST 消息传输过程

模板的存在使得在传输总数据量不变的情况下，通过减少传输通用标签含义信息的数据，来提高有效数据信息的条数，同时模板在字段语法上引入一种标准化、正规化的格式，能够使计算机和金融证券参与者读懂其中的含义。模板通过规定域的前后位置顺序和具体表达结构，字段域对应操作符以及其采用的二进制编码表示方法来完成对消息流的部分编码。通常 FAST 协议的模板以 XML 文件的形式存放。XML 代码部分代码如下所示。

XML 部分模板示例：

```
</template>
<template name="NGTSTransaction" id="3201">
  <string name="MessageType" id="35"><constant value="UA3201"/></string>
  <int32 name="DataStatus" id="10121" presence="optional"><default/></int32>
  <int32 name="TradeIndex" id="10011"><increment/></int32>
  <int32 name="TradeChannel" id="10115"><copy/></int32>
```

```

<string name="SecurityID" id="48" presence="optional"><copy/></string>
<int32 name="TradeTime" id="10013" presence="optional"><copy/></int32>
<int32 name="TradePrice" id="10014" presence="optional" decimalPlaces="3"><default/></int32>
<int64 name="TradeQty" id="10015" presence="optional" decimalPlaces="3"><default/></int64>
<int64 name="TradeMoney" id="10016" presence="optional" decimalPlaces="5"><default/></int64>
<int64 name="TradeBuyNo" id="10179" presence="optional"><default/></int64>
<int64 name="TradeSellNo" id="10180" presence="optional"><default/></int64>
<string name="TradeBSFlag" id="10192" presence="optional"><default/></string>
</template>

```

以上代码以逐笔委托 UA3201 为例,模板的内容主要是传输数据的编号、数据类型、各个数据字段域的标签含义、数据字段采用的加解密域操作符、小数点的精度等标签信息。

2.2.2 停止位编码实体

停止位编码实体作为 FAST 传输编码的一个重要的特征,其本质是一个字节序列。其中每个字节的首位为最高有效位 (the most significant bit),其作用是判断下一个字节是否需要与上一个字节合并从而构成一个整体。如果该位最高有效位为 0,则表示需要进行合并。反之如果最高有效位为 1,则表示该字节为字段域的最后一个字节,不需要合并。每一个字节跟在停止位后的 7 个位元为有效数据位 (significant data bits)。通过将每一个字节的有效数据位按照最高有效位判断进行拼接,最终构成停止位编码实体的实体值。例如接收到的 FAST 二进制数据流如下:

01100111 11100111 11000111 11100100

按照上述规则,四个字节中的三个字节最高有效位为 1,因此这段二进制数据流最终被划分为三个域。其中第一个字节最高有效位为 0,因此它需要与后一个字节拼接成新的实体值。需要注意的是,在划分和拼接过程中,最高有效位仅作判断依据,并无实际数值含义,因此需要去除。通过解析最终三个停止位编码实体的实体值如下:

11001111100111 1000111 1100100

根据上述实体编码的特征可以看出,实体值的位元总个数最小长度为 7 位,且长度变化始终为 7 的整数倍。

2.2.3 域数据类型

字段域按照数据类型分为整数型、ASCII 码字符串型、Unicode 字符串型以及字节向量型四种类型。通过证券交易协议模板可知,域数据类型主要涉及的类型为整数型和 ASCII 码字符串型。整数由停止位编码实体表示。在字节和位元的顺序上,所有整数域采用大尾端 (big-endian) 的表示方式,其中位元和字节均采用网络字节顺序。对于单个字节来说,高位在前,低位在后,对于多个字节来说,高字节在前,低字节在后。本质上,实体值为对应域整数的二进制表示形式。而 ASCII 码字符串型与整数型一致,同样由一个停止位编码实体来表示。但是在解译时,实体值被作为 7 位的 ASCII 码字符序列。通常,ASCII 码字符串型被用在表示六位 ASCII 证券代码上。

2.2.4 存在位图

存在位图作为位元的一个序列，其表示 FAST 数据中的字段域，并根据当前模板中的规定来使用位元。存在位图采用停止位编码实体来表示。在使用规则上，按照模板中字段域的位置顺序来分配存在位图位元，即模板中先出现的指令比后出现的按照规则优先分配位元。位元分配作用域仅在当前数据字段的存在位图中。此外，针对特殊的域操作符可能存在不占用位元的情况，是否占用位元由域操作符和域是否为必要还是可选来共同决定。例如，一个域如果为必要的、且对应操作符为常量，则其在存在位图中不占用 Pmap 值位元。反之，如果常量操作符的域为可选，则该域需要占用存在位图位元。对于较为特殊的无操作符，无论其域是否可选，在存在位图中其均不占用 Pmap 值位元，且其值必须在数据流中显示。具体不同域操作符使用存在位的规则总结如下表所示。

表 2-1 不同域操作符使用存在位规则表

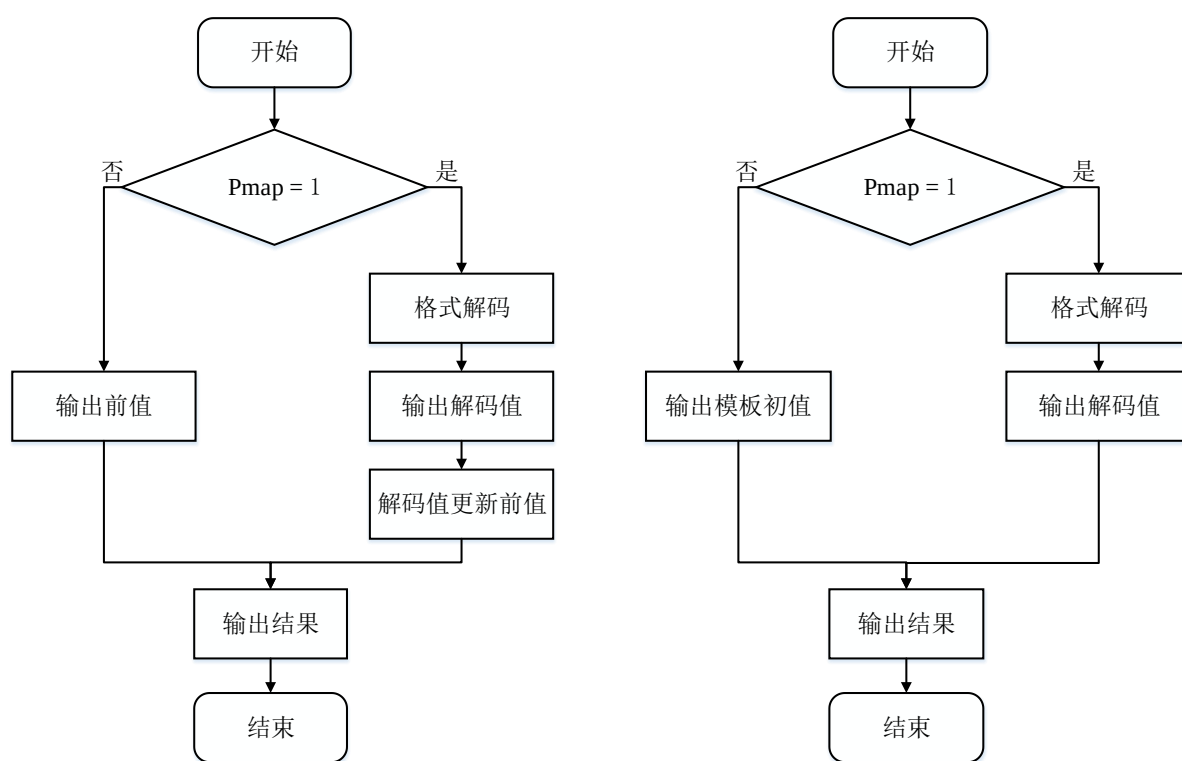
操作符		无	常量	复制	缺省	差值	自增	换尾
是否占用位元	必要	否	否	是	是	否	是	是
	可选	否	是	是	是	否	是	是

2.2.5 域操作符

域操作符规定了数据各字段域编码的解码方式。其中一些域操作符的逻辑依赖前值和初值。每一个需要依赖前值的域相互独立，它们所构成域的前值集合被称为字典。每一个域的前值都有其各自的名称及特定类型的值。通常在处理开始时，前值均属于未定义状态，通过第一轮数据更新赋予相应的数值。初值通常指模板中的初值，是在模板中事先设定好的。

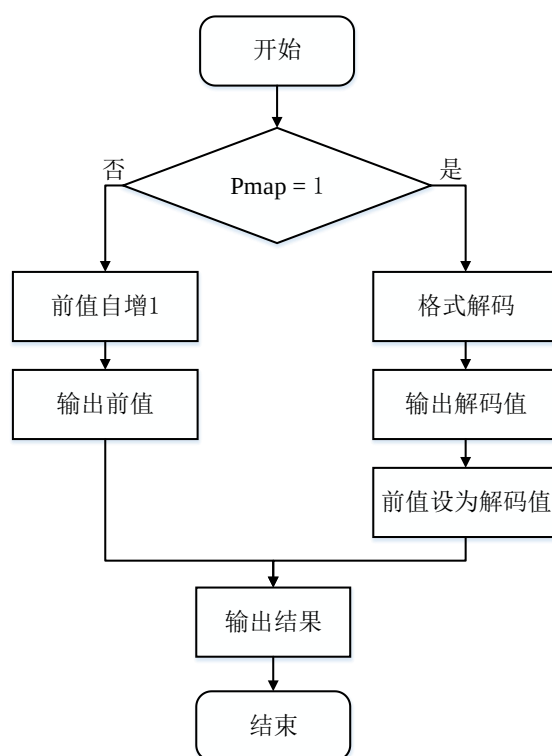
域操作符从类别上被分为七大类，分别是无操作符（No Operator）、常值操作符（Constant Operator）、缺省操作符（Default Operator）、复制操作符（Copy Operator）、自增操作符（Increment Operator）、差值操作符（Delta Operator）、换尾操作符（Tail Operator）。交易所的数据传输过程中，根据模板的需要，主要涉及的操作符有无操作符、常值、复制、缺省以及自增操作符。其中无操作符表示该字段域中没有特定的操作符，仅通过字段域对应数据类型和停止位编码规则解码。无操作符不占用存在位图位元，因为数据始终存在数据流中。常值操作符规定字段域表示的值相同，为模板中的初值。与无操作符一样，常值操作符也不占用存在位图的位元。复制操作符规定域值在数据流中是可选存在的。如果值在数据流中存在，即对应 PMAP 值位元为 1，则表示流中数据为停止位编码实体的实体值，同时它将成为新的前值。反之，若不存在，即对应 PMAP 值位元为 0，则表示对应域输出的值为前值。缺省操作符规定域的值在数据流中出现时，即对应 PMAP 值为 1，流中数据为停止位编码实体的实体值。反之如果 PMAP 值为 0，则直接输出对应的初值。自增操作符同样规定域的值在数据流中是可选存在的。如果值在数据流中存在，即对应 PMAP 值为 1 时，直接输出实体值，同时其成为新的前值。如果值在数据流中不存在，即 PMAP 值为 0 时，域值将在前值的基础上自增 1 作为实体值。

输出。同时，输出的值将作为新的前值。自增操作符通常应用在连续的单号做增量传输上。常用 FAST 域操作符流程图如图 2-5 所示。



(a) 复制操作符流程图

(b) 缺省操作符流程图



(c) 自增操作符流程图

图 2-5 常用 FAST 操作符流程图

2.3 本章小结

本章主要从两个方面介绍了金融计算中的数据协议。一方面是对行情商接收到交易所的裸行情数据进行 FAST 协议解码。另一方面则是金融投资者通过对行情商发送的逐笔委托和成交数据对行情盘口快照的推演和还原。此外，针对 FAST 协议，详细阐述了其发展过程，包括其前身 FIX 协议以及国内的 STEP 协议。同时，针对 FAST 协议压缩方式的流程，从 FAST 模板、传输流程、存在位图、FAST 操作符、停止位、数据类型等多个方面对其进行了介绍。

第三章 基于 Vitis 及 OpenCL 的 FPGA 开发

3.1 FPGA 介绍

FPGA 作为一个可以通过编程来改变内部结构的数字集成电路,本质上是一个芯片。不同于传统的专用集成电路,即一旦被设计好,其内部结构已经定型,无法改变。FPGA 可编程的特点,为设计的升级和拓展的灵活性提供了极大的便利。同时其异于传统 CPU 串行执行的特点,优势在于通过编程优化能够实现并行、流水计算等特性。同时由于没有了操作系统的资源调度和中断问题, FPGA 处理数据的延迟也相对稳定^[41-47]。

FPGA 芯片的基本架构主要由 6 部分构成,如图 3-1 所示,分别为可编程输入输出单元 (IOB)、基本可编程逻辑单元 (CLB)、嵌入式块 RAM、内部连接线 (Interconnect)、内嵌的底层功能单元以及内嵌专用硬件模块。

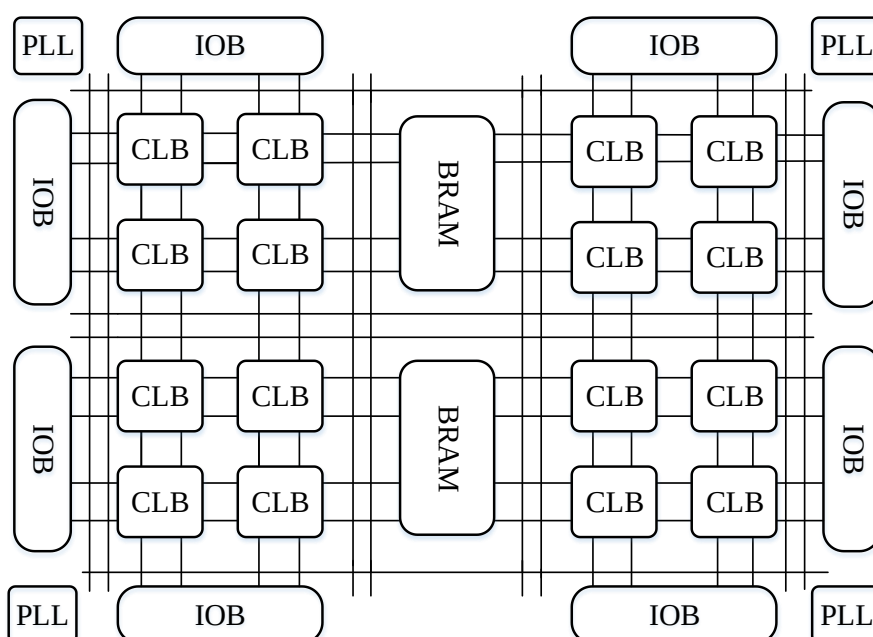


图 3-1 FPGA 芯片内部结构

可编程输入输出单元作为芯片的接口与外部电路进行交互,可利用软件编程完成对内部逻辑的配置,能够适配不同标准的电器设备以及其 I/O 物理特性。此外,还能够针对匹配的阻抗特性、上下拉电阻以及输出驱动电流的大小作出相应的调整。

基本可编程逻辑单元是由两部分构成,分别为查找表 (LUT) 和寄存器 (Register)。其中,查找表能够实现纯组合逻辑的功能,而寄存器可以配置成触发器,完成同步/异步复位、置位、时钟使能等功能。

嵌入式块 RAM 可配置成单端口 RAM、双端口 RAM、内容地址存储器 (CAM) 以及 FIFO 等常用存储结构。

内部连接线用以连通 FPGA 内部中的各个单元,连线的长度和工艺决定了连接线上信号的驱动能力和传输速度。根据制作工艺、尺寸大小和位置布局划分为四类,分别为全局性的专用布线资源、长线资源、短线资源和其他连接线。

内嵌的底层功能单元模块主要指 DLL (Delay Locked Loop)、PLL (Phase Locked Loop)、DSP 和 CPU 等软处理核 (Soft Core)。而内嵌专用硬核与其不同,在通用性上较弱,且并非必备。

3.2 FPGA 开发语言

FPGA 目前主流的开发语言为硬件描述语言 (Hardware Description Language, HDL) 和高层次综合语言 (High Level Synthesis, HLS)。

3.2.1 硬件描述语言

硬件描述语言作为普遍应用在硬件设计中的语言,主流采用的是 VHDL 和 Verilog 语言。两者都是通过文本的形式来描述硬件的结构和逻辑并实现对应的功能。其中 Verilog 更偏向于传统的 C 语言,常用 `always` 作为进程,而 VHDL 常用 `process` 作为进程。由于硬件描述语言可以细化到逻辑门级电路设计,因此其在层级优化上会更为精细。不过其缺点也很明显,即需要精细地设计逻辑关系,开发时间通常较长,且代码的维护和功能的更新成本也较大^[48-50]。

3.2.2 高层次综合

HLS 作为一种高层次的开发语言,可以将如 C、C++ 等高级语言结合指令级优化自动转化成抽象级硬件描述语言如 Verilog、VHDL 等。相较于传统的硬件描述语言, HLS 虽然失去了小部分的开发优化精度,但是其在开发效率上有了很大的提升。同时,开发人员软件、算法工程师无需过分了解底层硬件知识,可以更加关注程序本身功能逻辑的设计和算法实现^[51-57]。

编程语言上,开发者通过 C/C++ 或者 SystemC 完成 C 语言规格的程序编写。HLS 在开发流程上大致分为行为描述、行为级仿真、行为综合以及分析评测四个部分。其中,行为描述指用 C 语言规格的高级语言对需要实现的工程和具体的功能逻辑进行描述。与传统的编程不同,行为描述最终的目的是实现硬件电路,因此其在设计上需要确保尽可能地固定,且需要禁止使用指针、动态数组等不确定的功能。此外,针对循环变量,行为描述也需要尽可能估算出循环边界和循环次数,从而方便后期优化。行为仿真则是对 C 语言规格的高级语言设计的模块进行编译和验证。相较于 RTL 仿真,其可以无需以时钟为单位,从而导致更快的仿真速度,但是在精度控制上可能会存在一定偏差。行为综合是在行为仿真之后,将类 C 语言描述的功能逻辑转换成 RTL 描述的过程。HLS 则会将 C 语言规格转化成寄存器传输 (RTL) 来实现。对于顶层函数实参,会被综合成 RTL 的 I/O 端口。而 C 语言函数则会被综合到 RTL 层级内的块中。C 语言代码中的数组通常会被综合成 FPGA 设计中的块 RAM 或 UltraRAM 中。此外,在设计优化上, HLS 提供了众多的优化指令,如数组分割、循环展开、流水线设计等。开发者能够通过指令级的调整实现不同的优化,从而对比其优缺点进行改进,大大提升了开发效率。

3.3 Vitis 开发框架

Vitis 统一软件平台由 AMD Xilinx 于 2019 年首次推出，旨在可以让软件工程师和 AI 算法工程师等各类非硬件开发者利用硬件可编程、高自由度的优点进一步优化自己的工程和实验^[58-63]。其将 Vivado、HLS、SDAccel 以及 petalinux 等软件开发工具组合为一个整体，不同于传统采用 Vivado 和 SDAccel 的硬件开发模式，使得开发者不需要过分关注底层硬件的接口和通信问题，可以将更多的精力集中在应用设计本身的程序功能实现和优化上。Vitis 平台整个硬件设计的底层框架由 Vitis 目标平台，Xilinx RunTime (XRT) 库以及 Vitis Core 开发套件三部分构成。其中，Vitis 目标平台定义了软硬件的基本框架和应用开发环境，包括外部存储接口，自定义的输入输出接口等。XRT 库主要负责 HOST 端和 FPGA 加速卡之间的通信，包括用户 API，内核驱动等。Vitis Core 开发套件则包括了 V++ 程序编译器，性能指标分析器和代码调试器，用以对比分析程序设计优化的性能指标。同时平台本身包含一些应用层的加速库，可以更方便的配合优化开发者所写地内核程序。Vitis 开发平台框架图如图 3-2 所示。

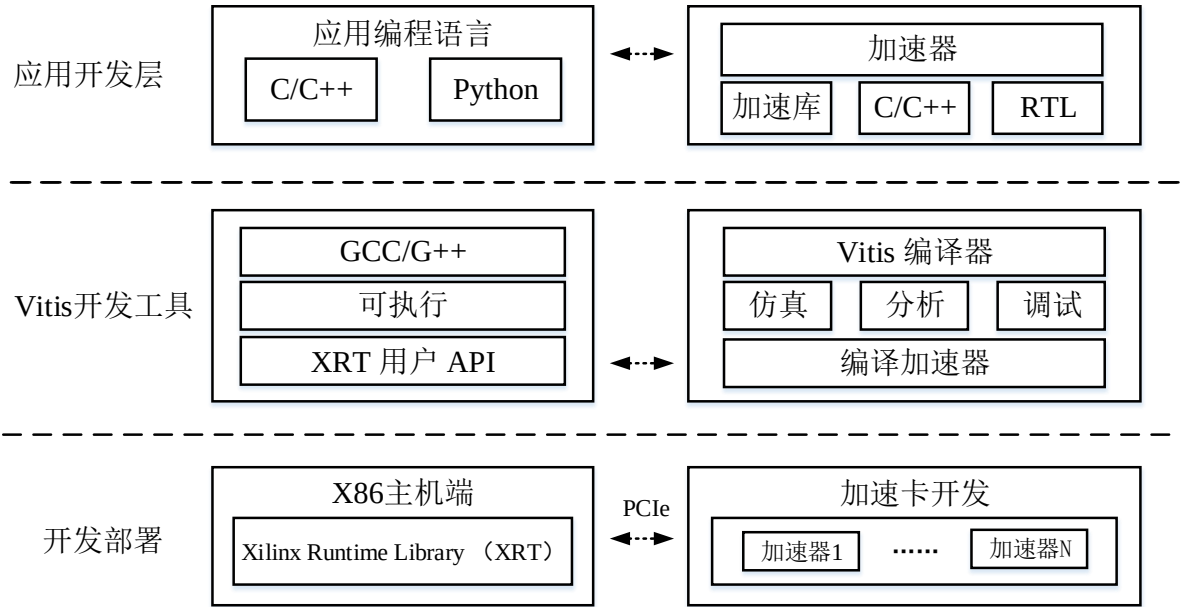


图 3-2 Vitis 开发平台框架图

Vitis 支持两种内核加速方式，一种是 PS 或 X86 硬件加速内核，其特点是能够控制内核。另一种是无 PS 或 X86 硬件加速内核，其特点是无控制信号端口，不能启停，自由运行。开发流程上，分为主机端设计和 FPGA 设计两部分，开发者可以使用标准编程语言如 C/C++，Python 等完成主机端的开发，可在 X86 或者嵌入式处理器上运行，通过 OpenCL API 调用来管理与 FPGA 加速卡上 XRT 的交互。而针对硬件内核的开发则可以使用 C/C++、OpenCL 或者 RTL 来完成。主机端程序会编译成可在主机端处理器运行的可执行文件（.exe），而内核程序则会编译成在 Xilinx 器件的可编程逻辑（PL）区域内运行的可执行器件二进制文件（.xclbin）。最终可实现硬件仿真或者是直接在硬件加速设备上运行。内核运行模型开发架构如图 3-3 所示。

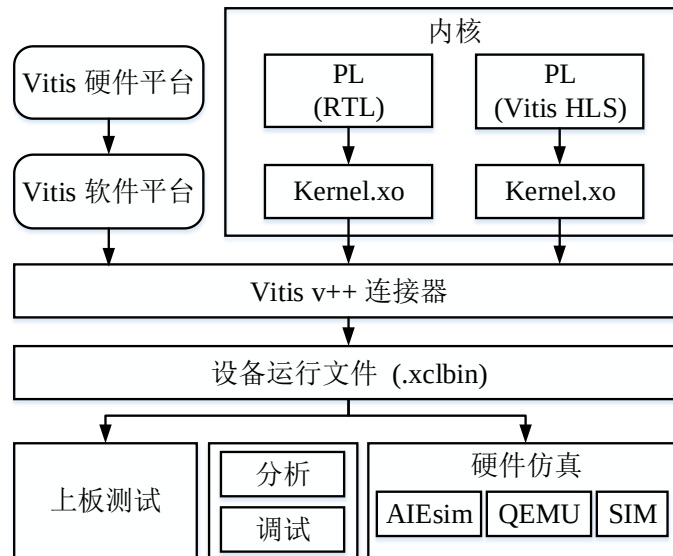


图 3-3 内核运行模型开发架构

3.4 基于 OpenCL 的 FPGA 开发

OpenCL 作为异构并行计算平台编写程序的标准框架,可以将异构计算映射到 CPU、GPU 和 FPGA 等不同的计算设备上。其将硬件底层架构抽象成一个模型,为异构计算开发者提供一个通用 API^[64-71] (Application Programming Interface)。OpenCL 异构并行计算架构根据其不同的表述特点分为四种模型,分别为平台模型 (platform model)、存储器模型 (memory model)、执行模型 (execution model) 和编程模型 (programming model)。这些模型从各个角度上共同构成了 OpenCL 的整体架构,且彼此之间既相互独立又存在关联。

平台模型是 OpenCL 对硬件的一个抽象描述,由主机端和 OpenCL 设备端两部分构成。通常,主机端为 X86 架构的 CPU 处理器或者是 ARM 处理器。平台模型结构如图 3-4 所示,对于 OpenCL 设备端,每一个计算设备 (Computing Devices, CD) 都包含有一个或者多个计算单元 (Compute Units, CU),而每一个计算单元又是由一个或者多个单元 (Processing Elements, PE) 组成。PE 是 OpenCL 设备端上执行数据计算的最小单元。

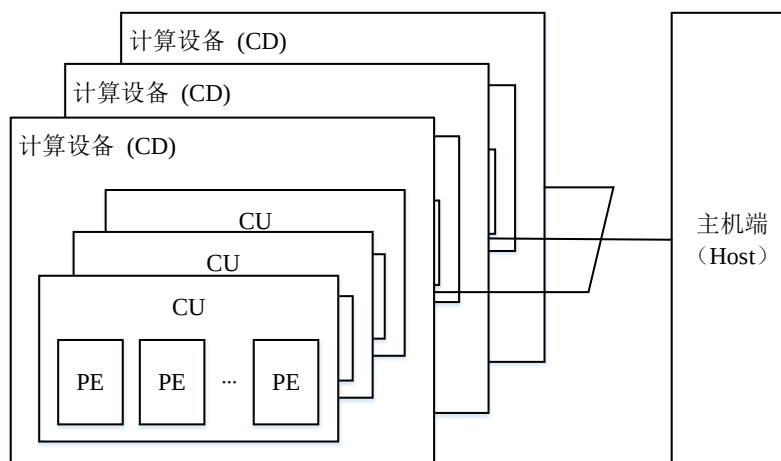


图 3-4 OpenCL 平台模型

执行模型将 OpenCL 程序分为两部分，分别是主机端程序和 OpenCL 设备端内核（Kernel）程序。内核程序在 OpenCL 设备端上执行，完成 OpenCL 设置的特定程序功能。在内核启动后，会创建一个整数索引空间。索引空间是 OpenCL 支持的一个 N 维值的网络，被称为 NDRange，其中 N 为 1, 2 或 3。内核、关联内核参数值和定义索引空间的参数定义了一个内核实例。如图 3-5 所示，执行内核的各个实例被称为一个工作项（work-item），工作项将由它的索引空间中的坐标来标识，这个坐标即工作组的全局 ID。工作组（work-group）由多个工作项组成，工作项在工作组中的坐标为其局部内存。

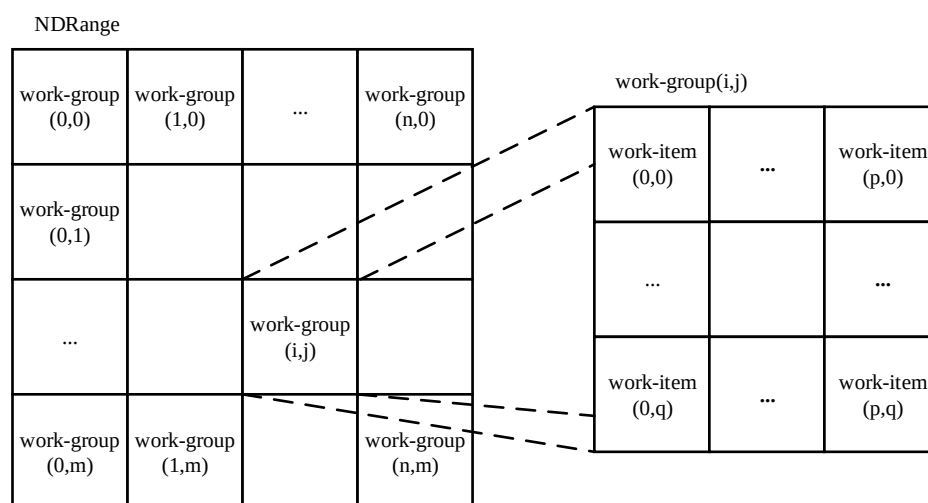


图 3-5 OpenCL 二维索引空间

存储器模型中的存储器区域包括主机和设备的内存。具体分为 5 类，分别是主机内存（host memory）、全局存储器（global memory）、常量存储器（constant memory）、局部存储器（local memory）、私有存储器（private memory）。其中全局存储器和常量存储器可以在主机端和 OpenCL 设备端或者多个 OpenCL 设备端之间共享，一个 OpenCL 设备关联局部存储器和私有存储器。

编程模型包含两种编程方式，分别是数据并行和任务并行。其中，数据并行时将计算的数据进行分割，并将计算任务下放到不同的计算设备上。适合低耦合的数据计算任务。而任务并行是针对少量的数据，但存在一定的耦合，需要分阶段并行完成。

本文采用基于 OpenCL 的 Vitis 平台来验证内核逻辑的正确性。首先初始化 OpenCL 的环境配置和命令队列，接着将设计的内核程序通过 PCIe 加载到加速卡中，最后完成 FPGA 的编程工作。而对于 OpenCL 的内存分配首先申请简单的缓存分配，然后将分配的缓存映射到 OpenCL 缓存对象，接着将主机内存迁移到加速卡中，并设置好内核参数，最后运行内核，等待内核运行完成后，将内核计算的结果传输回主机端内存。

3.5 本章小结

本章主要介绍了实验采用的硬件平台 FPGA 的基本概念和关于 FPGA 动态可重构技术的优势和不足。首先阐述了 FPGA 目前主流的开发语言 HDL 和 HLS，并着重介绍了 HLS 的开发流程和优缺点。接着介绍了实验采用的开发平台 Vitis 和总体的开发流程。最后介绍了异构平台框架 OpenCL，并从四个模型来描述 OpenCL 的特点和优势。

第四章 基于 FPGA 的 FAST 硬件解码器设计

4.1 设计思路

FAST 协议解码分为预处理 FAST 数据提取模块与核心 FAST 数据解码器两部分。预处理 FAST 数据提取模块的主要功能是提取嵌套在 STEP 数据中的 FAST 数据并将其送入 FAST 数据解码器中解码。以从上海证券交易所接收到的裸行情数据为例,实时行情数据以 FAST 编码的二进制数据形式,通过 Tag96 嵌入 STEP 消息中。针对这一特征,需要将 FAST 数据从裸行情数据中提取出来。STEP 消息采用“Tag=Value”的传输格式,其中“Tag=95”的 Value 部分为 FAST 数据长度。“Tag=96”的 Value 部分为 FAST 数据。对于裸行情数据,通过对连续的字符进行判断,当检测到三个连续字符为“95=”时,记录并存放等于号后数据解码所得到 FAST 数据大小。当检测到三个连续字符“96=”时,将等于号后的数据存放到 FAST 数组中,同时记录数据的长度,直到数据的长度等于 FAST 数据大小为止。提取 FAST 数据逻辑流程图如图 4-1 所示。

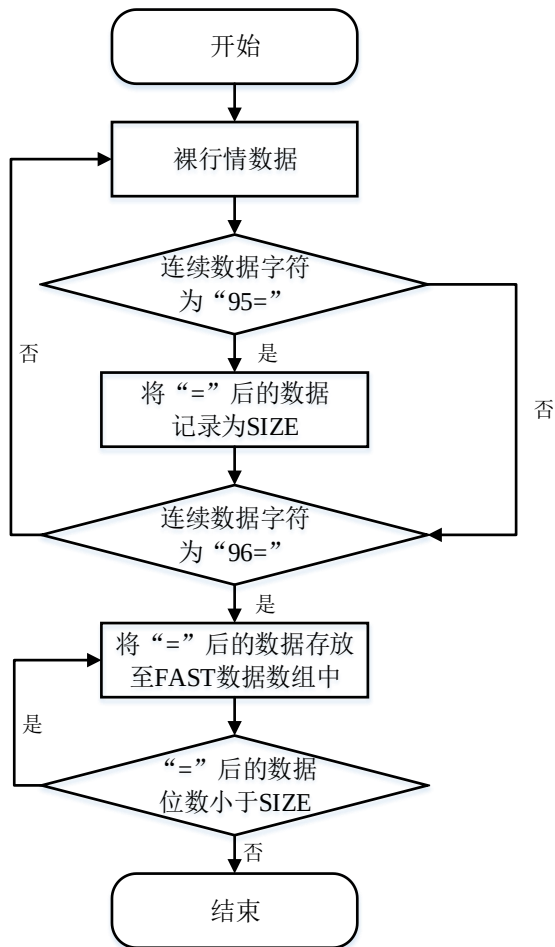


图 4-1 提取 FAST 数据逻辑流程图

核心 FAST 数据解码器由 Pmap 值提取、标记字段起始和终止位置、字段切分、数据类型判断和合并、字段数据解码五部分组成。

FAST 数据解码首先需提取 Pmap 值,该操作为后续字段切分的边界和字段解码的规则提供判断依据。同时,FAST 数据标记字段分割后,依靠位置存放数组进行字段切

分。分割完成的数据通过判断数据类型来决定是否进行字段合并。最终各字段的解码由 Pmap 值，分割合并的字段以及前值字典共同完成，并输出结果。此外，根据每一次的输出结果更新前值字典以供后续的数据解码。具体 FAST 解码器设计流程图如图 4-2 所示。

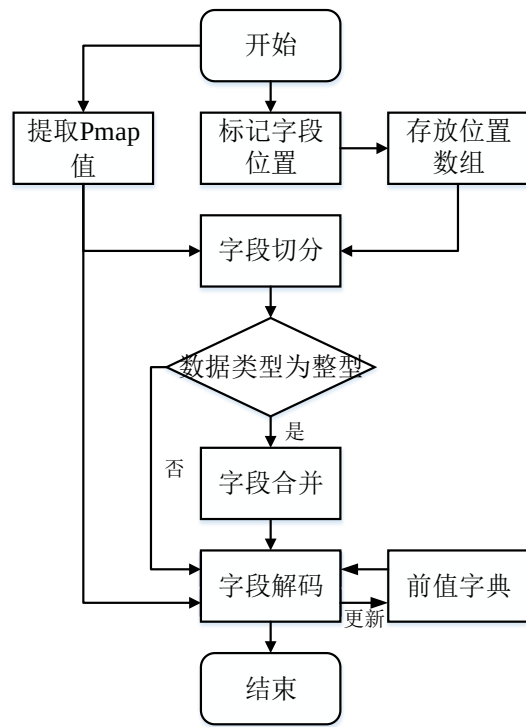


图 4-2 FAST 解码器设计流程图

4.1.1 预处理存在位图

存在位图作为整个解码过程中位元的一个序列，代表着数据流中各字段中的域需要根据当前的模板规则来使用 Pmap 值位元。存在位图与传统字段域一样，采用停止位编码实体来表示。逻辑上存在位图的表示长度为 8 的整数倍比特。

以交易所逐笔成交 UA3201 为例，存在位图的位宽表示占用了 16 个位。以 8 位为一个单位，去除首位的标志停止位，即最高有效位。实际表示的内容大小为 7 的整数倍比特，且与模板字段一一对应。同时，在 Pmap 值为 0 的情况下，对应字段域不存在于数据流中。因此，将实际表示内容位元的对应 Pmap 值进行累加，则获得了后续需要切分字段的个数。存在位图示例图如图 4-3 所示。



图 4-3 存在位图示例图

如图 4-3 所示。由于 UA3201 模板仅有 13 个域，其中一个为无操作符，始终出现在消息流中，不占用位元，所以存在位图位元实际为 12 个，又因为存在位图同样遵循停止位编码实体规则。即去除最高有效位后，实际传输数据的大小为 7 的整数倍比特。因此存在位图字段 12 后的两位为默认值 0，无实际意义。将图上 12 个字段对应的 Pmap 值相加计算获得值为 10，说明后续除无操作符字段域以外，还存在 10 个字段域数值。

4.1.2 字段标记与切分

经过对存在位图的预处理，获取到了各个字段对应的 Pmap 值以及之后需要分割的字段个数。根据 FAST 数据采用停止位编码的格式特点，即每 8bit 为一个最小单位，首位为最高有效位，对每 8 的整数倍位置进行标记判断，记录每一个“1”的位置，并将该位置的数组下标记录下来作为后续字段切分的起始位置和终止位置。

针对数据字段的切分，尽管每一个字段的起始和终止位置无耦合，且 FAST 数据的特征不定长，但其实际大小始终以 8 的整数倍比特在一定的范围内波动。模板中实际可能传输的数据字段条数最大为 12，其中 4 条为 int64 共计占 256bit，5 条为 int32 共计占 160bit，一条为模板类型，固定为 8bit，一条为字符串类型，固定位 48bit。因此累计单条最大数据长度为 472bit。为了确保 FAST 数据解码器接收数据的完整性，从而减少多次读取产生的延迟抖动，将设计接口大小设计为 512bit。根据以上特征，本文采用并行切分的设计思想可以有效降低数据切分的时延，从而提升整体的解码效率。考虑到 FAST 数据存放在一个数组中，而并行切分需要对数组进行多次的访问，可能产生访问冲突从而导致数据读取效率降低的问题。对此，本文将 FAST 数据数组映射到多个存储器，通过提高存储器的访问接口，来提高切分的并行度，从而实现解码处理的低延迟。数据字段域切分示意图如图 4-4 所示。

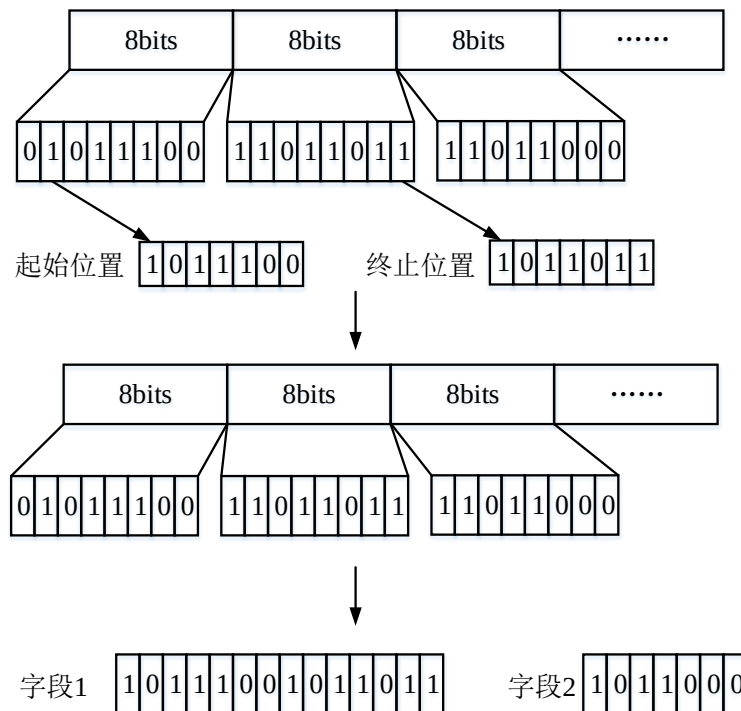


图 4-4 数据字段域切分示意图

如图 4-4 所示,以 8 比特为一组作为最小单元,对最高有效位进行判断。如果首位为 1,则将该位后移七位的数组下标作为该字段的终止位置,同时将该位后移第八位的数组下标作为下一个字段的起始位置。通常,第一个字段的起始位置为数组第二元素下标。通过存在位图将 FAST 数据分成对应的字段个数。起始位置数组和终止位置数组的元素个数与分割字段的个数一致。

当完成一条 FAST 数据的标记位置数组后,借助内存映射互相独立的访问空间,对每一个独立 8 比特位的数组进行分割和合并,除去每 8 位的最高有效位,将每个映射空间的 7 个有效数据位按照起始和终止位置进行合并,最终形成每一个字段的停止位编码实体的实体值。

4.1.3 处理单元内核复制

由于分割完成的有效数据字段存放在不同的存储空间中,这些数据相互独立,互不影响。因此在设计上,为了提高数据的总体吞吐量,采用并行解码的设计思路。对于分割完成的字段数据处理,首先判断数据类型以及对应的 Pmap 值,通过这两个因素,对处理单元进行数据解码处理。针对数据类型,主要分成两大类,一类是整型,另一类是字符型。对整型而言,需要将整个存储字段的数组作为一个整体进行解码,而对字符型而言,则以七位有效数据位为一个单位,单独进行解码。最终解码得到的结果为对应 Pmap 值为 1 时的输出结果。如果字段对应的 Pmap 值为 0,此时需要按照模板中对应的 FAST 操作符进行对应的解码。以上海证券交易所接收到了逐笔委托 UA3201 为例。十二个字段对应的操作符有无操作符、常值、复制、缺省以及自增操作符。其中,常值操作符和无操作符均不占用 Pmap 值的位元。无操作符始终出现在数据流中。因此实际需要考虑的仅有复制、自增、缺省以及常值操作符。其中,缺省和常值依赖于模板中的初值,即当对应 Pmap 值为 0 且对应模板中的操作符为缺省或者常值时,输出模板对应位置的初值。复制和自增均需要涉及初值设定。复制操作符即将初值复制输出,而自增操作符需要在原有的初值上自增 1 输出解码结果。复制和自增操作输出后均需要对初值进行更新。

针对上述逻辑,为了设计并行的解码操作,对内核进行复制。内核处理的输入端为字段数据、数据类型、前值字典、对应 Pmap 值以及 FAST 操作符。数据字段需要通过数据类型判断来决定是否进行字段合并。对于整型数据类型,需要对字段数组进行合并整体进行解码处理,而对于字符数据类型,则以每 7 比特为一个块,分别进行解码处理。在解码的过程需要通过 Pmap 值的判断来决定是否需要采用 FAST 操作符。如果 Pmap 值为 1,则解码器仅需要按照数据类型进行解码即可,反之,如果 Pmap 值为 0,则需要采用 FAST 操作符进行操作。当 Pmap 值为 0 时,需要判断该字段的 FAST 操作符类型,如自增、复制等操作符需要涉及前值运算的,则需要涉及前值存储器单元,根据前值存储器中的值与输入数据进行解码计算。涉及前值计算的操作符,无论是否使用到该操作符的运算,在解码完成后均需要把结果作为新的前值保存起来,对前值存储器的对应数据进行覆盖更新。

FAST 解码单元整体设计核心为将并行分割完成的字段数据通过类型判断和合并将最终需要处理的数据送入解码器中。通过 PMAP 值和 FAST 操作符的联合判断来决定解码器解码是否需要使用特定的 FAST 操作符和前值字典。最终，根据解码的结果对前值字典进行更新。FAST 解码单元框架如图 4-5 所示。

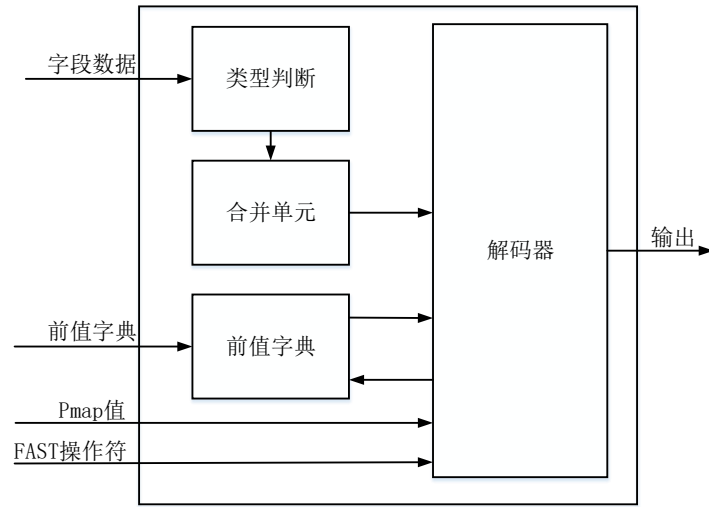


图 4-5 FAST 解码单元框架图

由于 FAST 数据各个字段通过标记和分割分别存储在不同的数据字段寄存器中，这些寄存器之间相互独立，无相关性。因此，针对各个字段的解码过程也无耦合。为了确保传输解码数据的完成性，字段最大为 64 位整型，而字符串类型相对固定，通常以 48 比特表示。因此，设计数据传输接口位宽固定为 64bit。由于 FAST 数据单个字段的解码处理所消耗的时钟周期为 1~8 个 Cycles。在未采用并行设计的前提下，总消耗为 12~96 个 Cycles，产生的延迟抖动性较大，不易于解码模块延迟的稳定性。通过内核复制，可以将解码延迟控制在单个字段域最大解码延迟上。根据 FAST 模板的域字段个数，对 FAST 数据解码内核处理单元模块进行复制。以上海证券交易所 UA3201 逐笔成交为例，将 FAST 解码单元复制成 12 份，实现 FAST 数据字段 12 路并行解码，从而降低整体数据字段解码的延迟和抖动，提高解码数据的吞吐量。FAST 解码单元内核复制示意图如图 4-6 所示。

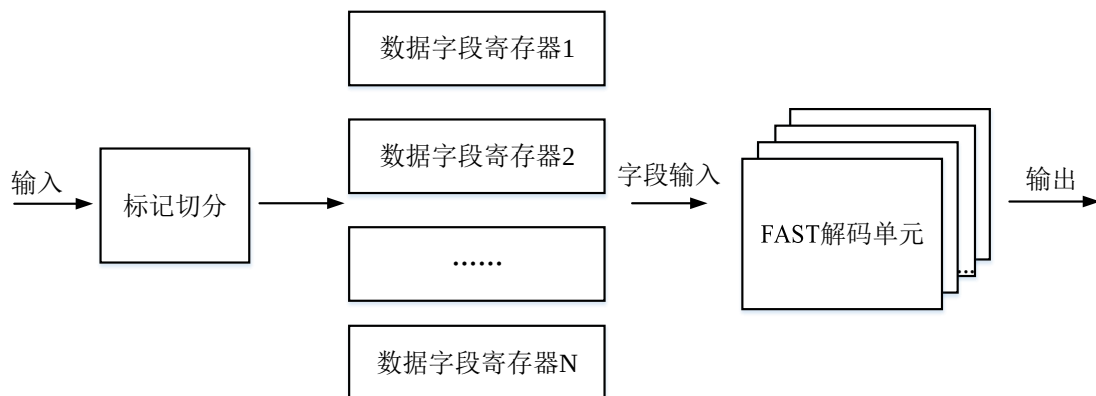


图 4-6 FAST 解码单元内核复制示意图

4.1.4 存储优化设计

由于 FPGA 硬件资源有限，这一点尤其体现在硬件存储资源上，而 FAST 解码器设计在数据字段分割存储以及前值字典的存储均需要占用大量的存储空间，对资源的需求较大。同时，存在位图和各个字段的存储范围跨度也较大。以模板编号为例，该字段在数值设置上占用了 64bit 的存储空间，但是其实际存储空间不超过 16bit。由于这样的存储范围跨度较大，容易造成采用传统的存储数据类型会浪费大量的存储空间的问题。针对以上问题，对于各字段数据，计算实际有效的存储范围，同时定制化地设计存储类型的位宽，从而优化存储空间，提高存储空间利用率。存储优化设计可以将更多的字段存储和计算数据存放在 FPGA 片上存储空间中，从而发挥片上存储高带宽读写速率的优势，提高传输数据的吞吐量，进而降低解码的延迟。

在满足字段存储需求的同时，通过定制化数据类型存储优化可以将存储空间利用率提升 1~2 倍。在一定程度上，提高了有效数据的存储利用占比。数据类型存储优化对比图如图 4-7 所示。

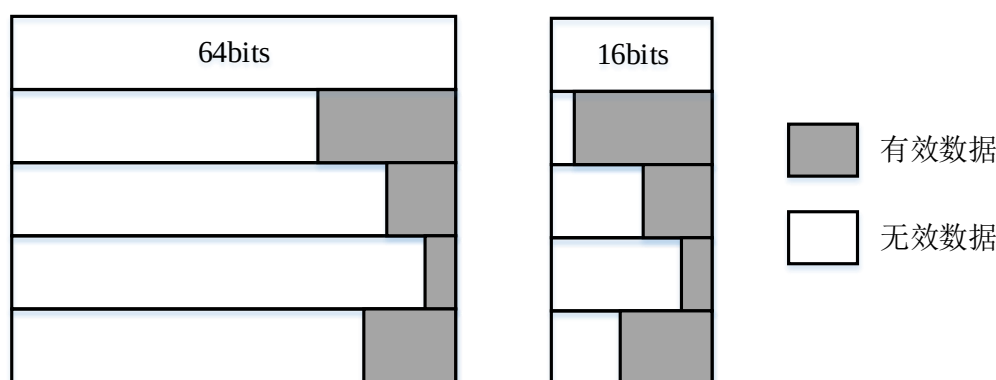


图 4-7 数据类型存储优化对比图

FPGA 的硬件存储资源主要分为片上存储和片外存储，其中片上存储通常为 SRAM，而片外存储通常为 HBM 和 DDR4。片上存储相比片外存储拥有更高的数据读写带宽，但是，其存储空间远小于片外存储，因此在设计空间存储中，考虑到前值字典等存储的读写较为频繁，需要优先将这一部分数据存放在片上存储中，通过其高带宽的读写速率，降低前值字段读写以及解码数据更新处理的操作延迟，从而进一步降低解码过程中的延迟，实现超低延迟的解码单元设计。

4.1.5 硬件流水设计

考虑到 FAST 数据压缩强耦合性和增量传输的协议特点，不同 FAST 数据前后存在前值字典的读写和更新操作，针对多条 FAST 数据，数据之间无法做到并行解码处理。由于字段域数据处理前后无耦合，因此可采用硬件流水设计。在设计上，为了确保流水设计的效率最优，将总体处理过程细分成多个步骤，尽可能使每一个部分的延迟（Latency）相等。将 FAST 解码设计主要分为标记、切分、合并、解码四部分处理，由于四部分均不涉及复杂运算，处理延迟均为一个时钟周期，同时，由于各个模块之间的处理不存在交叉关联，而是前后呈现单进单出的数据相关性，即前一个模块的输出作为

后一个模块的输入，因此在整体上能够实现最优化的硬件流水设计。流水设计处理对比图如图 4-8 所示。

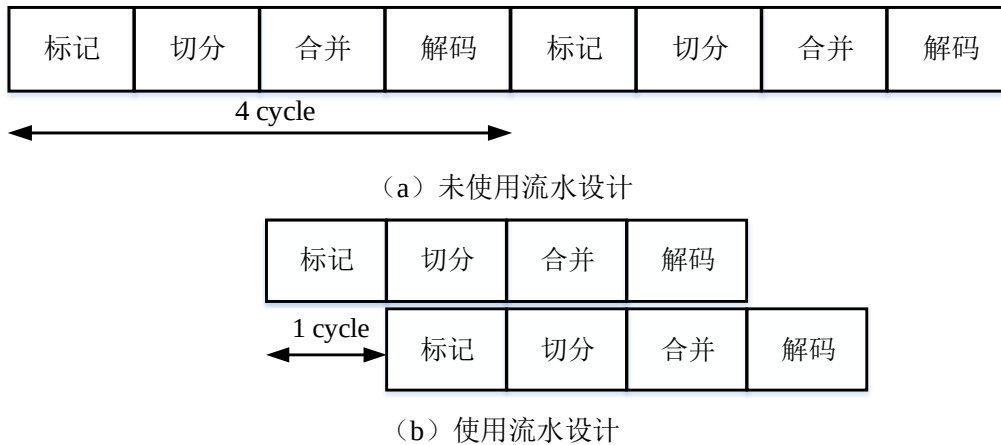


图 4-8 流水设计处理对比图

流水设计尽管无法优化降低单条数据的解码延迟，但是其针对连续多条数据的处理，可以提高整体 FAST 数据解码的吞吐量，从而进一步提高总体的解码速率，降低解码延时。

4.2 系统实现方法

4.2.1 基于 OpenCL 和 HLS 异构系统实现

总体框架设计采用 OpenCL 和 HLS 协同开发的设计方式，并基于 Vitis 开发平台进行实现。设计主要由两部分组成，一部分为基于 OpenCL 的主机端设计与实现，另一部分则是基于 U50 Alveo 的加速卡实现。

OpenCL 负责数据在主机端（CPU）和 FPGA 端（U50）的传输和接口通信工作。首先，主机端接收事先存放在内存上的裸行情数据，然后完成对裸行情数据的读取，FAST 数据、字段数据、前值字典等计算数据存储空间的申请和初始化，以及处理内核的调度和管理等工作。具体设计过程为：首先导入 XRT 和 OpenCL 的报头，找到系统中的 Xilinx 加速卡并为其编号。由于实验中仅采用一块目标卡，因此将其标记为 0。然后初始化 OpenCL 的环境和命令队列，接着将二进制文件导入 Alveo 加速卡中，加载内核源代码。

对于存储分配和设置，首先进行简单内存分配，其中包含等待切分暂存的 FAST 行情数据内存和分割完成后的各字段内存空间存储申请，FAST 数据存储空间大小申请为 512bit，而各字段内存则根据具体字段含义设置，最大不超过 64bit。然后使用主机内存指针映射 OCL 缓冲器，将分配的缓存映射到 OpenCL 缓存对象。接着迁移主机内存到 Alveo U50 加速卡中，通过 OpenCL API 设置内核参数，并为内核运行排队，最终将解码完成的行情数据传输回主机完成 OpenCL 对主机端的配置设计。Xilinx Vitis 平台提供了 OpenCL 平台 API 方便用户自主调用。其主要负责创建上下文、命令队列以及申请内存分类和缓存映射，通过设置内核参数和管理内核执行流程来管理内核的运行。基于 OpenCL 的主机端设计流程图如图 4-9 所示。

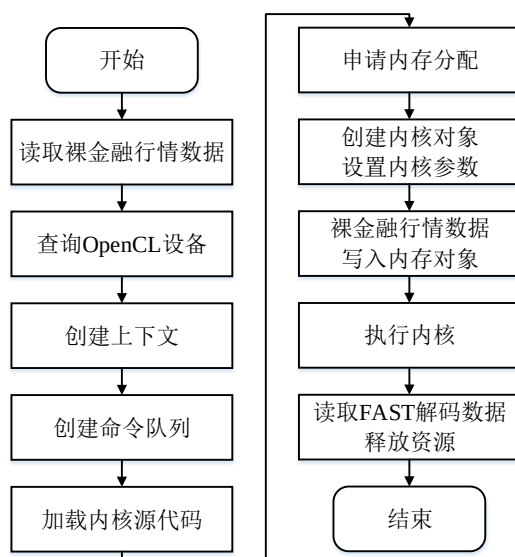


图 4-9 基于 OpenCL 的主机端设计流程图

此外，由于裸行情数据本身的数据量较小以及设计上对存储结构进行了优化，因此 Xilinx Alveo U50 加速卡片上的 SRAM 足以满足单次需要解码的 FAST 数据。因此，在对存储空间的申请上仅申请了 SRAM 的片上存储。

对于 FPGA 端，数据的输入主要是裸行情数据。数据解码主要分成两部分，一部分是基于 STEP 协议的解码模块，其主要功能是通过连续的字符判断筛选出“Tag=96”的 FAST 数据以及“Tag=95”的 FAST 数据长度，并将数据信息送至 FAST 数据解码模块进行解码。另一部则是基于 FAST 协议的 FAST 数据解码模块，其主要功能是根据 4.1 的设计思路章节对传输的 FAST 数据进行 Pmap 值提取、数据字段切分起始和终止位置标记、数据域字段切分、数据类型判断与合并以及数据域字段解码等操作。最终将解码完成的数据返回并读取。基于 OpenCL 的解码器设计实现流程图如图 4-10 所示。

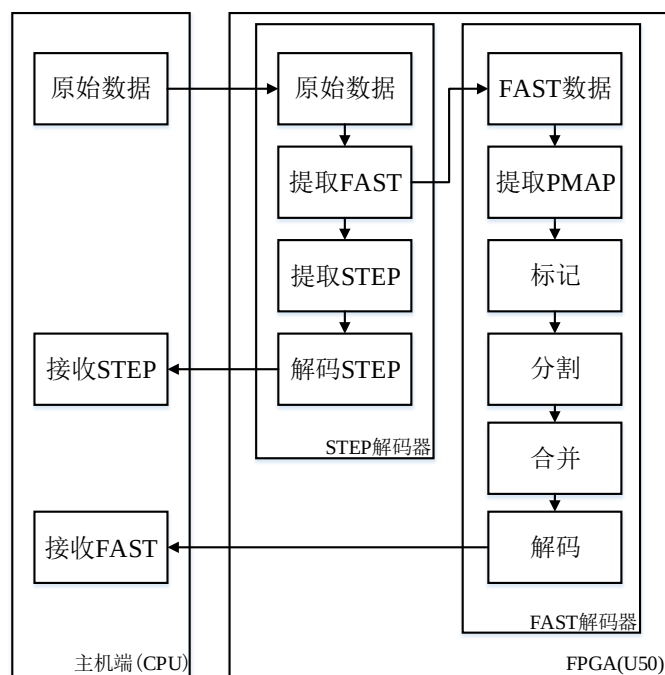


图 4-10 基于 OpenCL 的解码器设计实现流程图

4.2.2 任意精度的数据类型优化

C 语言传统的整型数据类型字节长度以 8 的整数倍为主。在 FAST 数据存储设计中,某些字段的最大位宽可能远小于传统的整型数据类型字长。为避免传统的数据类型可能会导致存储空间资源上的浪费,通过引入 HLS 任意精度的数据类型进行存储优化,任意精度数据类型的优势在于引入了相比传统 C 语言数据更小位宽的数据变量,并且能够运行编译器通过 C 语言仿真。更小的位宽往往意味着在综合布线时能够在 FPGA 上生成更小更快的硬件运算符,从而增加 FPGA 逻辑单元的布局密度,能够满足更高时钟频率下的程序运行。以 C++ 语言为例,将存储数据类型设置为 `ap_int<N>` 或者 `ap_uint<N>` 类型,其中 N 表示自定义范围 1~1024 的位宽大小。

具体针对 FAST 各个数据域字段进行优化设计,虽然字段数据在模板中的数据类型定义为传统的 32 位整型和 64 位整型,但是实际字段的范围则远小于给出的数据范围。以上海证券交易所 UA3201 逐笔委托为例,模板类型的字段数据均以四位表示,因此针对这一字段自定义数据类型设置为 `ap_int<4>`,而针对如成交数量、成交总金额这类的数据字段,则仍然采用模板标准定义的数据类型大小,从而确保数据不会因为存储溢出导致精度丢失。核心优化思想为在确保正确存储数据的前提下,尽可能对数据类型的位宽进行压缩,从而节省数据存储上不必要的资源浪费,最终提高 FPGA 的数据计算性能。

因此,任意精度的数据类型既可以满足数据存储的需求,同时还可以高效合理地利用 FPGA 上的存储资源。通过提高 SRAM 存储空间的利用率,用以存放高频访问的 FAST 数据字段数据,从而提高数据的读取速率,降低 FAST 字段解码延时。

4.2.3 FAST 数据字段切分优化

由于需要对 FAST 数据的每一个比特位进行组合和运算,因此采用数组来存储 FAST 数据以及后续切分的独立数据字段域。与传统的主机不同,FPGA 综合时,通常会将数组作为内存(RAM、ROM 或者 FIFO)来实现。此外,数组的实现体现在两个方面,一个是顶层函数接口,通常作为访问外部内存的 RTL 端口来综合实现。另一个则是函数内部的设计,通过数组大小来区分,存储大小小于 1024 的数组将作为 SRL 来综合,而大于 1024 的数组将综合到块 RAM、LUTRAM 或 UltraRAM 中。

在 FPGA 综合期间,RAM 端口还可以配置端口类型是单端口还是双端口。一方面 FAST 数组切分的过程需要频繁地访问数据数组里的元素,端口的个数限制无法满足后续字段解码模块并行处理的设计需求,另一方面针对 Pmap 值的访问,也受限于内核端口的访问瓶颈。

为此,通过数组最优化指令重新配置数组结构,将数组元素映射到新的存储块中,数组分割指令分为三种类型,分别为 Block 切分、Cyclic 切分以及 Complete 切分。其中 Block 切分是将数组元素进行切分,切分个数通过 Factor 参数决定,然后按照分块顺序依次进行填充,每一块直至填满再开始填充下一块。Cyclic 切分分配则是将数组元素循环地填充到每一个分组块中,通常每一块的元素个数相同或者相差一个元素。而 Complete 的分割方式则是将每一个数组元素均分割对应到单独的一个寄存器存储中。三种数组分割的切分存储映射示意图如图 4-11 所示。

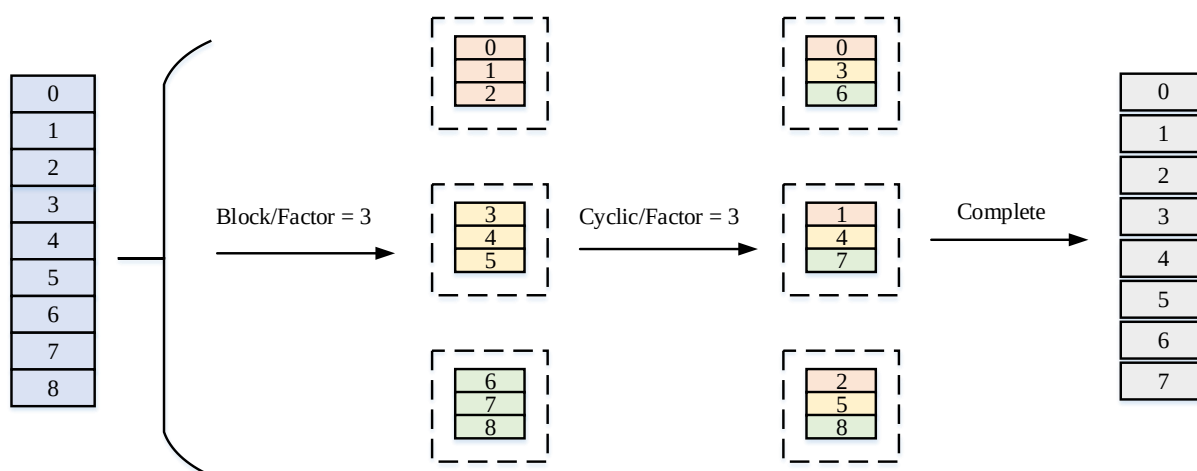


图 4-11 数组分割的切分存储映射示意图

其中 **Factor** 表示指定创建的阵列数量。如果阵列的元素数量并非该因子的整数倍，那么排列在后的阵列相对所含的元素数量较少。

针对 FAST 数据的特征和解码存储需求，主要体现在 FAST 数据数组的切分和存在位图数组位元的并行访问上。为了确保能够并行的标记和切分 FAST 数据数组，根据 FAST 数据最高有效位的特点，对于数组下标为 8 的整数倍的数组元素能够并行的访问，采用 **Block** 数组切分，将 FAST 数组每 8 位为一组进行分割，使 FAST 数组分割成多个小的数组，映射到多个寄存器块上。确保在切分操作时，能够在各自数组接口上单独实现，从而提高并行访问的限度。FAST 数组 **Block** 切分示意图如图 4-12 所示。

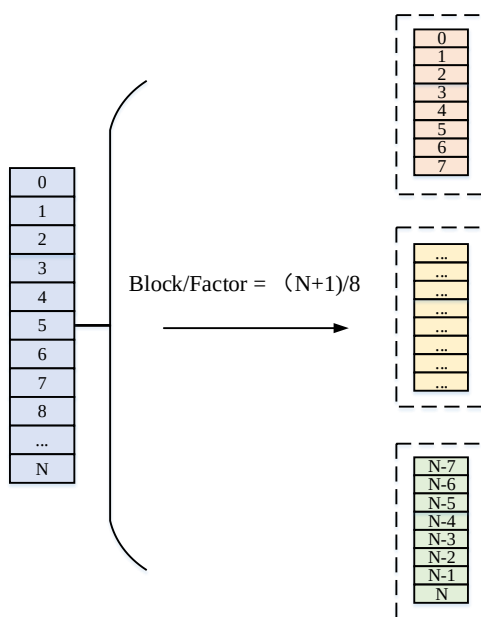


图 4-12 FAST 数组 Block 切分示意图

由于 FAST 字段解码的输入端涉及存在位图对应的 **Pmap** 值，而存在位图的切分和存储是以数组的形式存放，为了能够并行地对各字段进行解码，即需要同时访问存在位图数组中的每一个数组元素。此外，存在位图的数组元素个数较小，对此采用 **Complete** 分割，将存在位图数组按照数组元素独立拆开，这一过程对应于将存储器解析成独立寄

寄存器。同时使得字段的解码并行度为 FAST 分割字段个数。Pmap 值数组 Complete 切分示意图如图 4-13 所示。

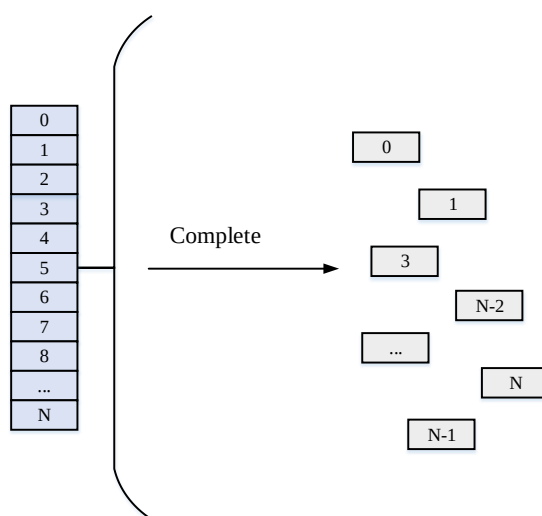


图 4-13 Pmap 值数组 Complete 切分示意图

其中 Block 为数组的元素个数，Factor 为每一个数据切分块的数组元素个数。Block/Factor 的结果即为数组通过切分映射优化后的访问并行度。

4.2.4 解码各模块 Dataflow 流水优化

针对硬件流水设计思想的实现，主要通过 HLS 中的 Dataflow 流水优化指令来实现。考虑到解码所需的存在位图 and 对应字段之间存在的数据耦合，FAST 解码器被设计分为四部分，分别是标记、切分、合并以及解码。Dataflow 优化是在这四部分模块处理过程中，两模块之间穿插通道，设置通道的寄存器类型为 FIFO，将前一个模块首先处理好的部分数据存放到 FIFO 中，供下一个模块处理，同时确保各模块之间的输入和输出均有顺序性。实现过程需要对数据的传输大小、FIFO 的深度进行测试评估，以防止 FIFO 深度不够，导致出现死锁问题。考虑到每次处理的数据字段域大小最大为 64bit，因此在深度设计上，FIFO 的深度能够与之匹配，从而使得四部分模块与模块之间呈递流水的处理过程，最终提升解码数据的吞吐量，降低总体解码的延时。Dataflow 对比实现示意图如图 4-14 所示。

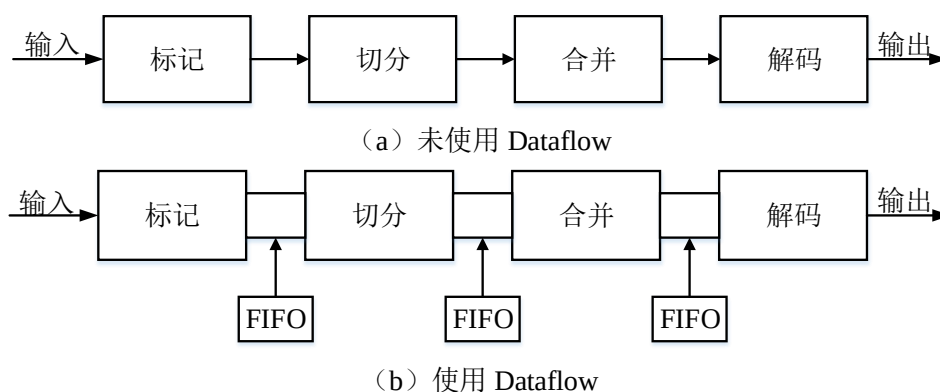


图 4-14 Dataflow 对比实现示意图

4.3 实验结果与分析

4.3.1 数据来源和环境模拟

实验数据来源是上海证券交易所接收到 2020 年 3 月 19 日的 Level-2 行情数据，主要涉及 UA3201 竞价逐笔成交数据信息以及 UA3202 竞价行情数据。将接收到的行情数据以 Pcap 文件格式存放，搭建 linux 服务器，通过 Tcpreply 重放工具对按照原速率进行数据回放，通过抓取端口 9129 的回放数据存放至主机端的内存中来模拟真实上交所行情数据的行情数据接收过程。具体环境搭建模拟回放数据示意图如图 4-15 所示。

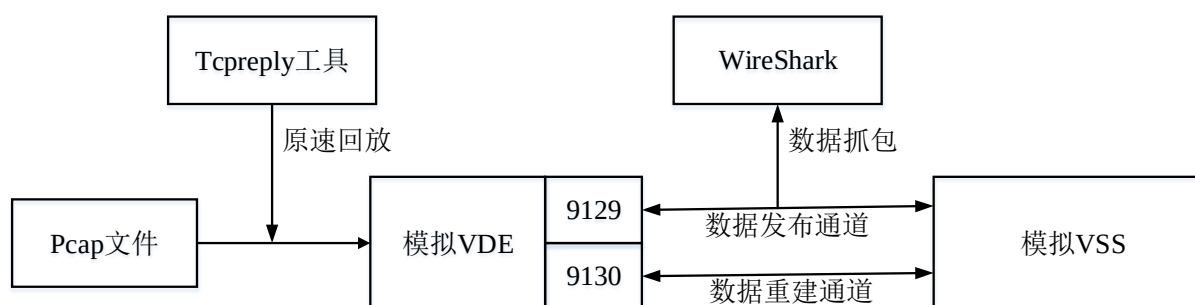


图 4-15 环境搭建模拟回放数据示意图

其中 VDE 为交易所提供给行情商的 Level-2 系统专用接入程序，用以获取 Level-2 行情数据。行情商仅能使用 VSS 作为接入端程序接收由 VDE 接口程序发来的 Level-2 数据。VSS 与 VDE 通过 TCP/IP 建立连接并部署在同一网关中以确保网络环境的稳定。通过 Tcprelay 建立一个回环发送模拟，将发送端和接收端部署在同一台服务器上，同时模拟 VDE 提供两个端口与 VSS 连接，模拟端口号分别为 9129 和 9130，9129 端口提供数据的发送，9130 提供数据的重建。此外，VSS 仅能和 VDE 其中一个端口建立唯一的连接^[72-75]。通过 WireShark 从数据发送端口抓取行情数据完成数据的接收工作。

4.3.2 平台和软硬件环境

FPGA 硬件测试平台采用了 Xilinx 公司推出了 Alveo 系列 U50 加速卡，实际工作频率：300MHz，在硬件存储器容量上，配备了 8G 的外部 HBM 存储和 28MB 的内部 SRAM 存储。在计算资源上，拥有 87.2 万的查找表、174.3 万的寄存器以及 5952 的 DSP slice 资源，物理接口通过 PCIE 与主机端连接，卡槽类型为 Gen3x16，同时自带了一个 QSFP28 的网络接口，网口通信速度达到 100GbE，加速卡最大功率仅为 75W。开发工具在 Xilinx Vitis 2020.1 的开发平台上完成仿真、上板的测试工作，将测试结果通过 OpenCL 返回主机端打印验证 FAST 解码数据结果的准确性，并通过 Vitis 平台的分析器分析总体的性能。

对比测试软件服务器系统为 Linux Centos7，处理器采用 I9-9900X 10C20T，主频 3.5GHz，内存 64GB，由于操作系统的调度问题，数据测试存在延迟波动，采用多次测试取平均的方式对时延进行测试，同时为了确保数据本身导致的误差时延，确保软硬件的测试数据为同一组数据。

4.3.3 主要资源占比

经过存储优化，FPGA 片上的 SRAM 资源足以满足 FAST 数据的存储需求。因此，在数据空间定义和分配上，并未采用 BRAM 存储，U50 HBM 总带宽为 316GB/s，而内部 SRAM 总带宽可以达到 24TB/s，合理利用片上存储 SRAM 高带宽读写速率的优势，可以有效降低解码时延。此外，合并和解码的工作主要通过位运算实现，因此并没有使用到 DSP 资源。主要硬件资源占用如表 4-1 所示：

表 4-1 主要硬件资源占用

硬件资源	使用	可用	占比
BRAM	0	2688	0%
DSP	0	5952	0%
FF	10070	1743360	~0%
LUT	51912	871680	5%

4.3.4 解码延迟抖动性对比

对于同样大小的数据测试解码延迟，横坐标为测试数据编号，纵坐标为解码延迟。FAST 数据由多个字段域构成，其中多数字段域的波动范围相对固定。例如模板域对应的大小为固定的 4 个十六进制字符，仅有如价格和数量等字段域有小幅度波动，但仍然可以计算波动范围。例如如果前一日收盘价为 10 元，则订单价格根据股票涨停和跌停的机制，价格的波动范围为 9~11 元。对应数值上的表达精确到分为 900~1100。因此二进制表示位数范围为 10~11 位，波动幅度较小。通过实际测试数据筛选，数据长度在 75 个十六进制字符为基准上下浮动且该长度的样本数量较多，具有一定的代表性。因此，测试选用了三十个大小相同的数据，均为 75 个十六进制数字符，分别测试其在纯软件，硬件不使用 Dataflow 优化以及硬件使用 Dataflow 优化的解码延迟。由于硬件具有固有的时钟周期，因此其在 FPGA 上的硬件解码延迟相比软件解码更加稳定。由于并行、流水以及接口等优化，实际解码波动仅为 2~3 个时钟周期，在加速卡 300MHz 的工作频率下，解码抖动延迟控制在 10ns 以内。解码延迟抖动性对比图如图 4-16 所示。

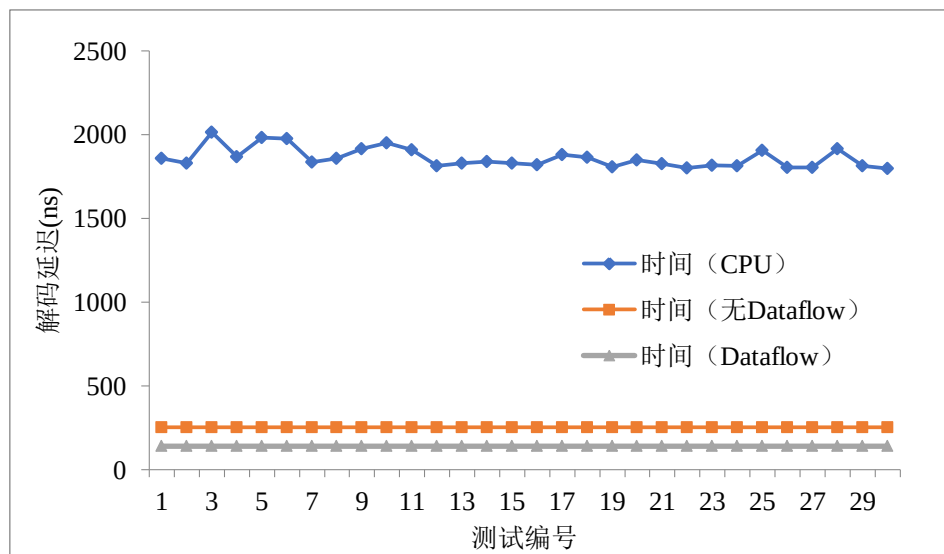


图 4-16 解码延迟抖动性对比图

4.3.5 解码延迟与数据字段长度关系

由于数据字段的标记和切分所需要的时钟周期与字段的长度有关,因此数据字段长度与解码延迟存在一定的关联性。同时,通过并行设计将解码优化到单个字段域的解码过程,即在解码模块,延迟取决于耗时最多的字段域。相比较而言,解码波动较小。测试表明,随着字段长度的增加,解码的延迟也会相应地有所提高,但是其变化浮动较小。解码延迟与数据字段长度关系如图 4-17 所示。

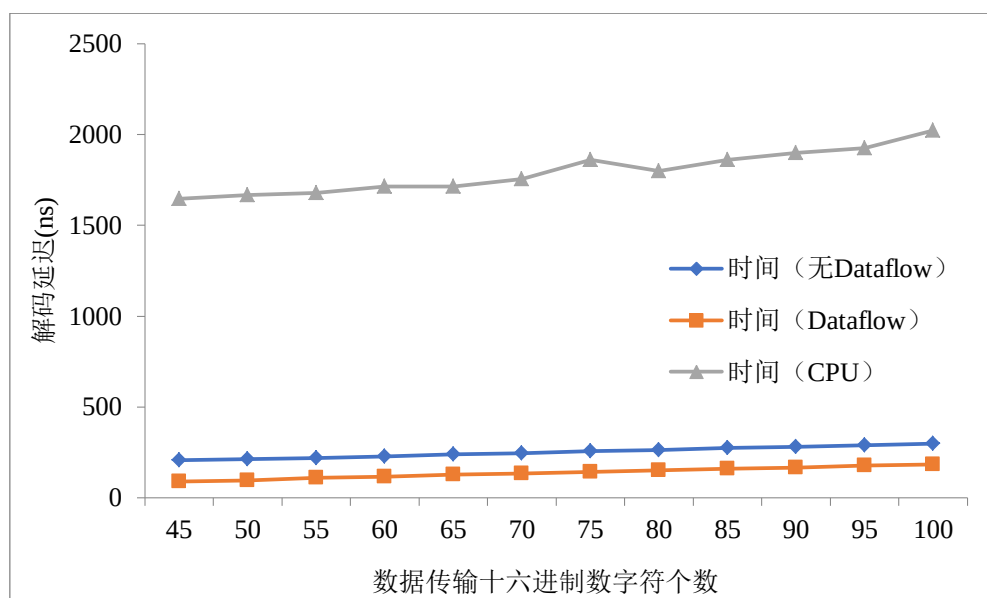


图 4-17 解码延迟与数据字段长度关系图

4.3.6 FAST 解码延迟对比分析

由于国内上交所采用的是 STEP 嵌套 FAST 的编码模式,因此本实验数据对比主要以国内研究为主,从开发语言、频率、硬件开发器件以及解码延迟等多个方面进行对比。国外^[31]由于其解码的字段个数较少,除存在位图外,仅选用了 6 个字段进行解码,因此在解码的延迟上会更快一些。Accelize Market Feed Handlers 是美国一家公司所提供硬件解码金融消息的产品。根据本文所使用 FPGA 板卡的实际工作频率为 300MHz,将单个周期的延迟消耗参数设置为 3.33ns。相比以往实验研究^[25],在相同频率下,通过 Dataflow 等优化,在开发周期缩短的同时,FAST 解码速度提升了 23.7%。FAST 解码延迟分析比较如表 4-2 所示。

表 4-2 FAST 解码延迟分析对比

工作	开发语言	频率	器件	解码延迟
Accelize Market Feed Handlers	-	-	Xilinx Virtex-6 XP6V550-2G	200ns
文献 ^[30]	RTL	125MHz	Xilinx Virtex-4 FX100	500ns
文献 ^[25]	SystemC	240MHz	Xilinx Kintex XC7K325	300ns
文献 ^[31]	HLS	156MHz	Kintex-U 1517E	115ns
本文 (无 Dataflow)	HLS	300MHz	Xilinx alveo U50	296.6ns
本文 (Dataflow)	HLS	300MHz	Xilinx alveo U50	183.1ns

4.4 本章小结

本章主要将 FAST 协议解码设计分为提取 Pmap 值、字段域起始和终止位置标记、字段域切分、有效数据位合并以及字段域解码五个方面。首先通过设计并行、流水以及存储优化来降低总体的解码延迟，提高解码效率。在 Vitis 开发平台上，基于 OpenCL 和 HLS 指令级优化，在 Xilinx Alveo U50 加速卡实现解码的超低延迟，最终通过 Vitis 分析器与同环境 CPU 测试平台对比分析硬件资源利用以及单条数据解码延迟。此外，分析解码延迟的影响因素以及与其他文献不同实验的结果参数作对比。

第五章 基于 FPGA 的行情数据硬件解析器设计

5.1 行情盘口快照解析

5.1.1 行情盘口快照和委托字典

行情盘口信息是除指数外所有其他证券基本信息、10 档行情深度信息以及买卖一上的前 50 笔委托数量等的总称，其中的证券基本信息主要包含成交证券编号（symbol）、交易订单成交时间（nTime）、前一日收盘价（nPreClose）、开盘价（nOpen）、最高价（nHigh）、最低价（nLow）、最新成交价（nLast）等。行情盘口信息订单生成时间的范围为上午 9:25 至 11:30 以及下午 13:00 至 15:00。行情盘口信息主要反映了证券交易市场的波动和趋势走向，对用户策略的制定和投资意向选择起到重要的作用。行情交易的盘口信息整体上是连续的，伴随着逐笔委托和逐笔成交的不断撮合完成。目前交易所提供给用户的行情信息每 3 秒进行一次刷新，每次刷新生成的行情盘口数据被称为行情盘口快照。由于行情盘口快照的上述特征，导致交易决策的分析样本较少且时效性低，无法适应数量庞大且瞬息万变的交易市场。

委托字典是包含全部买卖订单交易价格和交易数量的集合，分为买方委托字典和卖方委托字典，两者相互独立。目前交易所提供行情信息中的部分买卖信息仅为委托字典的子集。由于委托字典是由逐笔委托和逐笔成交解析完成产生，因此在时效性上其拥有毫秒级的刷新速度，同时在数据深度上，其亦能够展现百档甚至千档买卖价格和数量，从而能够更为完整地反映市场的价格变化趋势，最终为投资者交易决策的制定提供更为全面的参考。

5.1.2 逐笔委托和逐笔成交

逐笔委托是由用户发起的订单请求，其主要结构体信息包含的属性有证券编号（Symbol）、委托订单生成时间（nTime）、委托单号（nOrder）、委托价格（nPrice）、委托数量（nVolume）、委托类别（chOrderKind）、委托代码（chFunctionCode）以及撮合机编号（setid）。其中业务发生日的数据类型为整型，表示数据最小单位为 1 毫秒，时间范围为上午 9:15 至 11:30 以及下午 13:00 至 15:00。例如 9:15 对应的表示方式为 91500000。委托价格为小数点后两位。委托类别分别是市价单（对应字段域值为“1”）、定价单（对应字段域值为“2”）以及本方最优（对应字段域值为“U”）。委托代码用以判断该订单的买卖方向，分为买方（对应字段域为“1”）和卖方（对应字段域为“2”）。逐笔委托单号是由撮合机撮合多个证券按照订单时间的先后产生顺序进行标号。在同一台撮合机下，即撮合机编号相同，对单支证券而言，其单号总体呈现递增的趋势，且唯一不连续。而对所有出自同一个撮合机的证券而言，其特征为总体递增、连续且唯一。其中，仅有限价单和本方最优需对委托字典进行更新处理。此外，每一笔限价单的逐笔委托价格均需要根据委托代码判断买卖方向从而对对应买卖的最优价格进行实时更新。本方最优作为一种特殊的限价单，消息信息中仅提供订单数量，无价格信息显示，因此，订单价格需根据当前买卖方最优价格进行更新。

逐笔成交是由交易所撮合机根据委托订单匹配生成,其消息结构体包含的主要属性有证券编号(symbol)、成交订单生成时间(nTime)、成交编号(nIndex)、成交价格(nPrice)、成交数量(nVolume)、成交类别(chOrderKind)、成交代码(chFunctionCode)、卖方委托序号(nAskOrder)以及买方委托序号(nBidOrder)。其中,成交时间范围为上午 9:25 至 11:30 以及下午 13:00 至 15:00。成交类别根据订单类型分为成交(对应字段域为“F”)和撤单(对应字段域为“4”)。当订单类型为成交时,买卖委托序号均有数值显示,需根据买卖委托序号索引订单价格对买卖委托字典进行更新。当订单类型为撤单时,买卖委托序号仅有撤单方向拥有数值显示,另一方显示为 0。同时对应的撤单价格为 0,需要通过索引单号获取实际的订单价格来对对应的委托字典进行更新。此外,每一笔成交单对应的成交价格数据需要对行情盘口的最新成交价进行更新。

5.1.3 行情信息解析

行情信息解析依靠逐笔委托和逐笔成交的逻辑判断和计算实现。处理订单的顺序按照每一条消息内的订单生成时间决定,而非接收订单的实际时间。由于订单生成时间的最小单位为毫秒,因此在交易高峰期(通常是上午 9:25 和下午 13:00 两个交易起始时间)会存在大量相同订单生成时间的订单。当逐笔委托和逐笔成交订单生成时间相同时,优先处理逐笔委托,后处理逐笔成交。逐笔成交和逐笔委托的本质都是通过订单价格和订单数量对委托字典进行更新操作。但是由于逐笔成交按照订单类型分为成交和撤单两种情况。其中撤单仅有数量,没有价格(消息默认价格信息为 0),因此需要对价格进行更新,可通过订单号索引到用户下单时的订单价格。在这一过程中,需要注意在处理每一条逐笔委托订单时,对价格和订单进行存储。此外,因为成交价格与用户申请订单价格存在偏差,因此也需要通过订单索引价格来完成对订单价格的更新。成交分为全部成交和部分成交两种情况。

盘口	价格	数量
卖五	10.05	200
卖四	10.04	200
卖三	10.03	200
卖二	10.02	200
卖一	10.01	200
买一	10.00	200
买二	9.99	200
买三	9.98	200
买四	9.97	200
买五	9.96	200

(a) 盘口一

盘口	价格	数量
卖五	10.05	200
卖四	10.04	200
卖三	10.03	200
卖二	10.02	200
卖一	10.01	300
买一	10.00	200
买二	9.99	200
买三	9.98	200
买四	9.97	200
买五	9.96	200

(b) 盘口二

盘口	价格	数量
卖五	10.06	200
卖四	10.05	200
卖三	10.04	200
卖二	10.03	200
卖一	10.02	100
买一	10.00	200
买二	9.99	200
买三	9.98	200
买四	9.97	200
买五	9.96	200

(c) 盘口三

图 5-1 行情盘口快照变化示意图

如图 5-1 所示。例如,此时用户下单了卖方限价单,金额为 10.01 数量为 100,则 会由 a 图变为 b 图。如果用户下单了买方限价单,金额为 10.02,数量为 200。则会由 a

图变为 c 图。用户将会以 10.01 的价格成交卖一 100 单，并以 10.02 的价格成交卖二的 100 单，同时由于卖一 10.01 的价格全部成交导致后四个档位依次顺延，原本的卖六作为卖五显示在盘口上。此外，如果用户的买方限价单价格金额为 15.00，数量为 200，实际成交依然如上所示，采用最优价格成交，且当成交完成时，一共产生两笔成交订单，需要对实际成交订单单号索引到用户下单价格，最终在委托字典中对数据进行计算操作。最终输出行情盘口快照信息为买卖委托字典、最新成交价格、股票编号和行情信息时间。

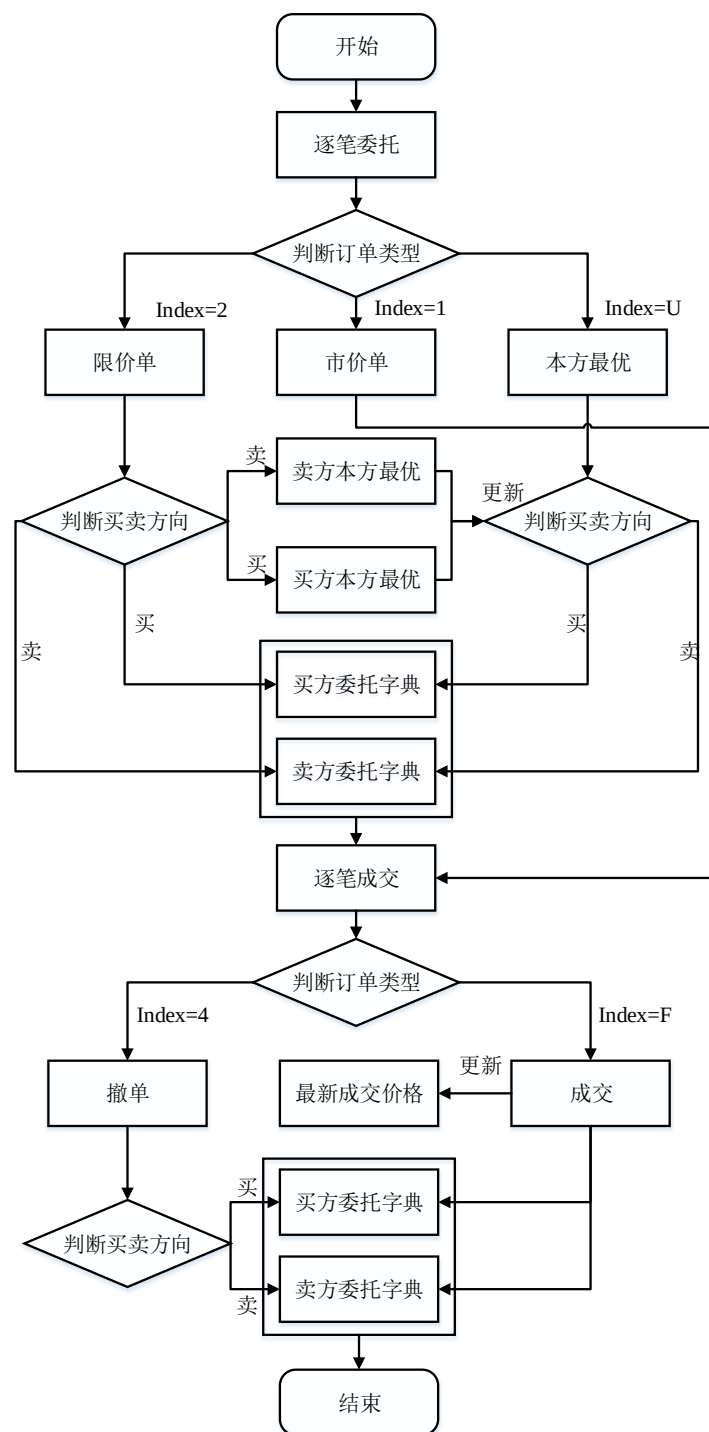


图 5-2 行情信息解析设计流程图

行情信息解析设计流程图如图 5-2 所示。以订单成交时间最小单位 1 毫秒为一个时间间隔，对逐笔委托和逐笔成交进行处理。先处理委托后处理成交。逐笔委托首先对订

单类型进行判断。如果是市价单不做任何处理，如果是限价单则需先判断买卖方向，从而对对应的委托字典进行处理。同时需要根据买卖方向更新本方最优价格。如果是本方最优，则根据本方最优价格对价格进行更新。完成以后通过买卖方向对对应的委托字典进行更新。当委托数据全部处理完成，开始进行逐笔成交数据的处理。通过判断订单类型来完成数据的处理，如果是撤单则判断买卖方向，并对对应的委托字典进行更新。如果是成交，则需要对买卖委托字典双方同时更新。此外，成交的价格还需要对最新成交价格进行更新。当该时刻成交价格全部完成后输出最终的买卖委托字典。

5.2 设计思路

5.2.1 基于 OpenCL 的主机端设计

行情数据经过 FAST 解码完成后，被接收存放在主机端内存中，以供后续的行情信息解析。因此，需要基于 OpenCL 完成对主机端的设计。通过 OpenCL 完成对逐笔委托、逐笔成交数据的读取、存储空间的应用和初始化以及内核调度和管理等工作。考虑到逐笔成交价格的特殊性，即需要通过订单号进行价格更新，因此，需要对每一笔逐笔委托的单号和价格进行存储，这导致 FPGA 加速卡的片上存储资源不足，所以存储空间内存分配申请主要涉及两部分。一部分是简单内存分配，用于存放逐笔委托、成交的数据。另一部分是外存 HBM 内存分配，用以存放每一笔逐笔委托的单号以及对应的价格。与 FAST 协议解码的主机端设计类似，首先读取委托和成交数据，并完成初始化操作，然后查询 OpenCL 设备，创建上下文以及命令队列，接着加载内核源代码，之后进行内存的分配和缓存映射。完成上述设置后，将内存从主机端迁移，然后创建内核对象，分别是逐笔委托、逐笔成交、买卖盘口、行情快照、委托价格单号键值表，接着将委托和成交数据写入内存对象，执行内核。等待完成后，读取行情信息，释放资源完成主机端的设置。基于 OpenCL 的主机端设计流程图如图 5-3 所示。

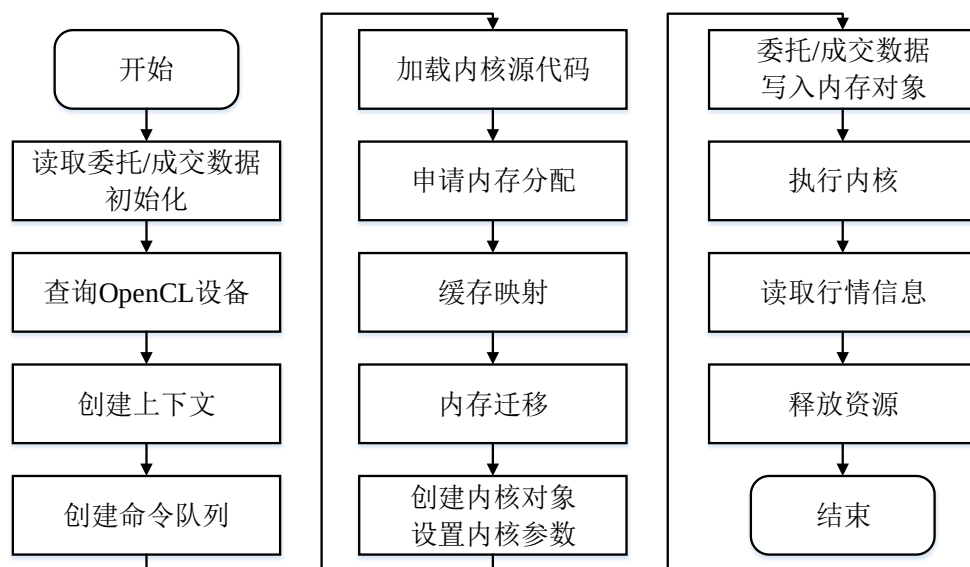


图 5-3 基于 OpenCL 的主机端设计流程图

5.2.2 硬件并行设计

针对行情数据撮合的规则特征,在逐笔委托和逐笔成交订单生成时间相同的情况下,遵循委托在前,成交在后的处理顺序。将逐笔委托和逐笔成交订单的处理分成独立的两部分。逐笔委托首先通过订单类型判断是否需要委托字典进行操作,如果需要操作,则再对买卖方向进行判断,委托字典分为买方委托字典和卖方委托字典。同一笔逐笔委托仅对一方委托字典产生更新操作。此外,需要对每一笔逐笔委托信息中的单号和价格进行对应存储,首先考虑低延迟查找和存储设计,采用直接哈希存储。以单号作为键(Key),价格作为值(Value),对键值表进行存储更新。由于以上两部分无关联,且相互独立,因此,可以采用并行设计处理,同时完成委托字典和键值表存储两部分操作,从而提升每一个逐笔委托单元处理模块的处理速度。逐笔委托单元模块框架图如图 5-4 所示。

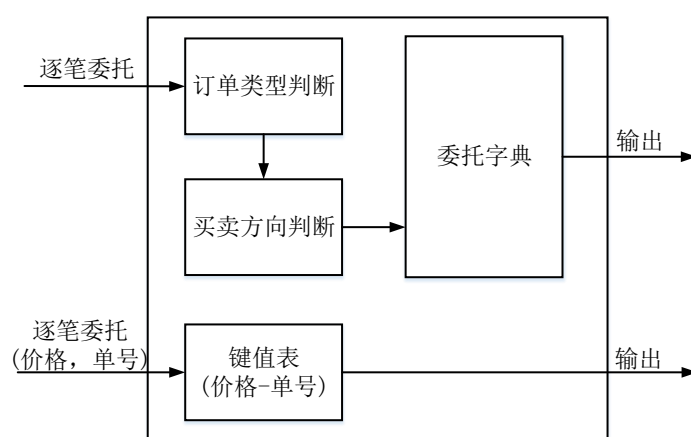


图 5-4 逐笔委托单元模块框架图

对于逐笔成交,由于实际成交的价格与委托价格存在偏差,需要先通过逐笔委托预先存储的单号价格键值表对成交单号进行索引,在查找到对应价格后对成交价格进行更新。然后根据逐笔成交的数量和价格对实际的委托字典进行更新操作。由于两者处理前后存在数据相关性,需要递进处理,所以需要首先更新成交价格,防止价格误差导致委托字典计算错误。逐笔成交单元模块框架图如图 5-5 所示。

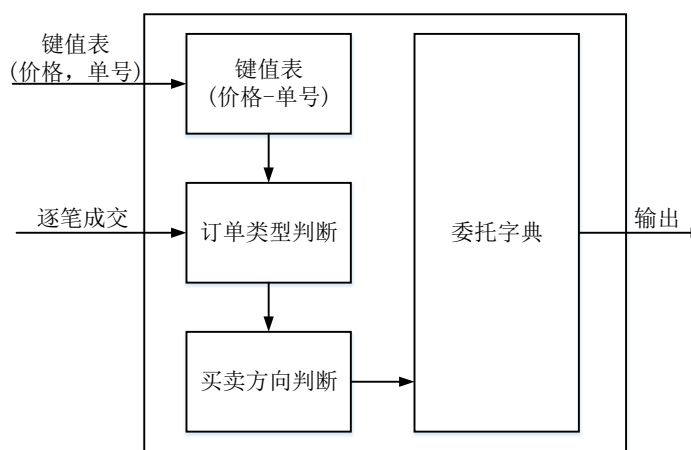


图 5-5 逐笔成交单元模块框架图

由于不同证券编号的行情数据相互独立，无耦合。因此，可以将收到的逐笔委托和逐笔成交数据按照股票编号进行分类。由于每一类股票处理相互独立互不干扰，因此可以采用并行的处理设计。每一支股票的处理均有独立的委托处理模块以及委托字典。同时逐笔委托的卖方和卖方订单处理仅针对各自的委托字典，两者在处理上也没有关联，也可以并行完成更新操作。同理，逐笔成交的成交也是分别对买卖委托字典进行更新，而撤单仅按照买卖方向进行单侧操作，因此其在处理上也是独立分开的。根据上述两种特征，分别对不同股票以及单支股票买卖双方的处理过程进行并行化设计，有利于提高委托字典更新处理的并行度，降低总体的行情数据解析延时。通过并行优化设计，计算设计并行度为 $2N$ ，其中 N 为并行行情数据解析的股票个数。行情信息解析并行性设计分析图如图 5-6 所示。

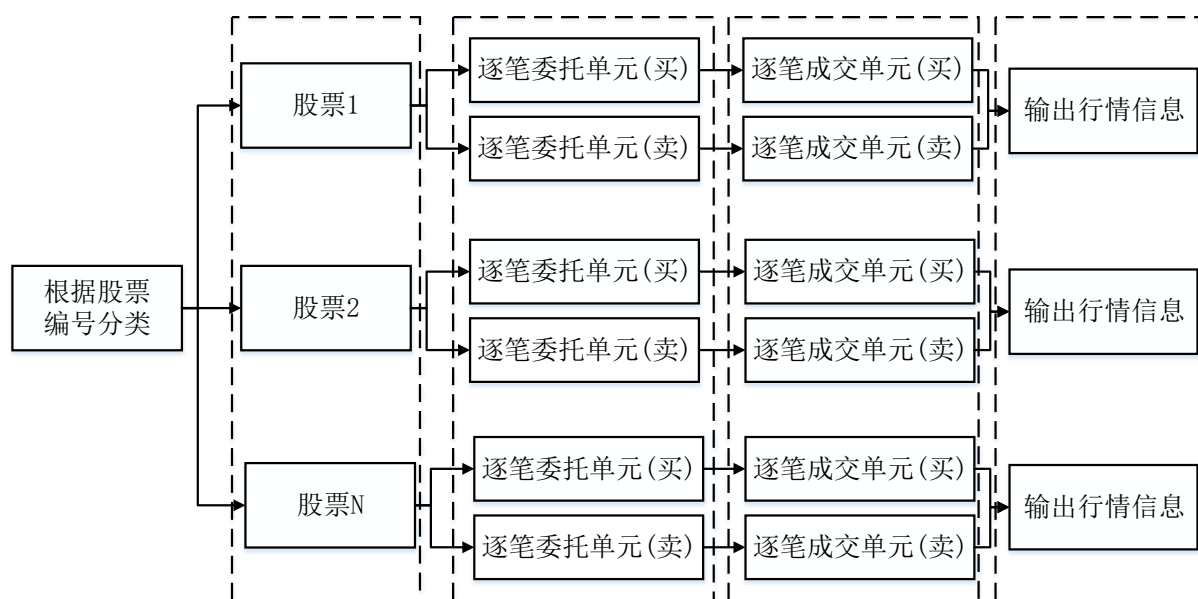


图 5-6 行情信息解析并行性设计分析图

以上的输出行情信息主要针对的委托字典的输出，除此以外还有输出行情信息时间、最新成交价等信息。其中输出行情信息时间与处理的委托成交时间一致，并非实际生成时间。而最新成交价格由每一单逐笔成交类型中的成交订单决定进行覆盖更新。通过将以上输出数据整理最终完成整个行情盘口快照信息的输出。

5.2.3 委托字典存储优化设计

根据行情数据的推断逻辑，逐笔委托的本质其实是对委托字典进行增添操作，而逐笔成交的本质则是对委托字典进行删减操作。其中主要涉及存储、查找、插入和运算等操作。其中运算仅涉及加减操作，延时消耗占比较小，主要耗时占比在键值的查找上，为了能够实现查找过程在时间上的最优，采用了哈希直接寻址，以订单价格作为键，该订单价格对应的交易数量总和作为值，构建订单价格数量的键值表。对于键值表的大小，因为接收到价格经过处理均为整型，最小单位为分，因此键值表的最高价格设置为当日最高价的 1.5 倍，从而确保不会因为数据溢出导致计算的错误异常。委托字典存储设计示意图如图 5-7 所示。

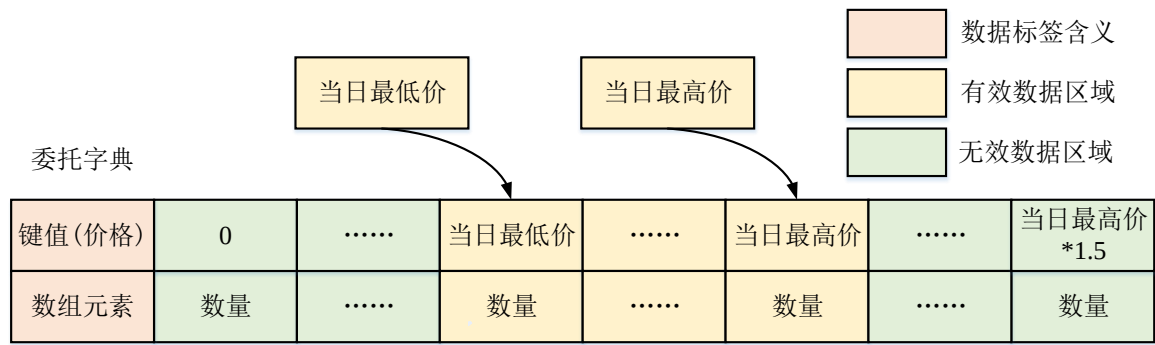


图 5-7 委托字典存储设计示意图

尽管这样的设置能够满足查找耗时的最优，但是对于数值较大的价格，由于实际价格存在范围区间，会产生大量的数据空间浪费，从而占用 FPGA 内部的存储资源，因此需要对委托字典的存储进行优化。考虑到每天股票的价格波动，委托字典的空间大小采用动态范围界定，利用股票价格涨停和跌停的交易规则，以前一日的股票收盘价为基准，向上浮动 10% 作为当日最高价，向下浮动 10% 作为当日最低价作为浮动范围界定键值表区间。同时，将当日最低价作为偏移量，确保在时间复杂度不变的基础上，提高数组空间利用率，以便 FPGA 能够拥有更多的资源空间，从而提高并行处理的股票数量，最终提高总体处理的解析速度。此外，对于前一日的收盘价参数需要在一天的股票行情信息处理完成后对其进行更新。委托字典存储优化示意图如图 5-8 所示。

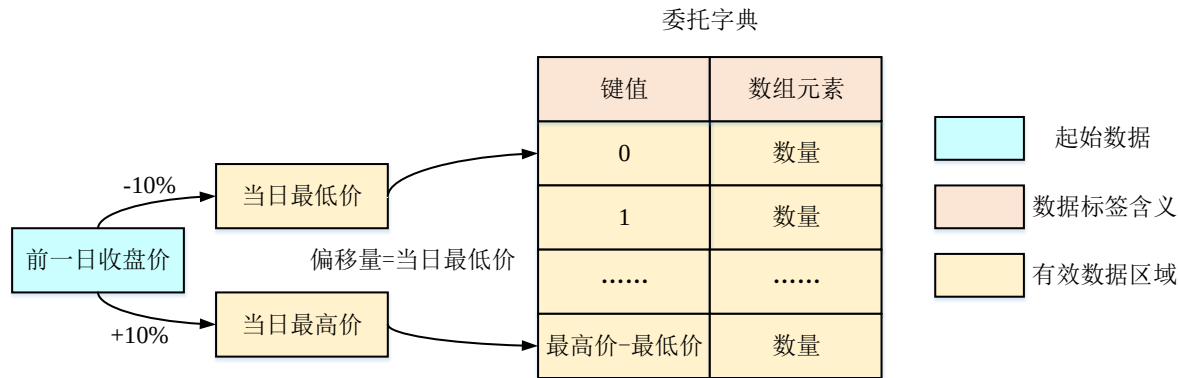


图 5-8 委托字典存储优化示意图

5.2.4 单号价格存储优化设计

由于逐笔成交价格的低时效性，在处理委托字典前需要借助单号索引价格对其进行更新操作，因此，需要对逐笔委托的单号和价格进行存储。逐笔委托的单号生成是由交易所的撮合机撮合完成，目的是防止数据在传递过程的丢失检测。因此，其特点表现在对于同一台撮合机撮合的多支股票，其单号连续且总体呈现递增的趋势，而逐笔委托中的通道编号即为撮合机编号。因此，对于单号价格的存储设计，与委托字典设计类似，亦采用哈希直接寻址法，以委托单号为键，对应的委托价格为值，建立单号价格键值表。键值表的个数则为通道个数。首先，根据逐笔委托数据的通道编号确定存储对应的键值表。键值表的长度通常设置为前一日最大单号数的 1.5 倍，由于单号具有连续不重复的特点，因此其键值表的利用率较高。同时访问存储的时间复杂度为 $O(1)$ 。单号价格键值表的存储方法 1 设计示意图如图 5-9 所示。

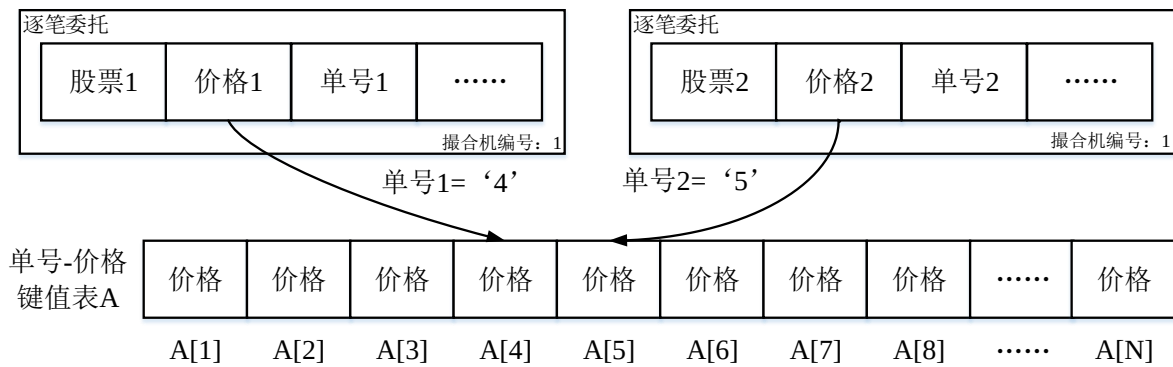


图 5-9 逐笔委托单号价格存储方法一示意图

尽管单条逐笔委托的数据访问很快，但是当同时多条数据并行处理访问存储时，由于键值表存储本身的接口访问瓶颈，这将会导致数据阻塞，并行访问存储受限。对此，可以通过为单支股票单独配置一个键值表存储，从而使股票与股票之间的访问存储相互独立，最终达到实现并行的目的。单号价格键值表的存储方法二设计示意图如图 5-10 所示。

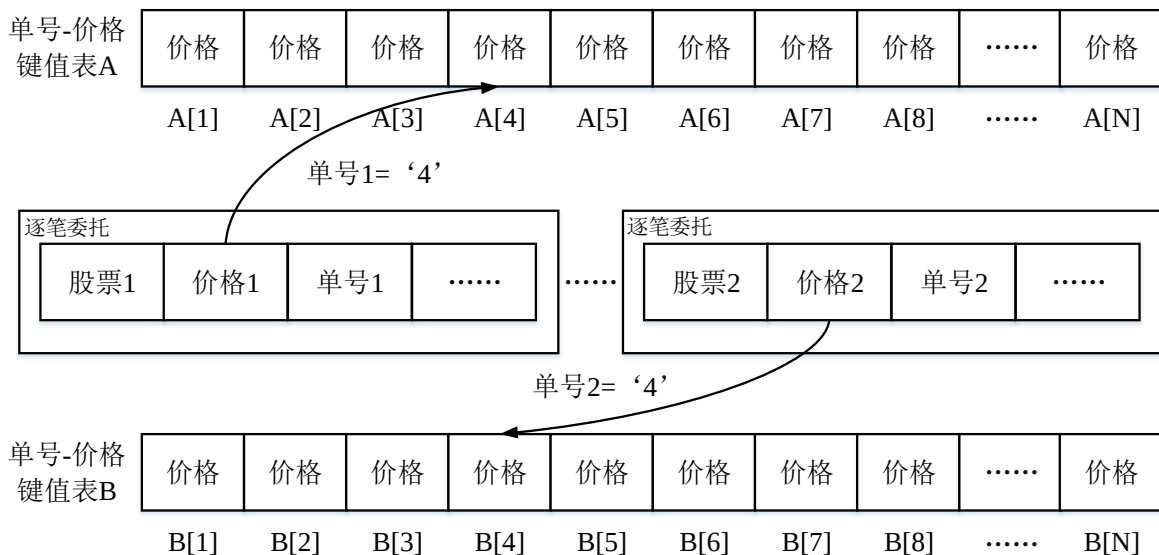


图 5-10 逐笔委托单号价格存储方法二示意图

尽管以上设计解决了并行访问存储的问题。但是，其缺点则是对于键值表的有效数据空间利用率极低，平均利用率甚至不足 0.2%。此外，由于 FPGA 本身的存储硬件资源有限，无法满足更多股票键值表的存储。因此需要兼顾并行度和存储空间进一步进行优化设计。

综合以上的优缺点，对逐笔委托的单号和价格的存储优化，便于后续的成交价格的查找更新。针对单支股票而言，其单号是跳跃且唯一的，但是总体上按照生成时间呈现递增的趋势。为了避免键值表的存储空间造成浪费，通过两个键值表来分别存储单号和价格，并用键值下标来将两者关联在一起，产生一一对应的存储关系，从而使存储空间的利用率提高。同时每一只股票均配置各自独立的两个键值表，以便后续的逐笔成交在进行索引更新价格的操作时能够并行处理，相互独立，从而避免不同股票之间对于键值

表的访问冲突。具体流程为逐笔委托按照以顺序号为键（顺序号按照单只股票的逐笔委托时间的顺序进行排列），单号为值存储在第一个键值表中，同时也以顺序号为键，价格为值存储在第二个键值表中。确保单号和对应价格这两者的键值相同。逐笔委托单号价格存储示意图如图 5-11 所示。

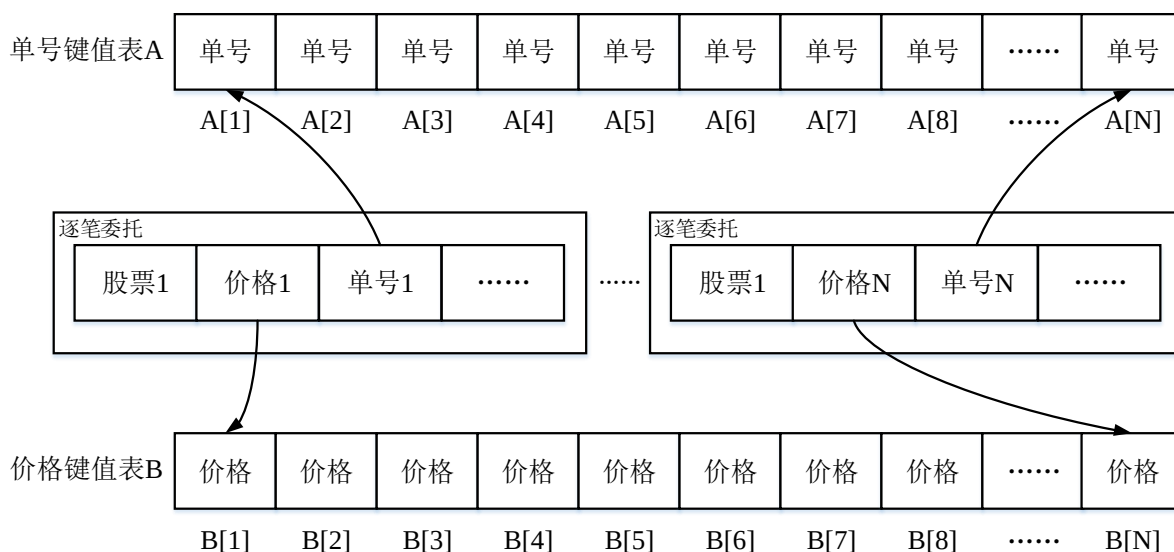


图 5-11 逐笔委托单号价格存储示意图

对于单支股票而言，其逐笔委托单号呈现递增的趋势，且单号唯一不重复。针对这一特征，在逐笔成交进行索引查找时对键值表采用二分查找，从而快速获取存储位置后对应到价格存储表最终获取更新价格。具体流程为逐笔成交按照成交对应的买方订单号或者卖方订单号索引第一个键值表，通过对表中数组元素的查找获取到对应的数组下标。再通过获取的数组下标作为第二个键值表中的键，通过直接索引的方式找到对应的价格，并对成交订单对应的价格进行更新，修正可能存在的价格误差。逐笔成交单号价格查找示意图如图 5-12 所示。

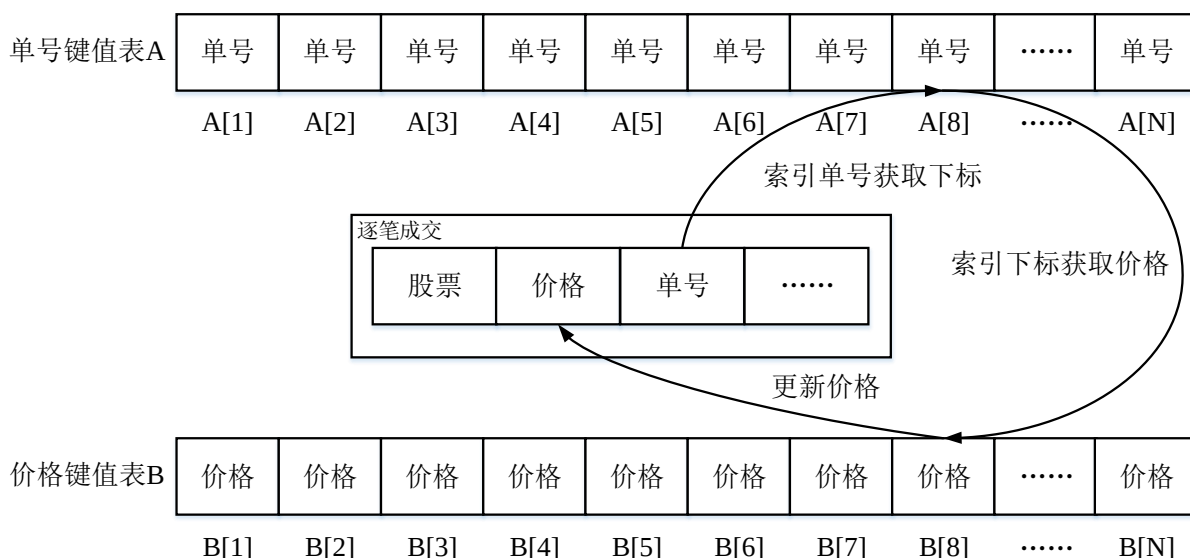


图 5-12 逐笔成交单号价格存储示意图

5.3 系统实现方法

5.3.1 基于 OpenCL 和 HLS 异构系统实现

针对行情信息解析设计, 总体框架采用 OpenCL+HLS 的设计模式, OpenCL 主要负责存储内存的调用和分配以及接口通信。主机端接收逐笔委托和成交数据后存放到内存中, 对行情解析数据进行预处理分类, 筛选掉非必要的信息, 仅留下需要进行内核计算加速的逐笔委托和逐笔成交数据。将数据经过 PCIe 搬运到全局存储器中, 通过内存映射将数据分配到对应的处理单元处理。其中 PE 作为工作项的处理单元, 拥有各自的私有内存用以存储需要计算的中间数据。后续将 Xilinx Alveo U50 FPGA 加速卡处理完成的数据返回并存放至全局存储器中, 再由 OpenCL 将全局存储器中的数据通过 PCIe 传回主机端内存中, 完成对行情数据核心的解析操作。基于 OpenCL 的总体框架图实现图如图 5-13 所示。

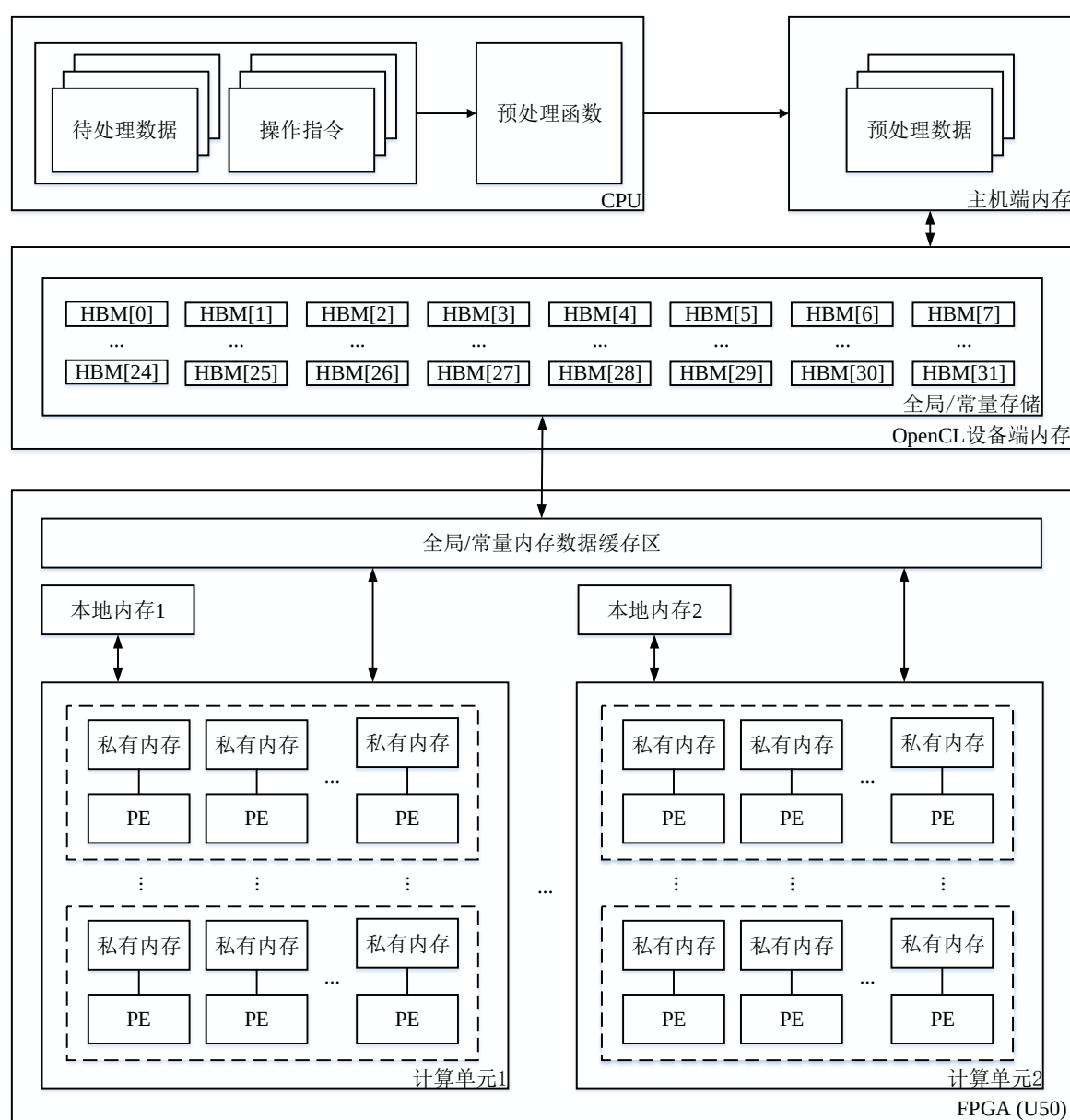


图 5-13 基于 OpenCL 的总体框架图实现图

5.3.2 存储优化实现

数据存储主要分为静态存储和一般存储。其中静态存储需要维持一整天的行情证券信息处理存储任务。因此，在存储上是一个累计的过程，对存储空间的要求较高，在行情回放解析的实现中主要是逐笔委托单号价格的键值表存储。通过数据计算估计发现其存储空间需求远大于 Xilinx Alveo U50 加速卡上的片上 SRAM 存储资源，SRAM 仅有 28MB 的存储空间不足以满足存储的需求。因此，对于单号价格的键值表存储规划存放至加速卡的片外 HBM 上，发挥 HBM 高容量的空间存储优势，从而容纳更多的股票，提高多支股票处理的并行度。而对于一般的存储，即仅参与一次运算，当运算完成后，接收到的第二条数据可以对上一条的运算存储空间进行覆盖。因此，这一部分的数据占用的数据空间较小，且需要进行数值运算。此外，涉及一些特殊数据的存储，如最新成交价、买方最优价格、卖方最优价格等小数值变量，一方面其存储空间需求较小，另一方面在进行解析数值计算时，对这一部分数据需要进行频繁的读写。因此，通过将这部分数据的存储空间放在加速卡的片上 SRAM 上，发挥 SRAM 高带宽的数据传输优势，提高数据处理速度，降低行情数据解析的延迟。

Xilinx Alveo U50 加速卡在硬件上拥有 8G 的 HBM 存储空间，软件配置上分为 32 个存储单元块，每个存储块拥有 256M 的存储空间。在程序存储分配上，需要人为地对存储区域进行划分映射，将不同编号的股票按照委托的数量进行分类，合理地利用每一个 HBM 存储块的空间，将单号价格键值表分散存储至不同的 HBM 存储块中，从而使得在逐笔成交价格索引更新处理时，能够尽量减少数据的访问冲突，最终提高并行读取和访问 HBM 存储的并行度。此外，考虑到后续可能的数据范围浮动以及后续的存储拓展可行性，将每一个 HBM 存储块预留 50% 的可拓展空间防止存储数据的溢出，确保每一个存储块的总计算估计数据存储空间不超过 150M。具体存储框架实现图如图 5-14 所示。

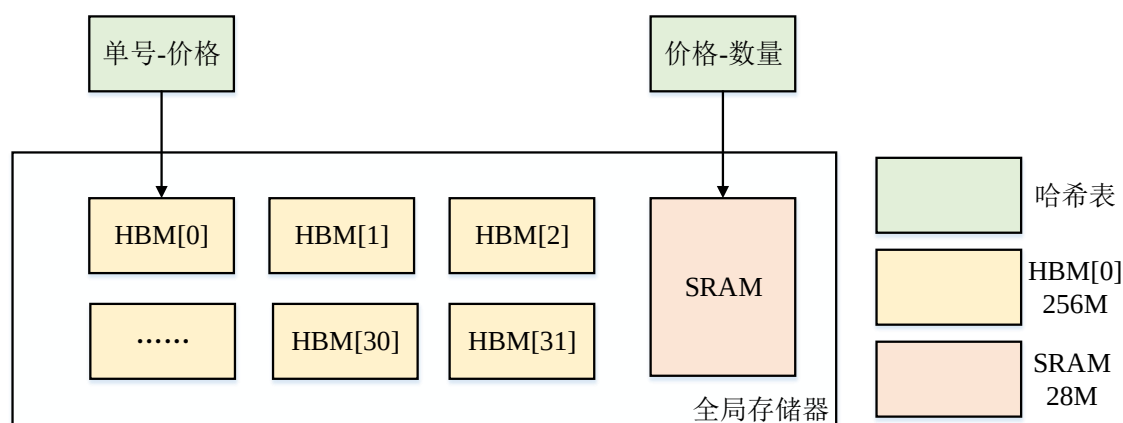


图 5-14 存储框架实现图

5.3.3 委托成交处理内核复制并行实现

针对顶层函数下未经函数封装的循环操作以及封装子函数内部的循环操作。默认情况下，HLS 会对循环进行迭代，每一个循环会被当作一个单独的个体。循环中的所有操作，如读取、计算、写入操作都会被相同的硬件资源迭代实现，这将使得在数据处理上

呈现顺序处理，从而导致处理延迟较长。为此，通过 HLS UNROLL 指令可以将循环展开。Unrolled 优化示意图如图 5-15 所示。

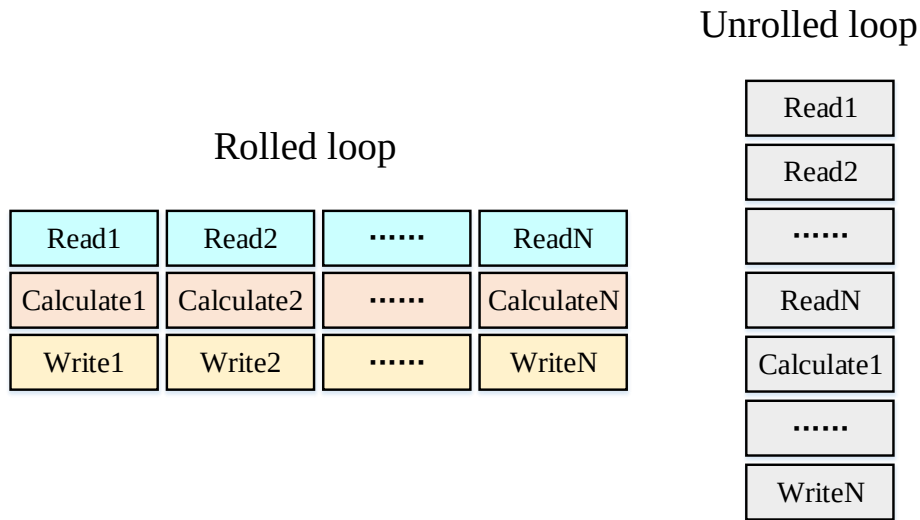


图 5-15 Unrolled 优化示意图

如图 5-15 所示，左侧图为默认迭代的情况，同色区域表示采用了同一个硬件资源完成对应的操作。当计算所需要的时钟周期数为 1 时，每一个迭代需要单独的时钟周期，总共需要花费三个时钟周期，同时需要计算所需要的硬件资源和一个单端口的 BRAM，而通过循环展开指令，即如右侧图所示，所需要的资源数为原来的 N 倍。但是，这也导致对数据的读写操作也相应变成了原来的 N 倍。如果读写的内容来自同一个 BRAM，而 BRAM 最多仅有两个端口，因此，通常在 UNROLL 展开的同时，也需要对数组进行 Partation 展开，使得整体的处理并行度是原来的 N 倍，从而大大减少顺序处理所消耗的时延。

针对多支股票的并行处理，通过 HLS 对单个处理单元进行内核复制。复制的个数根据股票的个数和 FPGA 加速卡内部资源数共同决定。在资源数以及全局内存输入带宽足够的情况下，可以根据用户的需求，最大限度地提高并行处理内核的吞吐量。

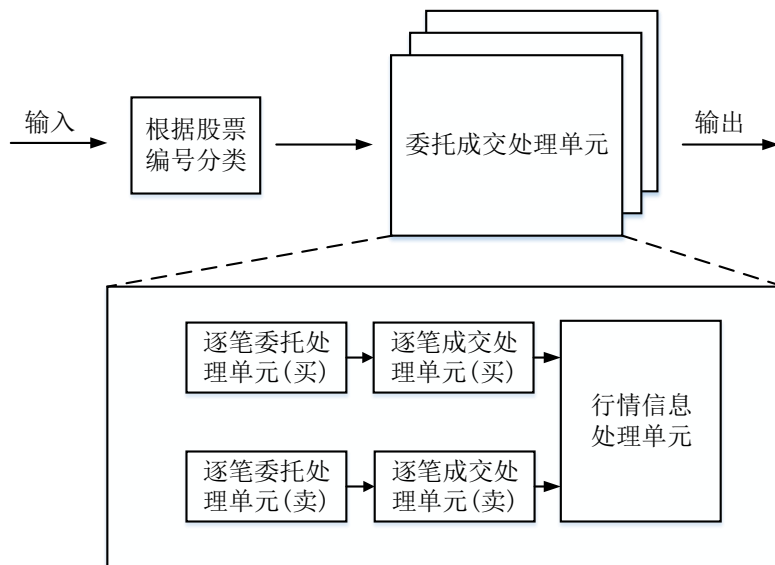


图 5-16 内核复制实现图

如图 5-16 所示。逐笔委托和逐笔成交数据按照时间戳的前后顺序通过股票单号字段进行股票编号分类。通过股票编号的个数确定委托成交处理单元复制的个数，从而达到多路并行处理的目的。理论上，在硬件资源足够的前提下，委托成交处理的并行度与实际复制单元个数一致。针对单个委托成交处理单元，根据数据对应字段判断买卖方向，将逐笔委托处理单元和逐笔成交处理单元复制为两份，分别处理买方和买方对应的数据。通过复制买卖内核，理论上可以将单个委托成交处理单元的处理时延缩短至原来的一半，从而进一步提升整个行情数据解析的吞吐量。

5.3.4 成交委托模块流水处理

针对逐笔委托和逐笔成交的模块设计，由于逐笔委托存储和计算相互独立，因此可以并行处理，而逐笔成交的特点是先通过成交单号查询委托存储的单号价格键值表来获得对应的价格从而对传输的价格字段进行更新操作。采用硬件流水设计思想，通过 HLS 优化指令来实现。HLS 针对流水设计给出了 Pipeline 和 Dataflow 两种优化指令。其中 Pipeline 主要应用在函数内部以及顶层函数下未封装的循环体，通常与数组分割 Partion 指令联合使用，提高数据吞吐量。Dataflow 主要应用于函数和函数之间。通过设计将逐笔委托主要分为三部分处理，分别为查询、处理以及存储，使得在查询更新价格的操作完成后，成交处理完成前，对下一个数据的价格进行查询更新操作。HLS 会分析封装函数的数据流，在它们之间创建通道，对于标量变量，通道类型通常为 FIFO。而对于非标量变量，如数组变量，则采用乒乓（PIPO）缓冲器。每一个通道均含有信息指示来判断 FIFO 或者乒乓缓冲区的状态是满还是空。通道的存在使得在成交操作完成前即可开始下一条数据的操作，从而降低行情数据解析延迟，提高数据的吞吐量。Dataflow 硬件流水实现流程图如图 5-17 所示。

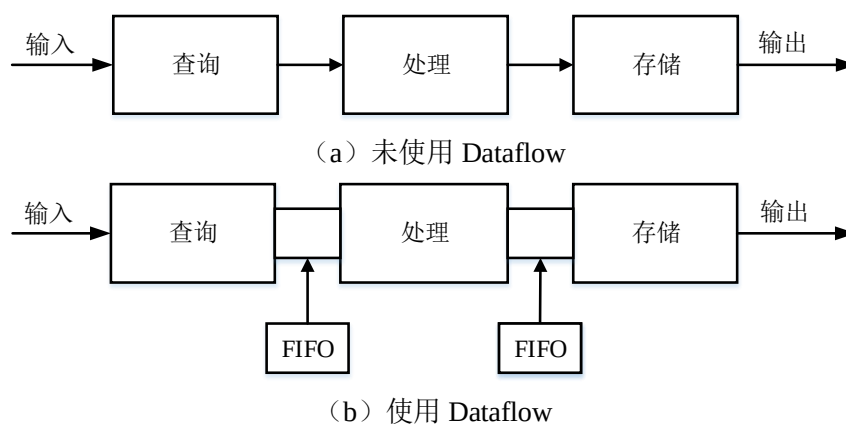


图 5-17 Dataflow 硬件流水实现流程图

5.3.5 二分查找硬件优化实现

针对逐笔委托单号的查找，由于其单号递增且唯一无重复的性质，因此在查找方式上采取二分查找。二分查找的时间复杂度为 $O(\log_2 N)$ ，因此其查找所需要的时钟周期数为 $\log_2 N$ ，其中 N 为键值表的数组元素个数。为了能够进一步缩短查找所需要的时钟周期，采用 FPGA 资源换时间的方式，通过分治和递归的思想将键值表分成两份，分别

并行地对两个子键值表进行二分搜索，从而将所需时钟周期数降低为 $\log_2(N/2)$ 。二分查找硬件优化示意图如图 5-18 所示。

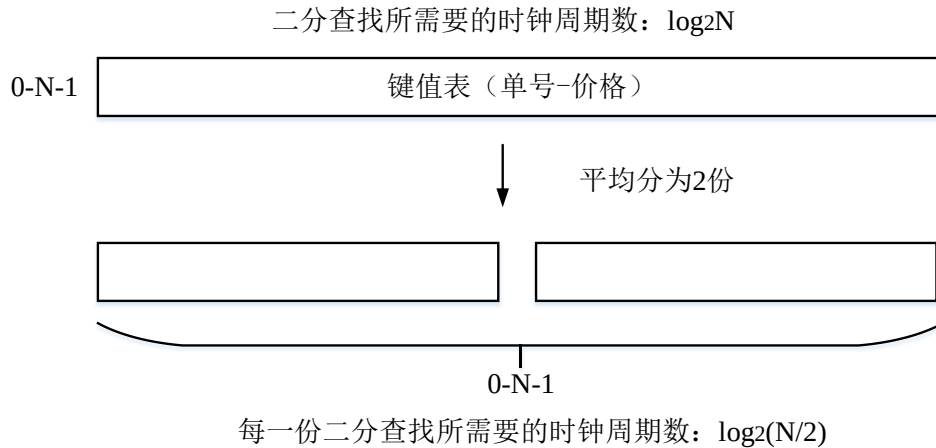


图 5-18 二分查找硬件优化示意图

理论上，键值表的划分个数越多，其所需要耗费的时钟周期数也越少，但对 FPGA 内部资源的消耗也越大。由图 5-19 所示，横坐标为减少查找循环的周期个数，纵坐标表示资源消耗的数量，这里主要涉及的资源为 FIFO 和查找表（LUT）。通过测试发现，消耗的资源会随着周期数的减少呈现指数级的增长，然而减少的周期数同样需要指数倍的分割数。减少三个时钟周期数需要将原键值表切分成四份，同时也导致 FIFO 和查找表资源消耗数的激增。因此，考虑到硬件资源的利用合理性，仅将键值表分成了两份分别进行查找操作。从而通过增加少量的硬件资源来获取更低的时钟周期数，最终降低查找所需要的时延消耗。

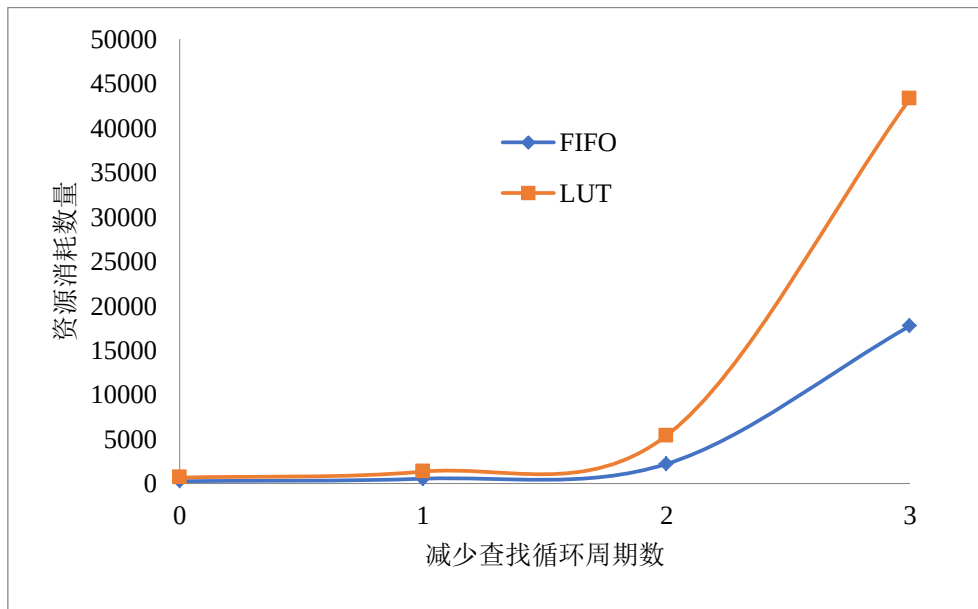


图 5-19 二分查找资源利用变化图

5.4 实验结果与分析

5.4.1 实验平台环境和硬件资源消耗统计

实验数据来源于行情商接收的证券数据，包含逐笔委托、逐笔成交以及行情数据。分别筛选出不同市值的股票进行测试。

FPGA 硬件测试平台：采用 Xilinx Alveo U50 加速卡，实际工作频率：300MHz，具备 8G 的 HBM 和 28MB 的 SRAM。在 Xilinx Vitis 2020.1 的开发平台上完成仿真、上板的测试工作，通过平台的分析报告分析测试内核设计的性能。

软件测试平台为 CPU：I9-9900X 10C20T，主频为 3.5GHz，内存大小为 64G。

由于行情解析的过程主要涉及对哈希表的访问、查找、插入和计算，因此并未使用到 DSP 资源。同时通过存储优化，一支股票仅占用了 0.8% 的 BRAM 存储，因此单支板卡能够容纳近百支股票的存储。FPGA 主要硬件资源使用表如表 5-1 所示。

表 5-1 FPGA 主要硬件资源使用

硬件资源	使用	可用	占比
BRAM	22	2688	0.8%
DSP	0	5952	0%
FF	7960	1743360	0.4%
LUT	34111	871680	3.9%

5.4.2 行情解析延迟抖动性测试

针对单支股票进行测试解析一条行情数据信息所需要的时间，通过测试三十次数据样本测试，CPU 行情解析由于受到中断机制和资源调度影响，行情解析延迟在 11000ns 至 13000ns 之间波动，波动幅度较大。而 FPGA 的延迟波动主要受二分查找次数影响，波动幅度在 1~4 个时钟周期以内。由于 FPGA 工作频率为 300MHz，因此抖动幅度控制在 14ns 以内。行情数据解析延迟抖动性测试对比图如图 5-20 所示。

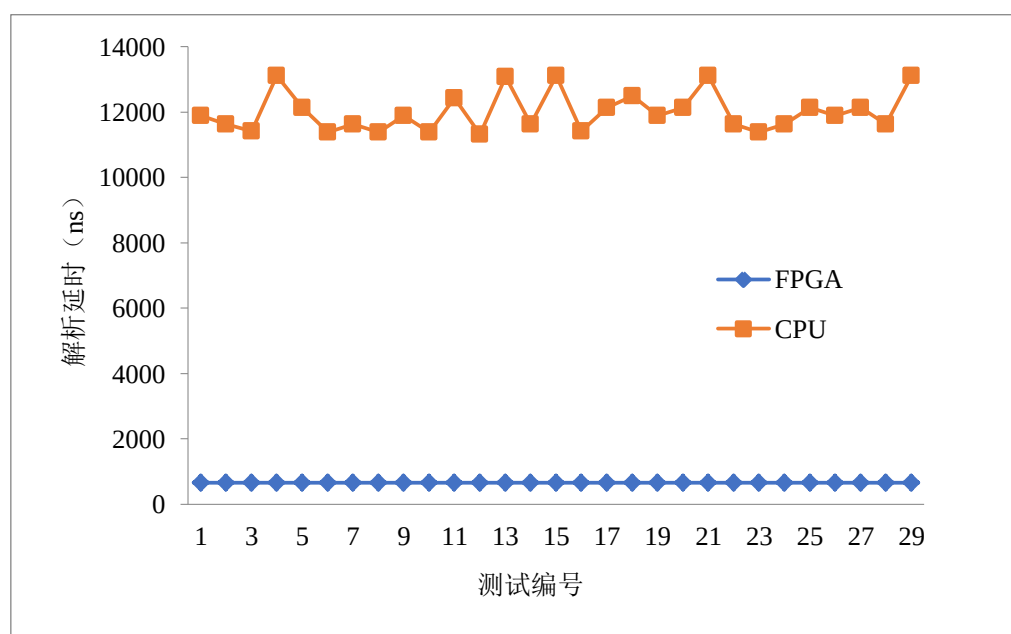


图 5-20 行情数据解析延迟抖动性测试对比图

5.4.3 行情解析延迟影响因素测试对比

通过对不同股票收盘价与行情解析延迟之间的关系进行分析,行情解析延迟主要受二分搜索的延迟和股票收盘价的大小影响。此外,由于二分搜索所消耗的延迟占总体的比例偏小,所以拟合曲线受股票收盘价影响,呈现线性的递增趋势。行情数据解析延迟影响因素测试对比图如图 5-21 所示。

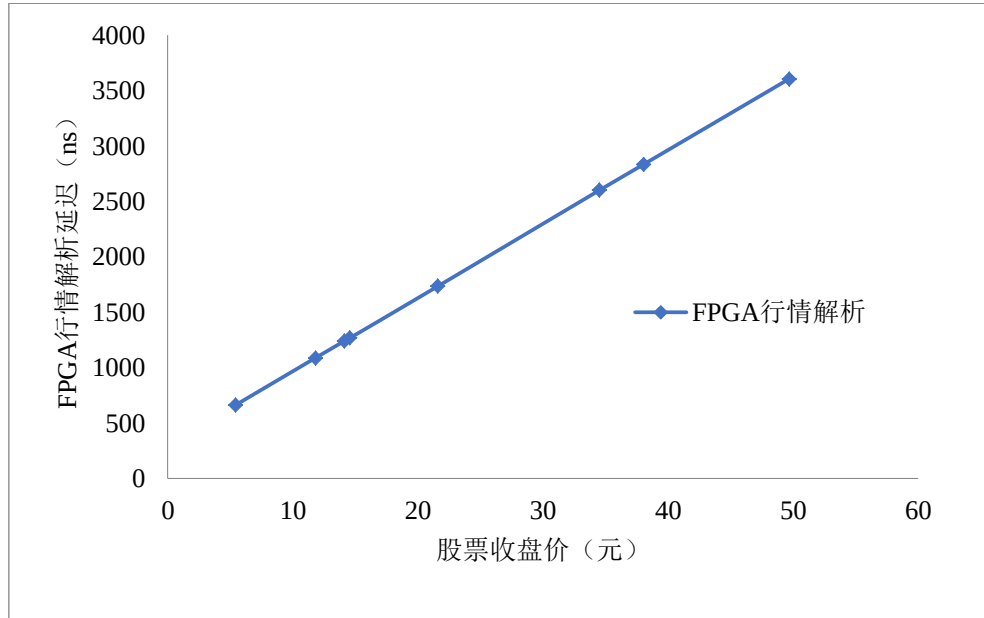


图 5-21 行情数据解析延迟影响因素测试对比图

通过拟合关系和设计分析推出行情解析延迟的关系表达式,如公式(5-1)所示。其中 $T_{Latency}$ 为行情解析的延迟,单位为 ns。由于初始化准备工作以及处理逐笔委托的过程相对固定,通过设计推断和 Vitis HLS 分析报告综合获得该部分的耗时始终为 9 个时钟周期。 x 为二分查找的数组长度, T_{clock} 为一个时钟周期的延时,由于实验硬件板卡默认主频为 300MHz,因此 T_{clock} 为 3.33ns, P_{close} 为股票前一日收盘价。

$$T_{Latency} = T_{clock}(9 + 2 \log_2(x) + [P_{close} \times 1.1] - [P_{close} \times 0.9]) \quad (5-1)$$

5.4.4 行情解析延迟对比

实验通过哈希表优化实现,包含插入、查找、哈希表计算等步骤,通过 HLS 实现需要 57 个时钟周期,对比国外实现所需 79 个时钟周期^[29],在同样采用 HLS 开发的情况下,解析速度提升了 27.8%。此外,在相同频率下,HLS 的解析延迟理论可以达到 142.5ns,与 Verilog 开发相比,其仍然存在一定的优势。通过对比国外行情数据解析延迟对比分析表如表 5-2 所示。

表 5-2 行情数据解析延迟对比分析

工作	开发语言	频率	器件	解析延迟
文献 ^[28]	VHDL	150MHz	COMBO-LXT	253ns
文献 ^[29]	HLS	400MHz	XCVU13P-FHGA2104	197.5ns
文献 ^[29]	Verilog	400MHz	XCVU13P-FHGA2104	155ns
本文	HLS	300MHz	Xilinx Alveo U50	189.8ns

5.5 本章小结

本章主要是基于 FPGA 的行情解析设计。首先描述行情解析所需要的参数数据以及最终生成的盘口逻辑。根据逻辑设计行情解析模块，重点体现在逐笔委托单号价格键值表的存储以及逐笔成交单号索引价格的查找算法优化上。存储设计上采用了 KVS 存储设计，在确保延迟优先的前提下，对存储空间利用率进行了优化，从而节省了所需要的硬件资源。在查找上，采用了二分查找的搜索算法并对其进行了优化。最终通过 HLS 指令级优化和 OpenCL 在 U50 加速卡上完成并实现了行情盘口快照解析并通过实验从延迟和抖动性上与 CPU 进行对比测试。

第六章 基于 FPGA 的量化金融计算加速系统

6.1 基于 FPGA 的量化金融计算加速系统软硬件实验环境

6.1.1 主机端测试平台

主机端为传统的 X86 架构服务器，搭载了 linux 操作系统，版本为 Centos 7.7.1908，CPU 为 Intel i9-9900X 10C20T，主频 3.5GHz，内存容量为 64G。Vitis 版本为 2020.1，XRT 版本为 202020.28.743_7.4.1708。交换机采用华为 S5720-28X-LI-AC。

主机端数据测试获取上，采用 Wireshark 抓包软件对端口 9129 进行抓包，并将抓包完成的数据存放起来。通过主机端程序存储并等待 FPGA 处理数据完成后对其进行对比，确保数据的正确性。Wireshark 抓包图如图 6-1 所示。

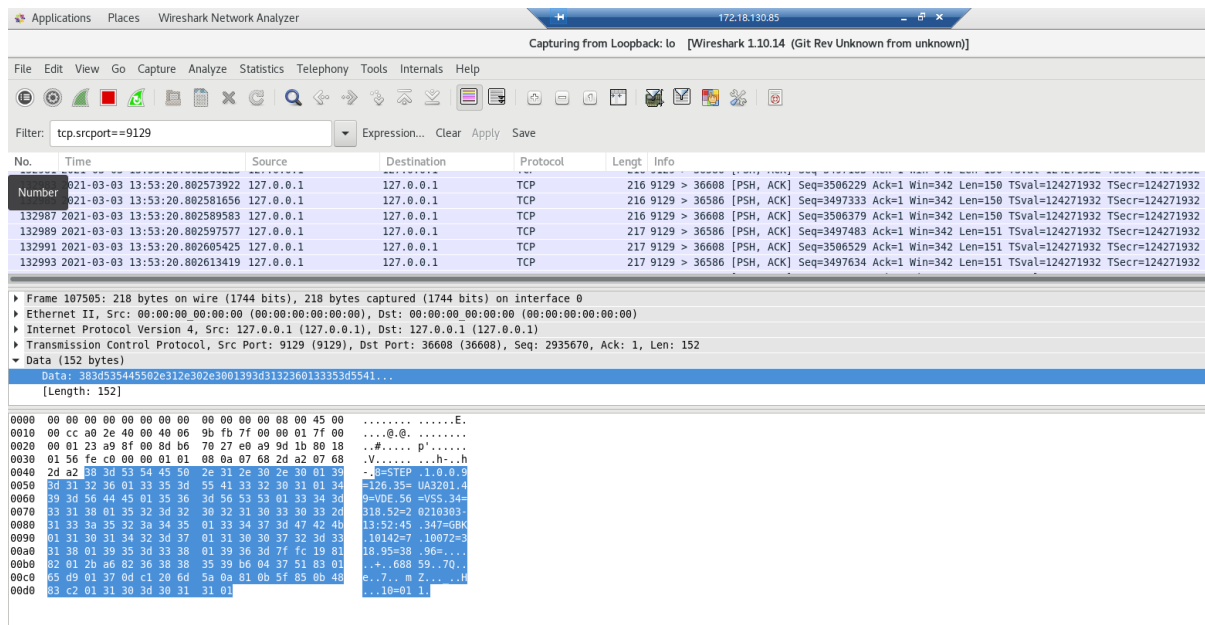


图 6-1 Wireshark 抓包图

6.1.2 FPGA 端测试平台

FPGA 端测试平台采用是 AMD Xilinx 推出的数据中心加速卡 Alveo U50 加速卡，旨在能够应用在如金融计算、机器学习等计算密集型应用领域^[76]。由于加速卡内无处理芯片，因此需要和传统 CPU 构成片上系统（System on Chip，SoC）。硬件测试平台一共部署了两块 U50 加速卡。Alveo U50 加速卡采用单卡槽设计，支持 PCI Express（PCIe）Gen3x16 或双 Gen4x8，同时，配备了 8GB 高带宽存储器（HBM2）和以太网网络功能。由于 U50 加速卡在运行时的最高功耗仅为 75W，因此在散热上采用的被动散热方案。蓝色指示灯亮起表明板卡处于正常工作状态。FPGA 硬件平台测试环境部署实物图如图 6-2 所示。



图 6-2 FPGA 硬件平台测试环境部署实物图

Alveo U50 加速卡主要由 PCIe 连接器、网络接口、卫星控制器(Satellite Controller)、维护接口(Maintenance Port)等组件构成。Alveo U50 加速卡内部组件原理图如图 6-3 所示。

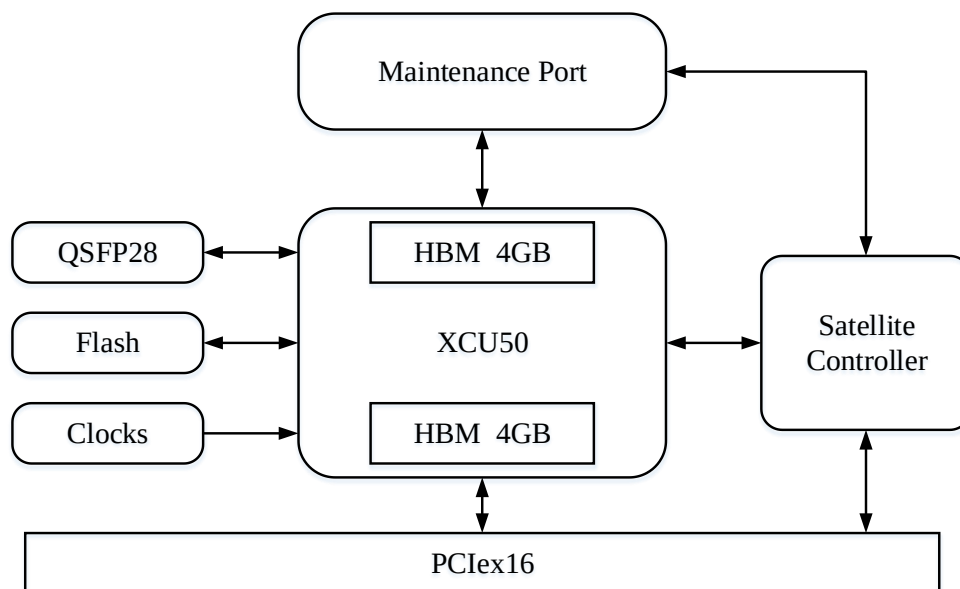


图 6-3 Alveo U50 加速卡内部组件原理图

在 PCIe 连接器上, Alveo U50 加速卡采用的 UltraScale+™ FPGA 包含了一个 PCIe4C 块。该 PCIe4C 支持 PCIe Gen3x16 的规格, 数据传输速率最高可达 8.0GT/s。此外, 还兼容 PCIe Gen4x8 规格, 支持高达 16.0GT/s 的传输速率。但是由于 Vitis 环境目前不支持四代线速, 所以实验采用的是 PCIe Gen3x16 的规格。

在网络接口上, Alveo U50 加速卡配备了四通道的 QSFP28, 能够接入高达 5W 的模块。QSFP28 可使用光学模块或者线缆连接的方式达到 100G 的接口通信。为了便于启动不同的以太网 IP 核, 为 QSFP28 接口提供了 161.1328125 MHz 时钟。

此外, Alveo U50 加速卡搭载了 TI MSP432 卫星控制器用以监测和控制电流、电压以及温度, 且可与主机服务器开发板管理控制器 (BMC) 交互, 并通过频带外通信来控制检测加速卡。维护端口上, 配置了 30 针的连接器, 用以支持 JTAG、UART、PMBus、RESET 等多种不同的功能和信息。Alveo U50 加速卡硬件资源参数如表 6-1 所示。

表 6-1 Alveo U50 加速卡硬件资源参数

规格	LUT	FIFO	DSP	GTY	36Kb 块 RAM	UltraRAM
资源	872K	1743K	5952	20	1344	640

6.1.3 系统测试对比方案

测试方案主要采用软硬件对比测试, 因此需要构建两个量化金融计算系统。一个为软件量化金融计算系统, 核心构建为高性能 CPU。同时其也作为行情数据的发送端。系统将 Level-2 行情数据发送给华为 S5820 光口交换机, 并由交换机将数据镜像复制打包发给软件量化金融计算系统和 FPGA 量化金融计算系统。分别测试两者从接收光口交换机接收行情数据并处理完成, 最终由 CPU 打印输出所需要的延迟消耗。测试环境部署示意图如图 6-4 所示。

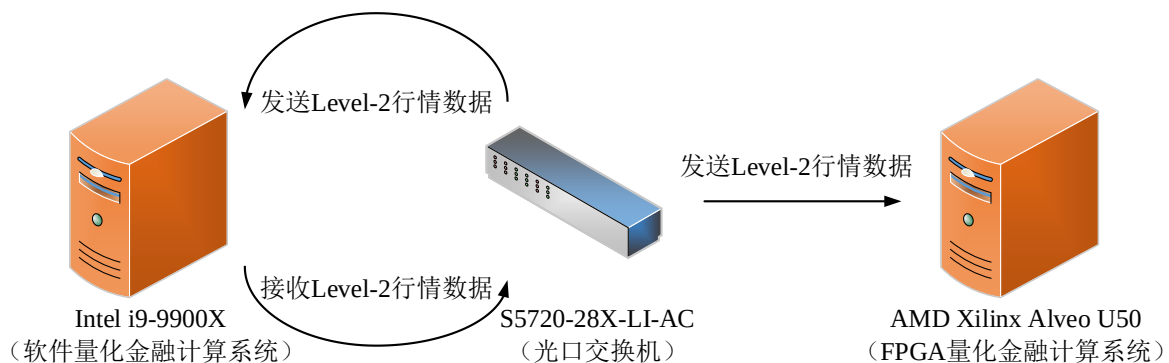


图 6-4 测试环境部署示意图

6.2 基于 FPGA 的量化金融计算加速系统设计与实现

6.2.1 量化金融加速系统总体框架设计与实现

基于 FPGA 的量化金融计算加速系统框架如图 6-5 所示。主要分为主机端设计和 FPGA 端设计。其中主机端主要是基于 OpenCL 的设计, 其主要作用是完成存储空间的申请、初始化和分配以及内核的调度和管理等工作。而 FPGA 端设计主要分为两大部分, 分别是网络协议卸载模块和量化金融计算加速模块。其中网络协议卸载模块采用了 Alveo U50 加速卡上配备的四通道的 QSFP28, 共计 40G 的网络传输接口通信。将 TCP 协议从 CPU 卸载到 FPGA 上, 主要通过 TCP 卸载引擎 (TCP Offload Engine, TOE) 完成 TCP 的卸载功能。而针对量化金融计算加速模块, 分为两个部分一个是 FAST 协议解

码模块，另一个是行情盘口回放模块。其中 FAST 协议解码模块首先需要对 TOE 处理完成的数据进行预处理操作，主要是从行情数据中提取字段域为 FAST 数据的部分和 FAST 数据开头的 Pmap 值为后续的解码做好预处理工作，然后依靠 Pmap 值的位元对 FAST 数据进行标记、切分、合并、解码等操作。最终把处理完成的行情数据送至行情盘口回放模块。行情盘口回放模块通过分类接收不同证券编号的逐笔委托和逐笔成交数据。并将其按照先处理委托后处理成交的顺序对委托字典进行更新计算。最终更新盘口数据，并将数据通过 OpenCL 返回至主机端内存进行校验分析比对。

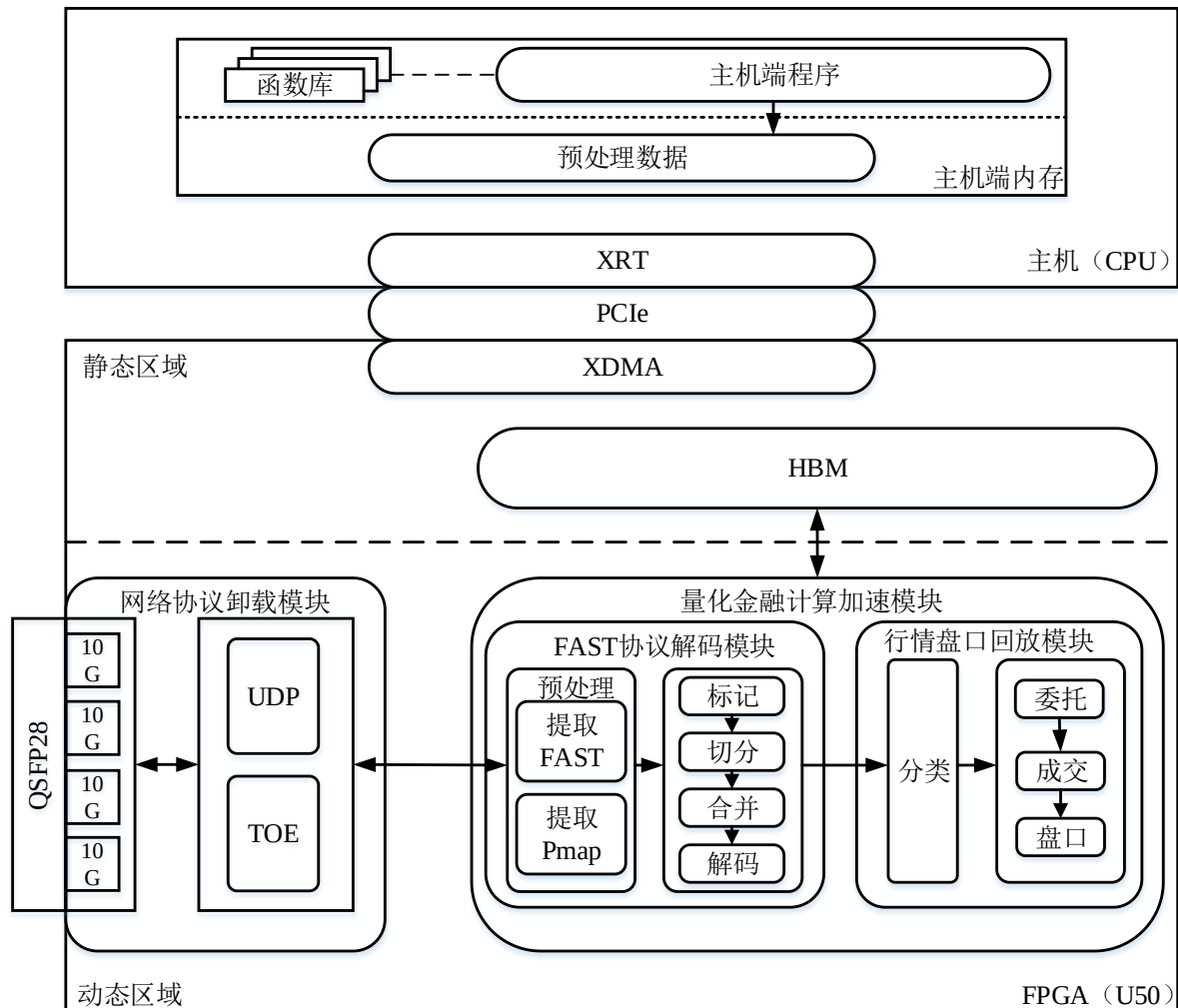


图 6-5 超低延迟量化金融计算加速系统设计框架图

6.2.2 基于 OpenCL 的量化金融加速系统主机端设计

实验过程中的主机端数据交互采用 OpenCL API 完成。在配置 AMD Xilinx Alveo U50 加速卡的过程一般为一次性操作，即卡的配置一旦完成会持续保持，直至断电或者加载固化其他内核进行重配置。在加入了 XRT 和 OpenCL 的报头后便可以使用库中的 API 函数^[77]。

针对主机端设计，首先需要初始化命令队列，然后加载二进制文件并将其编程到 FPGA。完成以上操作即可算完成预处理操作。后续需要通过 OpenCL 对内存进行申请和分配。由于行情盘口快照解析过程中需要的存储空间过大，片上存储无法满足存储需求。因此，将哈希表存储空间申请在 U50 加速卡的 HBM 上。对于内存占用较小且访问

频繁的数据，将存储空间申请在片上 SRAM 上，利用 SRAM 高带宽的优势提高数据的访问速度，从而提高数据的处理速度。内存申请分配完成后，将申请的内存发送到 Alveo 的全局内存中，通过 CL_MEM_USE_HOST_PTR 创建 OpenCL 缓存对象，其目的是让 API 无需分配自己的内存，就能提供自身的指针。此外由于计算数据仅作为读取不做写入操作，因此通过 CL_MEM_READ_ONLY 设置为只读。而返回数据仅作为写入，通过 CL_MEM_WRITE_ONLY 设置为只写。通过内核参数设置输入程序运行时需要传递的内核参数。完成之后需要将内核本身添加到命令队列，以便开始执行，等待内核执行完成后还要将数据从加速卡传回主机端，最终与 CPU 的输出结果进行对比。基于 OpenCL 的量化金融加速系统主机端设计流程图由图 6-6 所示。

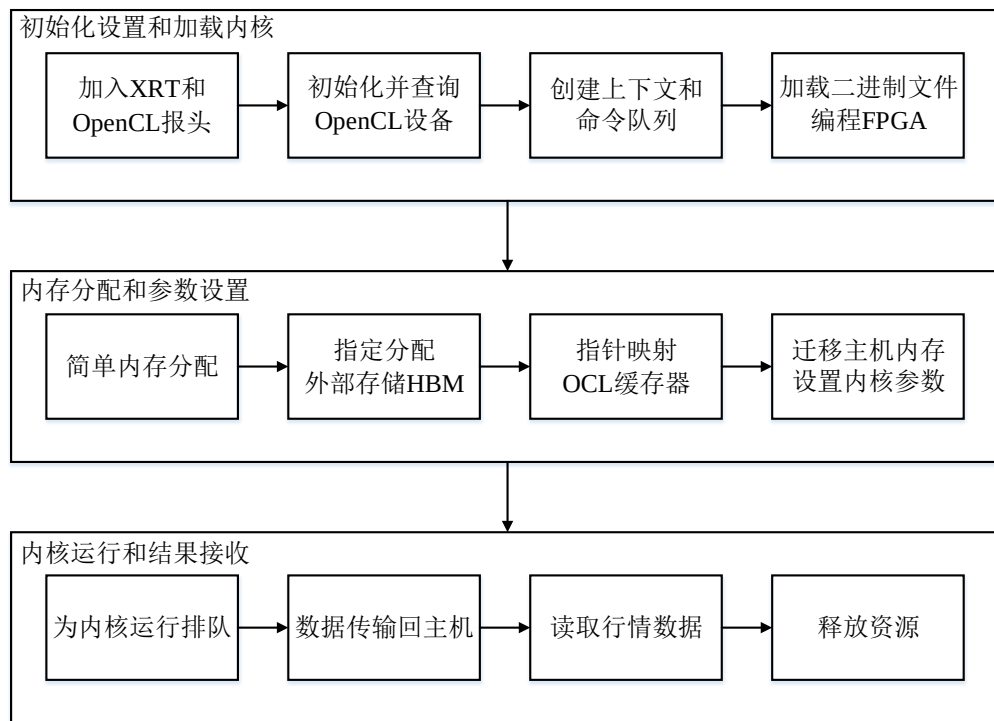


图 6-6 基于 OpenCL 的量化金融加速系统主机端设计流程图

6.2.3 基于 FPGA 的量化金融加速模块设计与实现

基于 FPGA 的金融加速模块由 FAST 协议解码模块和行情盘口回放模块构成。数据接收根据 STEP 协议格式判断 Tag35 对应的 Value 值判断接收数据的类型，因此在整个模块输入端接口上，与 FAST 解码模块的输入接口配置相同，确保数据在接口的传输速率上保持一致。通过筛选获得 UA5801 逐笔委托和 UA3201 逐笔成交的数据，按照 FAST 协议解码设计的优化流程处理数据最终获得完整可读的 FAST 各字段域解码。其后将 FAST 各字段解码结果输出作为行情盘口回放模块的输入进行计算。按照证券编号字段对数据进行分配，进入各自的处理单元进行计算。在单元计算中，按照订单时间域字段判断数据处理的先后，相同时间遵循委托在前，成交在后，最终生成盘口并输出。量化金融计算加速模块与行情盘口回放模块在输出接口配置上保持一致。此外，FAST 解码模块最终被分为两份，分别解码逐笔委托和逐笔成交，解码过程可并行进行。而行情盘口回放模块除整体的编号分配，还分为多个处理单元模块，处理单元直接互相独立，可

并行处理。处理单元的个数，为并行处理不同编号证券的并行度。量化金融计算加速模块框架图如图 6-7 所示。

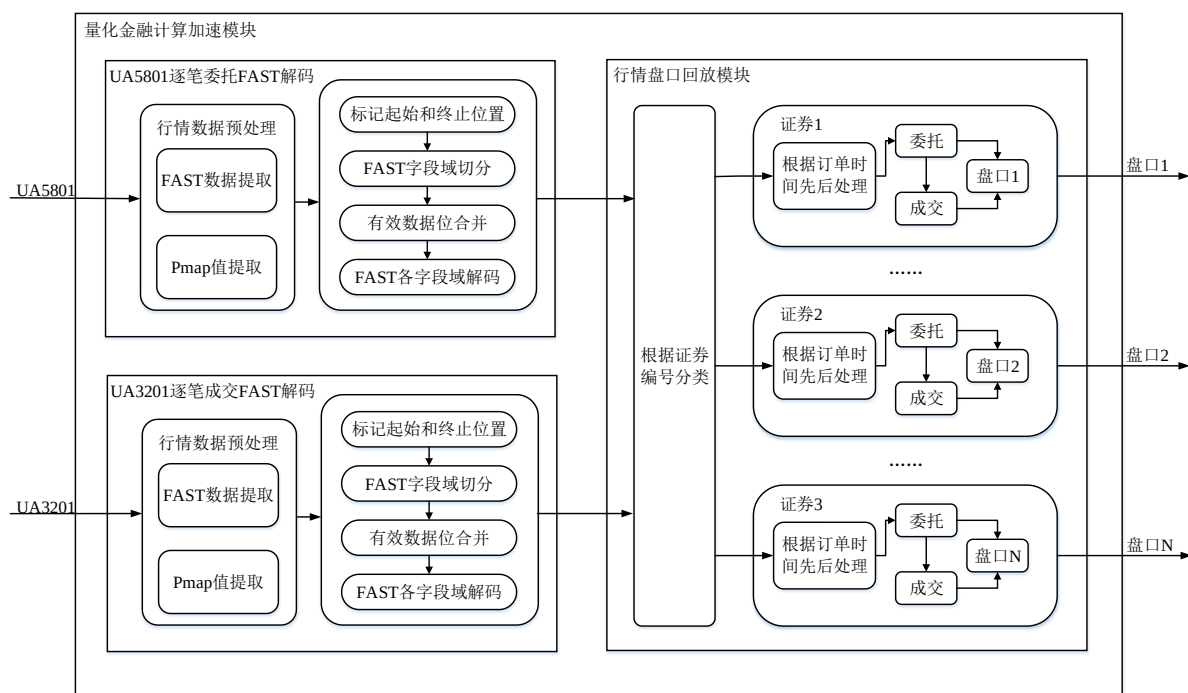


图 6-7 量化金融计算加速模块框架图

6.3 基于 FPGA 的金融计算加速系统实验结果与分析

6.3.1 实验数据来源和测试方法

实验数据来源于交易所 2020 年 3 月 19 日的 7 个不同市值的证券信息。逐笔委托交易数据时间为上午 9:15 至 11:30 和下午 13:00 至 15:00。逐笔成交和行情盘口快照时间为上午 9:25 至 11:30 和下午 13:00 至 15:00。测试的 7 种股票编号分别是 002951、000798、300130、300451、000895、300015、000333。峰值条数为 10ms 内需要处理的最大数据总条数。其中具体的数据参数信息如表 6-2 所示。

表 6-2 股票测试样本具体参数信息表

股票编号	市值（亿）	委托总条数	成交总条数	数据总条数	峰值条数
002951	10	5812	5072	10884	10
000798	20	2171	1868	4039	11
300130	50	16836	15433	32269	30
300451	100	29116	26386	55502	37
000895	500	56621	500993	107614	94
300015	1500	108368	100353	208721	200
000333	4700	122923	110789	233712	430

由上表可以看出，随着市值的增长，委托、成交的数据条数呈现递增的趋势，但是随着市值的跨越性增长，总体数据量趋于稳定。因此，更有利于对数据总体的有效宽度进行估量和预测。

6.3.2 量化金融计算加速系统延迟对比与分析

量化金融计算加速系统的整体测试包含网络通信、FAST 数据解码、行情数据解析三部分。其中网络通信采用通用的 TOE 模块测试和 UDP 模块测试，其中 TOE 的网络通信延迟为 495ns，UDP 为 470ns。而 FAST 数据解码延迟为 183ns，行情数据解析延迟为 190ns。整体系统延迟测试对比图如图 6-8 所示。

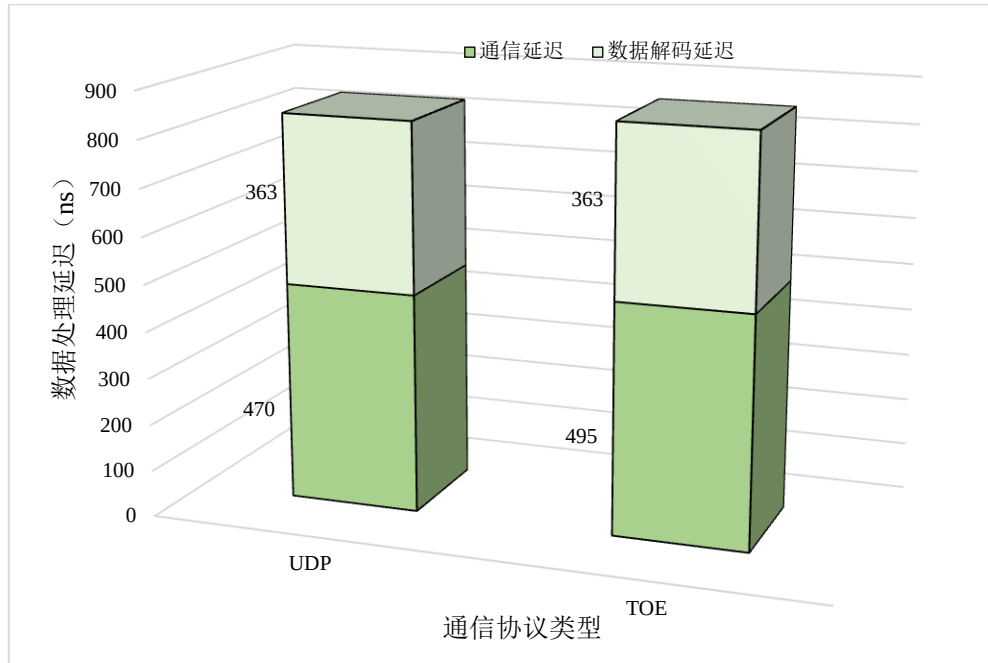


图 6-8 量化金融计算加速系统整体延迟占比对比图

由于 U50 加速卡的结构特性，其需要由主机端参与部分内存存储空间的读写和配置。同时，由于主机端与加速卡通过 PCIe 连接，因此，针对主机端和内核端对 HBM、DDR 的读写速率进行了测试。测试方法采用了 Xilinx 的 Xbtest 方案，采用相同大小为 256MB 的数据包进行读写测试。尽管 PC 在数据读写上不如 DDR，但是通过对于多个 PC，其能够实现叠加的数据带宽，从而提高读写速率。内存数据读写对比速率如表 6-3 所示。

表 6-3 内存数据读写速率对比

数据读取端	数据写入端	数据读取带宽	数据写入带宽
主机端	HBM	11940.3MB/s	11950.9MB/s
内核端	DDR	18.29GB/s	18.27GB/s
内核端	PC	14.28GB/s	14.29GB/s
内核端	四个 PC	55.83GB/s	56.87GB/s

6.3.3 金融计算模块 FPGA 资源消耗和分析

资源消耗方面由于行情盘口快照解析模块使用了 KVS 哈希存储用以存储价格和单号的信息以便查找。由于每一支股票都包含有其独立的哈希存储表，因此查找表资源的消耗会随着并行处理股票的增加而增加。同时由于部分子模块采用了流水架构，因此在模块与模块之间通过 FIFO 完成流水任务。此外，由于委托字典的存在，BRAM 的资源

消耗也随之增长，但是均只占用了部分资源。FPGA 硬件仍有更大的空间承担更多的并行任务。金融计算模块 FPGA 硬件资源消耗如表 6-4 所示。

表 6-4 金融计算模块 FPGA 硬件资源消耗使用

硬件资源	使用	可用	占比
BRAM	90	2688	6.7%
DSP	0	5952	0%
FF	110717	1743360	6.79%
LUT	89369	871680	10.25%

6.3.4 金融计算模块结果延迟对比与分析

由于整个测试时间较短，为了减少单条特殊数据对整个交易过程的影响偏差，通过测试固定时间间隔的行情数据进行总体测试。以 10ms 的数据为单位间隔。通过 OpenCL 完成从数据传输到量化金融计算解析返回行情盘口快照，并测试股票的行情盘口快照解析时延。通过计算处理同样数据信息的 CPU 进行对比。最终得到如图 6-9 所示的超低延迟量化金融计算加速模块延迟对比图。

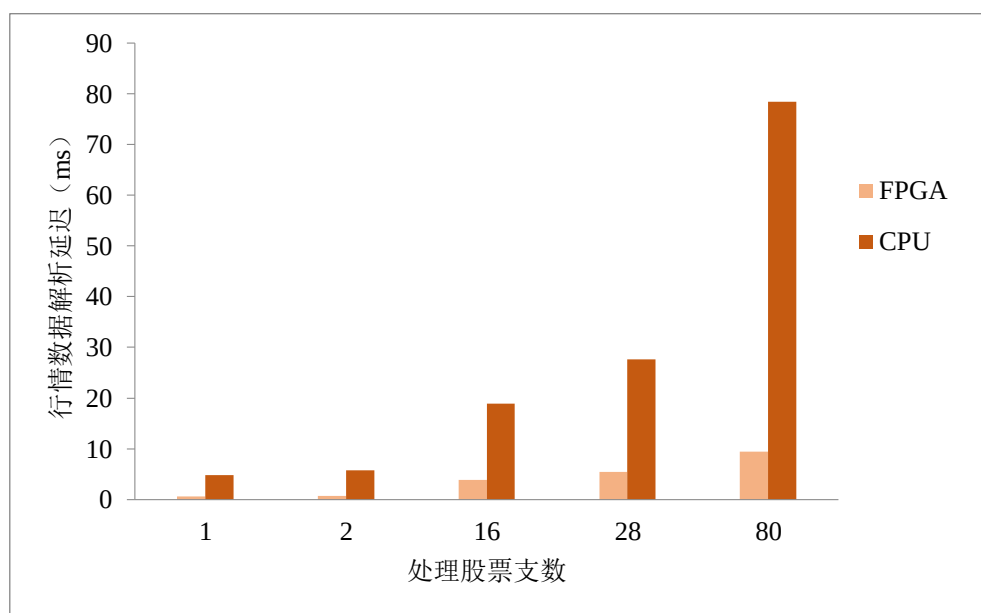


图 6-9 超低延迟量化金融计算加速模块延迟对比图

由图 6-9 可以看出，相比 CPU 处理数据延迟，对于单支或者小数量的股票并行，FPGA 并不存在过大的优势，但是随着股票数量的增加，FPGA 的延迟优势逐渐体现。在同时处理 80 支股票的情况下，FPGA 对于 10ms 时间间隔数据的处理平均延迟为 9.4ms，即可以在下一个 10ms 数据接收前，完成数据处理任务，从而避免因数据堆积导致的堵塞问题。相较而言 CPU 的处理由于其串行计算的特点，延迟消耗为 78.4ms。经过对比，FPGA 处理金融计算数据的速度为高性能 CPU 的 8 倍，在处理多支股票的情况下，优势更为明显。

6.4 本章小结

本章主要将四、五两章的协议解码和盘口解析合并为基于 FPGA 的量化金融计算模块，并与基于 FPGA 的网络协议卸载模块合并成整个金融计算加速系统。首先简要介绍了软硬件开发平台和加速卡的具体参数信息。通过上板处理单支不同类型的证券和多支证券的并行处理以及分析其硬件资源消耗来测试金融计算加速系统的整体性能。实验结果表明，无论是单只股票还是多支并行处理，基于 FPGA 的金融计算系统表现都优于传统的 CPU。

第七章 总结与展望

7.1 课题研究总结

本文采用了一种基于 OpenCL 和 HLS 的 FPGA 量化金融计算方法, 在实现超低延迟、高灵活性、可拓展的基础上, 解决了纯软件解码计算延迟高、FPGA 硬件解码计算开发周期长、更新困难的问题, 最终获得了良好的效果, 由此证明了本文的思路和方法具有一定的可行性。因此, 本文的工作有望推动量化金融计算领域的进展。

针对本文采用的设计实现方法, 总结其优势如下:

(1) 在量化金融计算的两个关键阶段, 即交易所到行情商的 FAST 数据解码和从行情商到证券投资者的行情盘口快照解析, 通过本文的设计方法分别实现了基于 FPGA 的 FAST 硬件解码器和基于 FPGA 的行情数据解析器, 并将其与网络协议卸载模块共同构成了量化金融计算加速系统, 发挥了 FPGA 并行流水以及运行时间可以精确到周期的特点, 实现了两部分数据解析和量化金融计算系统的低延迟低抖动。

(2) 由于 FPGA 具有可重构的特点, 因此其在协议更新、数据解析规则改变的情况下, 能够更有针对性地对硬件逻辑进行改变, 从而在适应量化金融计算过程中规则的频繁更新迭代方面更具灵活性。

(3) 其较于传统的纯 FPGA 开发, 基于 OpenCL 和 HLS 的 FPGA 开发模式能够使开发人员无需过分关注底层硬件接口配置和内存分配及管理重复造轮子的工作, 而只需关注主业务本身。因此, 可以说, 基于 OpenCL 和 HLS 的 FPGA 量化金融计算方法能够极大地缩短了开发周期, 从而更好地适应瞬息万变的量化金融交易市场。

存在的不足和限制:

(1) 相较于传统的 FPGA 开发, 基于 OpenCL 和 HLS 的 FPGA 开发, 尽管在开发周期上具有明显的缩短时间的优势, 但其在延迟精度上会略有损失。因此, 这一点对易受延迟因素影响的部分敏感交易过程来说, 容易产生误差, 从而造成一定的损失。

(2) 由于采用了 OpenCL, 同时采用的数据中心加速卡为纯 FPGA 板卡, 其需要与 CPU 构成异构计算平台, 并通过 PCIe 完成数据和指令的信息传输, 因此, 这种交互方式在 I/O 口的数据传输延迟会有所损耗。

7.2 展望

目前, AMD Xilinx 推出了针对数据中心的专用 FPGA 加速卡—Alveo 系列, 从早些年推出的 U280、U250、U200、U50 加速卡到近些年新推出的 U25、SN1000、U55C 加速卡。为解决目前高性能计算 (High Performance Computing, HPC) 工作负荷等多个关键性瓶颈, Alveo 系列为 HPC 和数据库工作的开发提供了更为先进的解决方案。具体表现在, 其在内存上扩大了 FPGA 的存储空间, 从而提供了更大的数据缓存, 能够为后续的复杂决策计算算法设计、大数据分析以及 AI 应用提供硬件支持。针对网络传输, 通过利用新的数据中心桥接技术和高速网络带宽以及专用的网络通用 IP 设计框架, 能够使得开发人员无需过分关注基础的网络架构, 而只需将更多的精力放在金融数据处理

的加速优化以及交易决策的指令和算法设计上。在此背景之下，更多不具备专业硬件知识的软件开发者和算法科学家亦能够采用加速卡完成算法设计上的加速计算。

本文主要是针对量化金融计算的通用先前方法进行了加速优化设计与实现。其中涉及了数据的解码和基本数据行情盘口的回放，不过其仅作为量化金融计算里一个通用的数据处理环节，实际的行情业务远远不止于此。通过实验也可以看出，上述的加速设计所占的资源仅占总硬件的一部分资源，因此，FPGA 依旧存在更大的空间去进行更为复杂的计算任务处理。随着人工智能、大数据等技术的兴起，Xilinx 专为加速卡系列定制了 Vitis AI 的人工智能算法加速平台，通过大数据预处理、分布式计算如 K-Core 算法以及数值统计算法如蒙特卡罗算法帮助进行硬件加速证券数据分析和交易决策制定将成为一个新的发展方向。

此外，从金融投资者的需求计算上来看，针对处理完的盘口数据，对其进行数据分析并制定相关的交易决策，从而实现软硬件结合的 FPGA 加速量化交易有望成为未来金融领域重要的发展方向之一。

致 谢

时光荏苒、白驹过隙，转眼间我已经成为了即将毕业的一名硕士三年级大学生了。回首这三年的时光，感触良多，从一开始的懵懂和对研究生生活的憧憬，再到之后通过不断的学习专业知识，并将其运用到横向课题中，将自己所学运用到实际生活中，最后通过提炼总结完成纵向论文的提炼和总结。这三年的成长过程历历在目，都将成为我人生中宝贵难忘的一段记忆。在此我想感谢三年来对我不断谆谆教导的老师、团结友爱互帮互助的同学们、在背后默默支持鼓励我的家人朋友们，并对他们致以真挚的祝福！

首先，我想感谢的是提供我科研平台和学习环境的母校—江南大学。感谢你给我提供的温馨且舒适的学习科研环境。实验室也提供了所需的实验设备和学术氛围，母校的校训也时刻激励着我，敦促我不断前行。在此离别之际，由衷的感谢母校三年来的关怀与培养，希望在不远的未来能够人才辈出，创造更好的辉煌。

其次，我要感谢的是在研究生生涯路上谆谆教导、和蔼可亲的两位老师-张曦煌教授和柴志雷教授。周丹平老师，感谢你们，不辞辛苦的培养着我，带我从迷茫懵懂到后来明确了自己的研究方向。在生活中，是你们教会了我们做人处事的道理。在科研上，是你们教会我要求真务实，脚踏实地。在面对课题进展不下去的时候，是你们和我促膝长谈，尽管生活和科研都很忙碌的你们，都会百忙之中抽空开展一周一次的例会，询问进展并给我们提出宝贵的建议。在论文写作上，是你们不厌其烦，一遍又一遍的指导我进行修改和理清思路。此外，你们多次教导我研究生最重要的不是知识本身的掌握，而是独立自主完成科研任务的能力。最后，你们还告诫我不要着眼于眼前和个人，要胸怀理想，着眼于祖国的发展，拥有一个浓烈的爱国之心。这些人生哲理让我受益匪浅，永世难忘。感谢所有教导过我的老师们，感谢你们的倾囊相授，让我在知识和人格上都不断变得丰满。我会谨记老师们的教诲和恩情，在未来的生活、工作中不断迎接各种挑战。

再者，我要感谢的是这三年来与我一同并肩作战的同窗们。你们来自五湖四海，怀着一个求学之心汇聚于此，在生活上对我的关怀和帮助，在学习上互相研讨和鼓励。三年转眼已到离别之际，但却给我留下了深刻的印象。在此，我要特别感谢我的师兄师姐们张新伟、李康、袁鸣、吴子刚、李佩琪、刘家航、郁龚健和吴健凤，祝你们工作学业顺利！感谢我的同窗们程雅贤、华夏、李慧琳、谢双镔、孙瑞鑫、奚智雯、朱铮皓、蔡晶晶、聂一、冯一飞、郑博文、高昊晖、钟经纬、吴双、尹成龙。真心的祝福你们在未来的工作和学业中大展宏图、前程似锦。感谢我的舍友方俊超、丁佳俊、陈旭志。感谢你们三年来在我生活和学习上的包容。此外，还想感谢我的师弟们赵海旭、樊荣、徐聪、王腾、叶俊超、缪永杰、刘爽等。最后，我想感谢一直以来默默支持我的家人朋友们，让我心无后顾之忧，一往无前。在未来，我也会依然努力拼搏，不辜负你们的期望！

最后，感谢百忙之中抽空审阅我论文的专家教授们，感谢你们的宝贵意见让我的论文更加完善。感谢答辩组的老师们抽出宝贵的时间来参加我的论文答辩并给予指导。在这里，我再次向各位老师、家人朋友们、同学们以及江南大学全体工作人员致以最真挚的祝福和最崇高的敬意！

参考文献

- [1] Wang J, Yang Q, Jin Z, et al. Research on quantitative trading strategy based on LSTM[C]. 2020 Asia-Pacific Conference on Image Processing, Electronics and Computers (IPEC). Dalian, China: IEEE, 2020: 266–270.
- [2] Banks S, Beadling P, Ferencz A. FPGA Implementation of Pseudo Random Number Generators for Monte Carlo Methods in Quantitative Finance[C]. 2008 International Conference on Reconfigurable Computing and FPGAs. Cancun, Mexico: IEEE, 2008: 271–276.
- [3] 北京证券交易所正式鸣锣开市[J]. 黑龙江金融, 2021(11): 7.
- [4] Acharya A, Sidnal N S. High Frequency Trading with Complex Event Processing[C]. 2016 IEEE 23rd International Conference on High Performance Computing Workshops (HiPCW). Hyderabad, India: IEEE, 2016: 39–42.
- [5] Gasanov I, Ereshko A. Overview of the Software Complex for High-Frequency Exchange Trading on the Stock Market[C]. 2021 14th International Conference Management of large-scale system development (MLSD). Moscow, Russian Federation: IEEE, 2021: 1–4.
- [6] Jia X, Keung Lau R Y. The Control Strategies for High Frequency Algorithmic Trading[C]. 2018 IEEE 4th International Conference on Control Science and Systems Engineering (ICCSSE). Wuhan, China: IEEE, 2018: 49–52.
- [7] Trading at the Speed of Light: How Ultrafast Algorithms Are Transforming Financial Markets[EB/OL]. <https://ieeexplore.ieee.org/document/9647710>. 2022-04-08.
- [8] Kohda S, Yoshida K. Characteristics of High-Frequency Trading and Its Forecasts[C]. 2021 IEEE 45th Annual Computers, Software, and Applications Conference (COMPSAC). Madrid, Spain: IEEE, 2021: 1496–1501.
- [9] Zoican S, Vochin M. Computing system and network architectures in high frequency trading financial applications[C]. 2016 International Conference on Communications (COMM). Bucharest, Romania: IEEE, 2016: 139–144.
- [10] Deng Y, Kong Y, Bao F, et al. Sparse Coding-Inspired Optimal Trading System for HFT Industry[J]. IEEE Transactions on Industrial Informatics, 2015, 11(2): 467–475.
- [11] Ye A, Chinthalapati V L R, Serguieva A, et al. Developing sustainable trading strategies using directional changes with high frequency data[C]. 2017 IEEE International Conference on Big Data (Big Data). Boston, MA, USA: IEEE, 2017: 4265–4271.
- [12] Chu C C F, Chan P K. Mining Profitable High Frequency Pairs Trading Forex Signal Using Copula and Deep Neural Network[C]. 2018 19th IEEE/ACIS International Conference on Software Engineering, Artificial Intelligence, Networking and Parallel/Distributed Computing (SNPD). Busan, Korea (South): IEEE, 2018: 312–316.
- [13] 王疏. 国内证券高速实时行情平台设计与实现[D]. 硕士毕业论文. 上海: 上海交通大学, 2017.
- [14] 上海证券交易所. 上海证券交易所股票成交年度概况[EB/OL]. <http://www.sse.com.cn/market/stock/data/overview/yearly>. 2022-4-10.
- [15] 深圳证券交易所. 深圳证券交易所股票成交年度概况[EB/OL]. <http://www.szse.cn/market/stock/in-dicator>. 2022-4-10.
- [16] 孙励, 万勇, 杨文婕. 基于神经网络的期货高频交易连续性“羊群行为”研究——以中国豆粕期货为例[J]. 广东石油化工学院学报, 2021, 31(06): 81–85.
- [17] 张思玉. 期货交易模型及算法设计与实现[J]. 电脑编程技巧与维护, 2021(10): 49–51.
- [18] 夏丙英, 蒋文保. 程序化交易安全风险及其分析[J]. 信息安全与技术, 2013, 4(11): 31–34.
- [19] 王力文. 基于泛化的套利交易系统的设计与实现[J]. 计算机科学, 2017, 44(S1): 529–533.

- [20] 黄秋波, 陈智, 苏厚勤. 证券交易复杂事件处理策略管理服务模块的设计与实现[J]. 计算机应用与软件, 2015, 32(04): 91-95+229.
- [21] dale-wilson.et al.quickfast [EB/OL].<https://github.com/objectcomputing/quickfast>.2017-3-15.
- [22] huangminghuang.et al. mFAST v1.2.2[EB/OL].<https://github.com/objectcomputing/mFAST>.2021-4-9.
- [23] rafmudaf.et al.Openfast.[EB/OL]. <https://github.com/OpenFAST/openfast>.2022-3-3
- [24] 徐广斌, 武剑锋, 王泊, 等. 基于扩展 FAST 协议的金融消息压缩方法[J]. 计算机工程与应用, 2012, 48(01): 138–141.
- [25] Li H, Fu Y, Liu T, et al. Fast Protocol Decoding in Parallel with FPGA Hardware[C]. 2014 IEEE 17th International Conference on Computational Science and Engineering. Chengdu, China: IEEE, 2014: 1669–1673.
- [26] 环宇翔, 崔建军, 郑立荣, 等. 基于 FPGA 的超低延迟硬件加速行情解析系统[J]. 交易技术前沿, 2020 年, 第四期: 37-44.
- [27] 金亭姝, 马辉. 上交所 level-2 行情 OpenCL 平台硬件解码[J]. 交易技术前沿, 2020 年, 第四期: 45-50.
- [28] Dvořák M, Kořenek J. Low latency book handling in FPGA for high frequency trading[C]. 17th International Symposium on Design and Diagnostics of Electronic Circuits Systems. Warsaw, Poland: IEEE, 2014: 175–178.
- [29] Puranik S, Barve M, Shah D, et al. Key-Value Store using High Level Synthesis Flow for Securities Trading System[C]. 2020 International Conference on Computing, Electronics Communications Engineering (iCCECE). Southend, UK: IEEE, 2020: 237–242.
- [30] Leber C, Geib B, Litz H. High Frequency Trading Acceleration Using FPGAs[C]. 2011 21st International Conference on Field Programmable Logic and Applications. Chania, Greece: IEEE, 2011: 317–322.
- [31] Boutros A, Grady B, Abbas M, et al. Build fast, trade fast: FPGA-based high-frequency trading using high-level synthesis[C]. 2017 International Conference on ReConFigurable Computing and FPGAs (ReConFig). Cancun, Mexico: IEEE, 2017: 1–6.
- [32] 张延彬, 张凤麒, 王忠勇. 基于 FPGA 的期货行情数据并行处理设计[J]. 计算机工程与设计, 2019, 40(07): 1866–1871.
- [33] 张凤麒, 张延彬, 王忠勇. 基于 FPGA 的 DDR3 六通道读写防冲突设计[J]. 电子技术应用, 2018, 44(07): 68-71+80.
- [34] 王艳秋. 基于 FIX 协议的多版本间数据转换的设计与实现[J]. 信息与电脑(理论版), 2014(04): 111–112.
- [35] 王常林. 基于 FIX 协议的网上证券交易系统的设计与实现[D]. 硕士毕业论文. 江苏: 江苏科技大学, 2010.
- [36] 邓少灵. 金融信息交换协议 FIX[J]. 计算机应用研究, 2002(12): 8-9+13.
- [37] 中国证券监督管理委员会. 证券交易数据交换协议. JR/T 0022-2014[S]. 北京: 中国证监会. 2014.
- [38] 上海证券交易所. 上海证券交易所交易网关 STEP 接口规格说明书 V1.00[EB/OL]. http://www.sse.com.cn/services/tradingtech/data/c/IS122_IITP_TDGW_STEP_CV1.00_20220228.pdf, 2022-02-28
- [39] 刘成德. 基于 STEP 协议的证券行情分析系统 (LEVEL-2) 的研究与实现[D]. 硕士毕业论文. 贵州: 贵州大学, 2008.
- [40] FAST-1.1 规范中文版全文参考.[EB/OL]. <https://wenku.baidu.com/view/29cd325ac67da26925c52cc58bd63186bceb92a1.html>, 2006-12-20.
- [41] Sunkavilli S, Zhang Z, Yu Q. New Security Threats on FPGAs: From FPGA Design Tools Perspective[C]. 2021 IEEE Computer Society Annual Symposium on VLSI (ISVLSI). Tampa, FL, USA: IEEE, 2021: 278–283.

- [42] Ji Y, Ji Y. Hardware Implementation Method of LDPC Efficient Decoding Based on FPGA[C]. 2021 7th Annual International Conference on Network and Information Systems for Computers (ICNISC). Guiyang, China: IEEE, 2021: 48–53.
- [43] Imaña J L. Low-Delay FPGA-Based Implementation of Finite Field Multipliers[J]. IEEE Transactions on Circuits and Systems II: Express Briefs, 2021, 68(8): 2952–2956.
- [44] Zou Y, Lin M. FERMAT: FPGA-Accelerated Heterogeneous Computing Platform Near NVMe Storage[C]. 2021 IEEE 29th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM). Orlando, FL, USA: IEEE, 2021: 262–262.
- [45] de Haro J M, Bosch J, Filgueras A, et al. OmpSs@FPGA Framework for High Performance FPGA Computing[J]. IEEE Transactions on Computers, 2021, 70(12): 2029–2042.
- [46] Sung G-M, Tung L-F, Wang H-K, et al. USB Transceiver With a Serial Interface Engine and FIFO Queue for Efficient FPGA-to-FPGA Communication[J]. IEEE Access, 2020, 8: 69788–69799.
- [47] Yang J, Guo H, Chen L, et al. SEU sensitivity evaluation of JTAG circuit used for SRAM-based FPGA[C]. 2017 IEEE 24th International Symposium on the Physical and Failure Analysis of Integrated Circuits (IPFA). Chengdu, China: IEEE, 2017: 1–5.
- [48] Bauer F, Braun F, Hauer D, et al. MELODI: An Online Platform for Mass Education of Digital Design - HDL to Remote FPGA[C]. 2021 31st International Conference on Field-Programmable Logic and Applications (FPL). Dresden, Germany: IEEE, 2021: 399–399.
- [49] Gouert C, Tsoutsos N G. Romeo: Conversion and Evaluation of HDL Designs in the Encrypted Domain[C]. 2020 57th ACM/IEEE Design Automation Conference (DAC). San Francisco, CA, USA: IEEE, 2020: 1–6.
- [50] Zhang Z, Li Z. ARM and FPGA Heterogeneous Accelerated Processing System Based on HLS and PCIe[C]. 2021 4th International Conference on Information and Computer Technologies (ICICT). HI, USA: IEEE, 2021: 300–304.
- [51] Lee H S, Jeon J W. Comparison Between HLS and HDL Image Processing in FPGAs[C]. 2020 IEEE International Conference on Consumer Electronics - Asia (ICCE-Asia). Seoul, Korea (South): IEEE, 2020: 1–2.
- [52] Thomas D B. Compile-Time Generation of Custom-Precision Floating-Point IP using HLS Tools[C]. 2019 IEEE 26th Symposium on Computer Arithmetic (ARITH). Kyoto, Japan: IEEE, 2019: 192–193.
- [53] Wang C, Huang S, Hwu W-M, et al. Extending HLS with High-Level Descriptive Language for Configurable Algorithm-Level Spatial Structure Design[C]. 2021 IEEE 29th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM). Orlando, FL, USA: IEEE, 2021: 261–261.
- [54] Choi Y-K, Chi Y, Wang J, et al. FLASH: Fast, Parallel, and Accurate Simulator for HLS[J]. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 2020, 39(12): 4828–4841.
- [55] Zhao J, Zhao Y, Li H, et al. HLS-Based FPGA Implementation of Convolutional Deep Belief Network for Signal Modulation Recognition[C]. IGARSS 2020 - 2020 IEEE International Geoscience and Remote Sensing Symposium. Waikoloa, HI, USA: IEEE, 2020: 6985–6988.
- [56] Zhou X, Yu Z J, Cao Y, et al. SAR Imaging Realization with FPGA Based on VIVADO HLS[C]. 2019 IEEE International Conference on Signal, Information and Data Processing (ICSIDP). Chongqing, China: IEEE, 2019: 1–4.
- [57] Ashcraft M B, Goeters J. Unified On-Chip Software and Hardware Debug for HLS-Accelerated Programs[C]. 2018 International Conference on Field-Programmable Technology (FPT). Naha, Japan: IEEE, 2018: 354–357.
- [58] Xilinx. Vitis High Level Synthesis User Guide(UG1399)[EB/OL]. <https://docs.xilinx.com/r/en-US/ug1399-vitis-hls>, 2021-12-15.

- [59] Xilinx.Vitis Unified Software Platform Documentation: Application Acceleration Development UG1393[EB/OL].<https://docs.xilinx.com/r/en-US/ug1393-vitis-application-acceleration/Getting-Started-with-Vitis>,2021-12-15
- [60] Mousoulitis P, Zogas S, Christakos P, et al. Exploiting Vitis Framework for Accelerating Sobel Algorithm[C]. 2021 10th Mediterranean Conference on Embedded Computing (MECO). Budva, Montenegro: IEEE,2021: 1–5.
- [61] Brown N. Weighing Up the New Kid on the Block: Impressions of using Vitis for HPC Software Development[C]. 2020 30th International Conference on Field-Programmable Logic and Applications (FPL). Gothenburg, Sweden: IEEE, 2020: 335–340.
- [62] Xilinx.Xilinx Runtime (XRT) Architecture [EB/OL]. <https://xilinx.github.io/XRT/master/html/index.html>,2022-3-29.
- [63] XilinxVitis 统一软件平台面向所有开发者解锁全新设计体验[J]. 单片机与嵌入式系统应用, 2019, 19(12): 94.
- [64] Korobeynikov A. Effective Implementation of "Kuznyechik" Block Cipher on FPGA with OpenCL Platform[C]. 2019 IEEE Conference of Russian Young Researchers in Electrical and Electronic Engineering (EIconRus). Saint Petersburg and Moscow, Russia: IEEE,2019: 1683–1686.
- [65] Yang A, Li Y, Shu H, et al. An OpenCL-Based FPGA Accelerator for Compressed YOLOv2[C]. 2019 International Conference on Field-Programmable Technology (ICFPT). Tianjin, China:IEEE,2019: 235–238.
- [66] Bai Z, Fan H, Liu L, et al. An OpenCL-Based FPGA Accelerator with the Winograd's Minimal Filtering Algorithm for Convolution Neuron Networks[C]. 2019 IEEE 5th International Conference on Computer and Communications (ICCC). Chengdu, China: IEEE,2019: 277–282.
- [67] Purkayastha A A, Raghavendran S, Thiagarajan J, et al. Exploring the Efficiency of OpenCL Pipe for Hiding Memory Latency on Cloud FPGAs[C]. 2019 IEEE High Performance Extreme Computing Conference (HPEC). Waltham, MA, USA: IEEE,2019: 1–7.
- [68] Jin Z, Finkel H. OpenCL Kernel Vectorization on the CPU, GPU, and FPGA: A Case Study with Frequent Pattern Compression[C]. 2019 IEEE 27th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM). San Diego, CA, USA: IEEE,2019: 330–330.
- [69] Koshiba A, Sakamoto R, Namiki M. OpenCL Runtime for OS-Driven Task Pipelining on Heterogeneous Accelerators[C]. 2018 IEEE 24th International Conference on Embedded and Real-Time Computing Systems and Applications (RTCSA). Hakodate, Japan: IEEE, 2018: 236–237.
- [70] Paulino N, Ferreira J C, Cardoso J M P. Optimizing OpenCL Code for Performance on FPGA: k-Means Case Study With Integer Data Sets[J]. IEEE Access, 2020, 8: 152286–152304.
- [71] Bensalem H, Blaqui ère Y, Savaria Y. Toward In-System Monitoring of OpenCL-Based Designs on FPGA[C]. 2019 IEEE International Symposium on Circuits and Systems (ISCAS). Sapporo, Japan: IEEE,2019: 1–5.
- [72] 深圳证券交易所.深圳证券交易所 Binary 行情数据接口规范(Ver1.11).[EB/OL]. <http://www.szse.cn/www/marketServices/technicalservice/interface/P020211013598379891667.pdf>,2021-08-01.
- [73] 上海证券交易所.上海证券交易所 LDDS 系统竞价 Level-2 行情接口说明书 V2.0.5[EB/OL].<https://www.sseinfo.com/services/assortment/document/interface/c/5659552.pdf>,2021-09-24.
- [74] 上海证券交易所.上海证券交易所 Binary 接口规格说明书 V0.50[EB/OL]. http://www.sse.com.cn/services/tradingservice/tradingtech/technical/development/c/IS122_TDGW_BINRY_V0.50_MTP_Test_20211029.pdf,2021-09-24.
- [75] 上海证券交易所.上海证券交易所 LDDS 系统 VDE 部署使用手册 V1.1.0[EB/OL]. <https://www.sseinfo.com/services/assortment/document/additions/c/5683973.pdf>,2022-01-13.

- [76] Xilinx.ALveo U50 数据中心加速器卡数据手册 DS965 V1.5[EB/OL]. https://china.xilinx.com/content/dam/xilinx/support/documents/data_sheets/c_ds965-u50.pdf,2020-02-18.
- [77] Xilinx.ALveo 加速卡开发白皮书 UG1352 V1.1[EB/OL]. https://china.xilinx.com/content/dam/xilinx/support/documents/data_sheets/c_ds965-u50.pdf,2019-10-23.

附录：作者在攻读硕士期间发表的论文、成果和科研项目

发表论文情况：

- [1] 张曦煌, 丁楠, 柴志雷, 冯一飞, 叶钧超. 低延迟低抖动的 FAST 解码器设计与实现.[J]. 应用科学学报 (CSCD 拓展, 已录用)
- [2] 丁楠, 柴志雷, 高昊晖, 冯一飞, 张曦煌. 低延迟抖动的证券行情数据解析系统设计与实现.[J]. 计算机与数字工程 (已录用)
- [3] 冯一飞, 丁楠, 叶钧超, 柴志雷. 领域专用低延迟高带宽 TCP / IP 引擎设计与实现[J]. 计算机工程 (CSCD 拓展, 已录用)
- [4] 冯一飞, 丁楠, 叶钧超, 柴志雷. 基于 OpenCL 的超低延迟行情加速系统设计与实现. [J]. 计算机应用与软件 (已录用)

获奖情况：

第四届全国大学生集成电路创新创业大赛

华东赛区三等奖